Programação Orientada a Objetos em Java

Material de estudo para a Prova 2: 10 exercícios e 5 provas simuladas

Conteúdo coberto: Herança (Aula 05), Interfaces, Polimorfismo e Tipos Genéricos (Aula 06), Coleções e Mapas (Aula 07) e, com peso menor, Exceções.

Como usar este material

1. Faça os **10 exercícios** da Parte I em ordem. A dificuldade cresce de um para o outro.
2. Resolva cada **Prova Simulada** da Parte II em um projeto novo do IntelliJ, no tempo de uma prova real (as provas de referência tinham cerca de 3h30). Não consulte os gabaritos antes.
3. Só então compare com os **gabaritos comentados** (Partes III e IV). Todos os códigos dos gabaritos foram compilados (Java 17+) e executados; as saídas mostradas nos enunciados vêm dessas execuções.
4. Termine pela **síntese de revisão** (Parte V).

O que as duas provas de referência mostram

Analisei as provas **Mário (padrão State)** e **Urna eletrônica**. Em comum, elas têm:

- Um **contexto narrativo** seguido de **diagrama UML** e de uma **tabela numerada de itens com pontos** somando 10,0. Os itens seguem a ordem natural de construção: criar pacotes e classes, depois interface, depois lógica, depois `Principal`.
- Cobrança de **herança, interface e polimorfismo** de forma funcional (delegar ao objeto de estado; `UrnaImpl implements Urna`), **Collections** como parte essencial (mapas em vez de listas para milhares de candidatos; lista de eventos; `Comparable` com desempates), `equals`/`hashCode`, `enum`, **classe utilitária não instanciável**, `StringBuilder`, formatação de saída e uma **exceção não verificada** própria.
- **Quadros** com formato exato de saída, **penalidades** (-1,0) e a regra "nada de imprimir fora de `Principal`" nas classes de domínio.

As cinco provas simuladas seguem esse molde (11 a 13 itens, soma 10,0, UML, quadros, penalidades) e **nenhuma é mais simples que as referências**. Elas partem de problemas inéditos e crescem em integração de conceitos: a Prova 5 combina genéricos com limites, método-modelo `final`, construtores de pacote com fábrica, varargs, lambdas e streams.

> **Nota sobre números decimais.** Os exemplos de saída usam ponto decimal (`12.50`). Dependendo do *locale* do seu sistema, `String.format` e `printf` podem imprimir vírgula (`12,50`). Isso não é erro. Se quiser reproduzir exatamente, use `Locale.US` ou execute com `-Duser.language=en`.
>
> **Nota sobre a pontuação das provas.** As referências trazem pontuação; usei o mesmo estilo. A distribuição de pontos dos simulados é **uma sugestão minha**, não uma pontuação oficial do professor.
>
> **Sobre Exceções.** Você não tem os slides dessa aula. Por isso usei apenas o básico convencional: `try`/`catch`/`finally`, `throw`, `throws`, exceções verificadas e não verificadas, `IllegalArgumentException`, `NumberFormatException` e validação de argumentos. Exceções aparecem em poucos itens, sem dominar nenhuma prova.

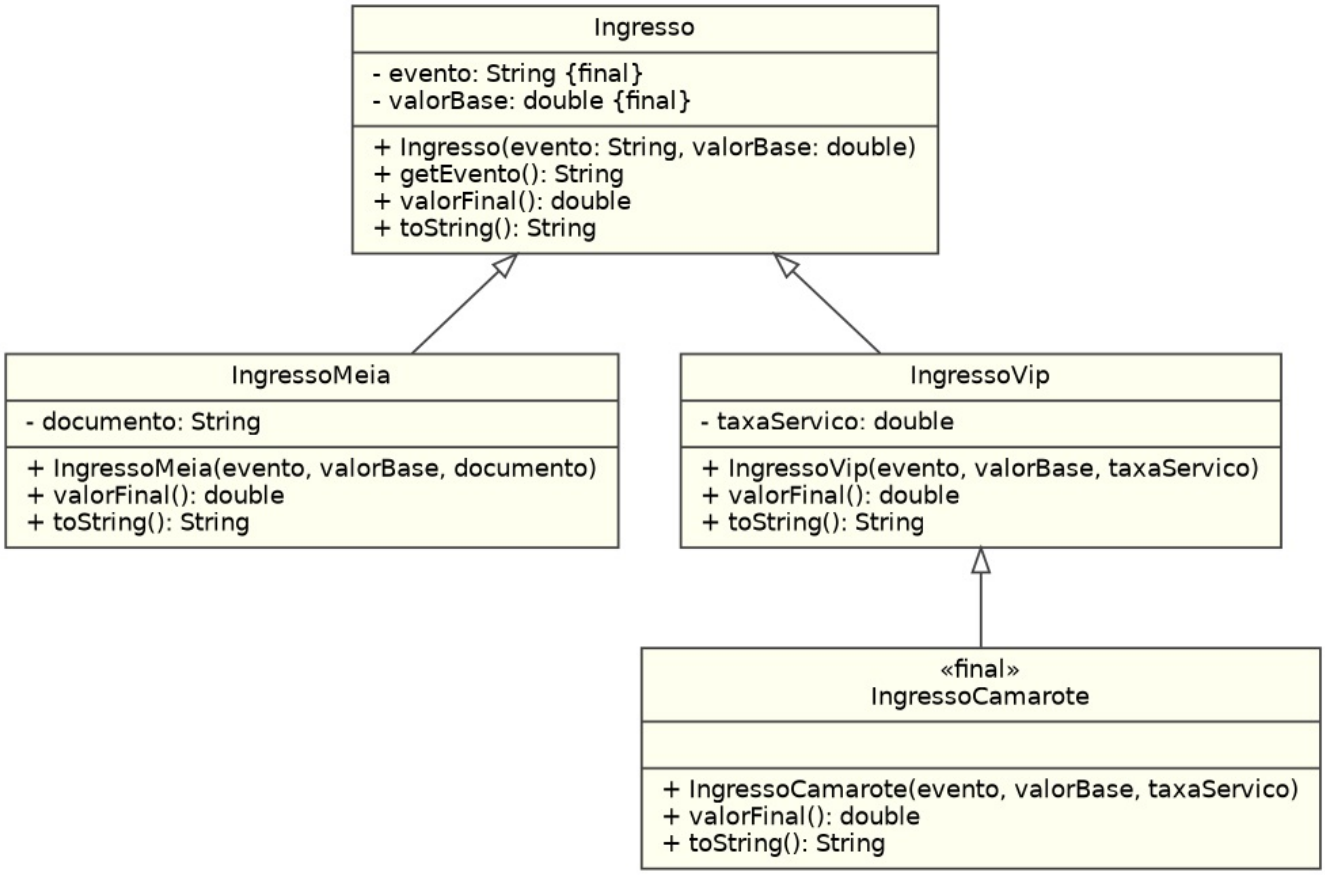
Parte I: Dez exercícios progressivos

Em todos os exercícios você deve **escrever ou modificar código Java**. Não há perguntas teóricas. Os campos "Conteúdos" servem só como referência de estudo. As soluções estão na Parte III.

Exercício 1: Ingressos de show

Dificuldade: ★☆☆☆☆ (fundamental) | **Conteúdos:** herança, `super`, `@Override`, `final`, `toString`

Uma casa de shows vende vários tipos de ingresso. Implemente a hierarquia abaixo, com atributos privados.



Regras

- `Ingresso` é concreta. `evento` e `valorBase` são imutáveis. `getEvento()` **não pode ser sobrescrito** por subclasses. `valorFinal()` devolve o `valorBase`.
- `IngressoMeia` paga **metade** do valor final do ingresso comum.
- `IngressoVip` soma ao valor final comum uma `taxaServico` fixa.
- `IngressoCamarote` **não pode ser herdada**. Seu valor final é o valor final do VIP **multiplicado por 1,2**, reaproveitando o cálculo da superclasse.
- Cada `toString()` das subclasses deve **reutilizar** o da superclasse, acrescentando o sufixo mostrado na saída esperada. O da superclasse formata o valor com duas casas decimais (use `Locale.US`).
- Nas subclasses, os construtores devem chamar o construtor da superclasse com `super`.

Faça o `Principal` abaixo funcionar e produzir a saída esperada.

```
import java.util.Locale;

public class Principal {
    public static void main(String[] args) {
        Ingresso[] ingressos = {
            new Ingresso("Rock in Rio", 300.0),
            new IngressoMeia("Rock in Rio", 300.0, "RG-1234"),
            new IngressoVip("Rock in Rio", 300.0, 50.0),
            new IngressoCamarote("Rock in Rio", 300.0, 50.0)
        };

        double total = 0;
        for (Ingresso ingresso : ingressos) {
            System.out.println(ingresso);
            System.out.println(String.format(Locale.US, "  valor final = %.2f", ingresso.valorFinal()));
            total += ingresso.valorFinal();
        }
        System.out.println(String.format(Locale.US, "Total = %.2f", total));
    }
}
```

Saída esperada

```
Ingresso[evento=Rock in Rio, valorBase=300.00]
  valor final = 300.00
Ingresso[evento=Rock in Rio, valorBase=300.00] [MEIA, doc=RG-1234]
  valor final = 150.00
Ingresso[evento=Rock in Rio, valorBase=300.00] [VIP, taxa=50.0]
  valor final = 350.00
Ingresso[evento=Rock in Rio, valorBase=300.00] [VIP, taxa=50.0] [CAMAROTE]
  valor final = 420.00
Total = 1220.00
```

Exercício 2: Central de notificações

Dificuldade: ★☆☆☆☆ (fundamental) | **Conteúdos:** interface, polimorfismo, varargs, classe anônima

Crie a interface `Notificador` com os métodos `String canal()` e `boolean enviar(String destinatario, String mensagem)`, que devolve se a mensagem foi enviada. Depois implemente:

- `NotificadorEmail` (canal `"EMAIL"`): só envia se o destinatário não for nulo e contiver `@`.
- `NotificadorSms` (canal `"SMS"`): só envia se o destinatário tiver **exatamente 11 caracteres, todos dígitos** (percorra a `String`).
- `NotificadorConsole` (canal `"CONSOLE"`): sempre envia.
- Ao enviar, cada um imprime uma linha no formato visto na saída esperada.
- `CentralNotificacoes`: construtor com **varargs** de `Notificador` (guarde uma **cópia** do vetor); `int enviarParaTodos(String destinatario, String mensagem)`, que devolve quantos canais conseguiram enviar; e `Notificador buscarCanal(String nome)`, que ignora maiúsculas/minúsculas e devolve `null` se não achar.

Faça o `Principal` abaixo funcionar. Ele cria um quarto canal (`PUSH`) por **classe anônima**.

```
public class Principal {
    public static void main(String[] args) {
        Notificador push = new Notificador() {
            @Override
            public String canal() {
                return "PUSH";
            }

            @Override
            public boolean enviar(String destinatario, String mensagem) {
                System.out.println("[PUSH] " + destinatario + ": " + mensagem);
                return !destinatario.isEmpty();
            }
        };

        CentralNotificacoes central = new CentralNotificacoes(
                new NotificadorEmail(), new NotificadorSms(), new NotificadorConsole(), push);

        int a = central.enviarParaTodos("ana@ifsp.edu.br", "Prova na terca");
        System.out.println("Canais que enviaram: " + a);

        int b = central.enviarParaTodos("16999998888", "Prova na terca");
        System.out.println("Canais que enviaram: " + b);

        Notificador sms = central.buscarCanal("sms");
        System.out.println("buscarCanal(sms) = " + sms.canal());
        System.out.println("buscarCanal(fax) = " + central.buscarCanal("fax"));
        System.out.println("SMS invalido? " + sms.enviar("123", "oi"));
    }
}
```

Saída esperada

```
[EMAIL] para ana@ifsp.edu.br: Prova na terca
[CONSOLE] ana@ifsp.edu.br <- Prova na terca
[PUSH] ana@ifsp.edu.br: Prova na terca
Canais que enviaram: 3
[SMS] para 16999998888: Prova na terca
[CONSOLE] 16999998888 <- Prova na terca
[PUSH] 16999998888: Prova na terca
Canais que enviaram: 3
buscarCanal(sms) = SMS
buscarCanal(fax) = null
SMS invalido? false
```

Exercício 3: Agenda de tarefas

Dificuldade: ★★☆☆☆ (básico a intermediário) | **Conteúdos:** `List`, `Comparable`, `Comparator`/lambda, `equals`/`hashCode`, iterador, encapsulamento de coleção

Complete as classes. `Tarefa` guarda título, prioridade (1 a 5), prazo (`LocalDate`) e se está concluída.

```
public class Tarefa implements Comparable<Tarefa> {
    // atributos privados; construtor Tarefa(String titulo, int prioridade, LocalDate prazo)
    // getters: getTitulo(), getPrioridade(), getPrazo(), isConcluida()
    // void concluir()
    // compareTo: 1) prazo mais antigo primeiro; 2) em empate, MAIOR prioridade primeiro;
    //            3) em empate, ordem alfabética do título
    // equals/hashCode: duas tarefas são a mesma se tiverem o mesmo título
    // toString: "Pagar boleto (p3, 2026-10-15)" e, se concluída, "Pagar boleto (p3, 2026-10-15, ok)"
}

public class Agenda {
    // a lista de tarefas NÃO pode ser exposta por nenhum getter
    public boolean adicionar(Tarefa tarefa)      // false se já existir tarefa com o mesmo título
    public boolean remover(String titulo)        // false se não existir; use um Iterator
    public boolean concluir(String titulo)       // false se não existir
    public Iterator<Tarefa> iterador()
    public List<Tarefa> ordenadas()              // CÓPIA ordenada pela ordem natural de Tarefa
    public List<Tarefa> porPrioridade()          // CÓPIA: prioridade desc., depois prazo asc. (use lambda)
    public List<Tarefa> atrasadas(LocalDate hoje)// não concluídas com prazo ANTES de hoje
}
```

A lista interna **não pode mudar de ordem** quando você chama `ordenadas()` ou `porPrioridade()`.

```
import java.time.LocalDate;
import java.util.Iterator;
import java.util.List;

public class Principal {
    public static void main(String[] args) {
        Agenda agenda = new Agenda();
        System.out.println(agenda.adicionar(new Tarefa("Estudar Collections", 5, LocalDate.of(2026, 10, 20))));
        System.out.println(agenda.adicionar(new Tarefa("Lavar o carro", 2, LocalDate.of(2026, 10, 18))));
        System.out.println(agenda.adicionar(new Tarefa("Entregar relatorio", 4, LocalDate.of(2026, 10, 18))));
        System.out.println(agenda.adicionar(new Tarefa("Pagar boleto", 3, LocalDate.of(2026, 10, 15))));
        System.out.println(agenda.adicionar(new Tarefa("Lavar o carro", 1, LocalDate.of(2026, 11, 1))));

        System.out.println("-- ordenadas (prazo, prioridade desc, titulo)");
        for (Tarefa t : agenda.ordenadas()) System.out.println(t);

        System.out.println("-- por prioridade");
        for (Tarefa t : agenda.porPrioridade()) System.out.println(t);

        agenda.concluir("Pagar boleto");
        List<Tarefa> atrasadas = agenda.atrasadas(LocalDate.of(2026, 10, 19));
        System.out.println("-- atrasadas em 2026-10-19: " + atrasadas);

        System.out.println("remover(Lavar o carro) = " + agenda.remover("Lavar o carro"));
        System.out.println("remover(Inexistente) = " + agenda.remover("Inexistente"));

        System.out.println("-- iterador");
        Iterator<Tarefa> it = agenda.iterador();
        while (it.hasNext()) System.out.println(it.next());
    }
}
```

Saída esperada

```
true
true
true
true
false
-- ordenadas (prazo, prioridade desc, titulo)
Pagar boleto (p3, 2026-10-15)
Entregar relatorio (p4, 2026-10-18)
Lavar o carro (p2, 2026-10-18)
Estudar Collections (p5, 2026-10-20)
-- por prioridade
Estudar Collections (p5, 2026-10-20)
Entregar relatorio (p4, 2026-10-18)
Pagar boleto (p3, 2026-10-15)
Lavar o carro (p2, 2026-10-18)
-- atrasadas em 2026-10-19: [Lavar o carro (p2, 2026-10-18), Entregar relatorio (p4, 2026-10-18)]
remover(Lavar o carro) = true
remover(Inexistente) = false
-- iterador
Estudar Collections (p5, 2026-10-20)
Entregar relatorio (p4, 2026-10-18)
Pagar boleto (p3, 2026-10-15, ok)
```

Exercício 4: Contas e comandos inválidos

Dificuldade: ★★☆☆☆ (básico a intermediário) | **Conteúdos:** exceção verificada própria, `throw`/`throws`, `try`/`catch`/`finally`, `NumberFormatException`

1. Crie `SaldoInsuficienteException`, **verificada**, com os atributos `saldoAtual` e `valorSolicitado` (e getters). A mensagem deve ser `Saldo insuficiente: saldo=<saldo>, solicitado=<valor>`.

2. Crie `Conta` (titular e saldo). O construtor lança `IllegalArgumentException` se o titular for nulo ou em branco, ou se o saldo inicial for negativo.

    - `depositar(double)` lança `IllegalArgumentException` se o valor for ≤ 0.
    - `sacar(double)` lança `IllegalArgumentException` se o valor for ≤ 0 e `SaldoInsuficienteException` se exceder o saldo.
    - `transferir(Conta destino, double valor)` **não deixa nenhuma conta alterada se a operação falhar**.

3. Crie `Processador` com o método estático `List<String> processar(Conta conta, String[] comandos)`. Cada comando tem o formato `"D:50"` (depósito) ou `"S:30"` (saque).

    - Falhas **não interrompem** o processamento. Cada falha vira **uma mensagem** na lista devolvida.
    - Trate: número inválido (`NumberFormatException`, mensagem `Numero invalido em '<comando>'`), formato diferente de `<letra>:<número>` (`Formato invalido: <comando>`), operação desconhecida (`Operacao desconhecida: <letra>`) e as exceções de `Conta`.
    - Um atributo estático `comandosProcessados` deve ser incrementado **para todo comando, com sucesso ou falha**, num bloco `finally`. Exponha-o com `getComandosProcessados()`.

```
import java.util.List;

public class Principal {
    public static void main(String[] args) {
        Conta conta = new Conta("Ana", 100.0);
        String[] comandos = {"D:50", "S:30", "S:500", "D:abc", "X:10", "S:-5", "D:20:30", "S:40"};
        List<String> erros = Processador.processar(conta, comandos);

        System.out.println("Saldo final = " + conta.getSaldo());
        System.out.println("Processados = " + Processador.getComandosProcessados());
        System.out.println("Falhas = " + erros.size());
        for (String erro : erros) System.out.println(" - " + erro);

        Conta outra = new Conta("Bia", 0);
        try {
            conta.transferir(outra, 1000);
        } catch (SaldoInsuficienteException e) {
            System.out.println("Transferencia recusada: faltam " + (e.getValorSolicitado() - e.getSaldoAtual()));
        } finally {
            System.out.println("Saldos: " + conta.getSaldo() + " / " + outra.getSaldo());
        }
    }
}
```

Saída esperada

```
Saldo final = 80.0
Processados = 8
Falhas = 5
 - Saldo insuficiente: saldo=120.0, solicitado=500.0
 - Numero invalido em 'D:abc'
 - Operacao desconhecida: X
 - Valor de saque deve ser positivo: -5.0
 - Formato invalido: D:20:30
Transferencia recusada: faltam 920.0
Saldos: 80.0 / 0.0
```

Exercício 5: Corrigir e estender o estoque

Dificuldade: ★★★☆☆ (intermediário) | **Conteúdos:** classe abstrata, `equals`/`hashCode`, `Map`, `Iterator`, encapsulamento, `Comparator`

O código abaixo foi escrito às pressas e tem defeitos. Ele **não compila**, e, depois de compilar, apresenta estes sintomas:

- `equals` devolve `false` para dois produtos com o mesmo código criados em momentos diferentes.
- Adicionar o mesmo produto duas vezes **não soma** as quantidades.
- `removerEsgotados()` pode lançar `ConcurrentModificationException`.
- A estrutura interna do estoque está exposta.

```
public abstract class Produto {
    private String codigo;
    private String nome;
    private double preco;

    public Produto(String codigo, String nome, double preco) {
        this.codigo = codigo;
        this.nome = nome;
        this.preco = preco;
    }

    public abstract double frete();

    public String getCodigo() { return codigo; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Produto outro = (Produto) o;
        return codigo == outro.codigo;
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + "[" + codigo + ", " + nome + ", " + preco + "]";
    }
}
```

```
import java.time.LocalDate;

public class ProdutoPerecivel extends Produto {
    private LocalDate validade;

    public ProdutoPerecivel(String codigo, String nome, double preco, LocalDate validade) {
        super(codigo, nome, preco);
        this.validade = validade;
    }

    public LocalDate getValidade() { return validade; }

    @Override
    public double frete() {
        return 12.0;
    }
}
```

```
public class ProdutoEletronico extends Produto {
    private int garantiaMeses;

    public ProdutoEletronico(String codigo, String nome, double preco, int garantiaMeses) {
        super(codigo, nome, preco);
        this.garantiaMeses = garantiaMeses;
    }

    public int getGarantiaMeses() { return garantiaMeses; }

    @Override
    public double frete() {
        return preco * 0.02;
    }
}
```

```
import java.util.HashMap;
import java.util.Map;

public class Estoque {
    private Map<Produto, Integer> quantidades = new HashMap<>();

    public Map<Produto, Integer> getQuantidades() {
        return quantidades;
    }

    public void adicionar(Produto produto, int quantidade) {
        quantidades.put(produto, quantidade);
    }
```

```
    public void removerEsgotados() {
        for (Produto p : quantidades.keySet()) {
            if (quantidades.get(p) == 0) {
                quantidades.remove(p);
            }
        }
    }

    public double valorTotal() {
        double total = 0;
        for (Produto p : quantidades.keySet()) {
            total += p.getPreco() * quantidades.get(p);
        }
        return total;
    }
}
```

```
import java.time.LocalDate;

public class Principal {
    public static void main(String[] args) {
        Produto a1 = new ProdutoPerecivel(String.format("LEI-%02d", 1), "Leite", 5.0, LocalDate.of(2026, 12, 1));
        Produto a2 = new ProdutoPerecivel(String.format("LEI-%02d", 1), "Leite", 5.0, LocalDate.of(2026, 12, 1));
        System.out.println("Mesmo produto? " + a1.equals(a2));

        Estoque estoque = new Estoque();
        estoque.adicionar(a1, 10);
        estoque.adicionar(a2, 5);
        System.out.println("Itens distintos = " + estoque.getQuantidades().size());
        System.out.println("Quantidade de leite = " + estoque.getQuantidades().get(a1));

        estoque.adicionar(new ProdutoEletronico("MON-01", "Monitor", 900.0, 12), 0);
        estoque.adicionar(new ProdutoEletronico("TEC-01", "Teclado", 150.0, 6), 0);
        estoque.adicionar(new ProdutoEletronico("MOU-01", "Mouse", 80.0, 6), 3);
        estoque.removerEsgotados();
        System.out.println("Valor total = " + estoque.valorTotal());
    }
}
```

Parte A. Corrija **todos** os defeitos. O estoque não pode mais ter o getter do mapa. Troque-o por `int quantidadeDe(Produto)` e `int tiposDeProduto()`. Dois produtos são iguais quando têm o mesmo código, **mesmo que sejam de subclasses diferentes**.

Parte B. Implemente em `Estoque`:

- `Produto buscarPorCodigo(String codigo)`: devolve `null` se não achar.
- `List<Produto> maisBaratosQue(double limite)`: preço **estritamente menor**, em ordem crescente de preço.
- `double freteTotal()`: soma `frete()` de cada produto distinto com quantidade maior que zero.
- `Iterator<Produto> iterador()`: percorre uma **cópia** das chaves, de modo que o chamador não consiga alterar o estoque.

Com sua versão, o `Principal` abaixo deve produzir a saída esperada.

```
import java.time.LocalDate;
import java.util.Iterator;

public class Principal {
    public static void main(String[] args) {
        Produto a1 = new ProdutoPerecivel(String.format("LEI-%02d", 1), "Leite", 5.0, LocalDate.of(2026, 12, 1));
        Produto a2 = new ProdutoPerecivel(String.format("LEI-%02d", 1), "Leite", 5.0, LocalDate.of(2026, 12, 1));
        System.out.println("Mesmo produto? " + a1.equals(a2));

        Estoque estoque = new Estoque();
        estoque.adicionar(a1, 10);
        estoque.adicionar(a2, 5);
        System.out.println("Itens distintos = " + estoque.tiposDeProduto());
        System.out.println("Quantidade de leite = " + estoque.quantidadeDe(a1));

        estoque.adicionar(new ProdutoEletronico("MON-01", "Monitor", 900.0, 12), 0);
        estoque.adicionar(new ProdutoEletronico("TEC-01", "Teclado", 150.0, 6), 0);
        estoque.adicionar(new ProdutoEletronico("MOU-01", "Mouse", 80.0, 6), 3);
        estoque.removerEsgotados();
        System.out.println("Itens distintos apos remover = " + estoque.tiposDeProduto());
        System.out.println("Valor total = " + estoque.valorTotal());
        System.out.println("Frete total = " + estoque.freteTotal());
        System.out.println("Buscar MOU-01 = " + estoque.buscarPorCodigo("MOU-01"));
        System.out.println("Buscar XXX = " + estoque.buscarPorCodigo("XXX"));
        System.out.println("Mais baratos que 100 = " + estoque.maisBaratosQue(100));
        int n = 0;
        for (Iterator<Produto> it = estoque.iterador(); it.hasNext(); it.next()) n++;
        System.out.println("Percorridos = " + n);
    }
}
```

Saída esperada

```
Mesmo produto? true
Itens distintos = 1
Quantidade de leite = 15
Itens distintos apos remover = 2
```

```
Valor total = 315.0
Frete total = 13.6
Buscar MOU-01 = ProdutoEletronico[MOU-01, Mouse, 80.0]
Buscar XXX = null
Mais baratos que 100 = [ProdutoPerecivel[LEI-01, Leite, 5.0], ProdutoEletronico[MOU-01, Mouse, 80.0]]
Percorridos = 2
```

Exercício 6: Ranking genérico

Dificuldade: ★★★☆☆ (intermediário) | **Conteúdos:** interface, polimorfismo, métodos genéricos com limite, `Set`, `Comparator`, varargs

Implemente:

- A interface `Pontuavel` com `String nome()` e `int pontos()`.
- `Jogador` (nome e pontos), que implementa `Pontuavel`. Dois jogadores são iguais se tiverem o mesmo nome. `toString`: `Ana(80)`.
- `Equipe` (nome), que também implementa `Pontuavel`. Ela guarda seus jogadores em um **`Set`**, de modo que o mesmo jogador não entre duas vezes. `boolean adicionar(Jogador)` devolve se entrou. Os pontos da equipe são a soma dos pontos dos seus jogadores. `toString`: `Azul[140]`.
- A classe utilitária `Ranking` (não instanciável) com **métodos genéricos** cujo parâmetro de tipo `T` aceite apenas `Pontuavel`:
  - `static <T ...> List<T> melhores(Collection<T> itens, int n)`: até `n` itens, por pontos decrescentes e, em empate, por nome crescente. **Não pode alterar a coleção recebida**.
  - `static <T ...> T lider(Collection<T> itens)`: lança `NoSuchElementException` com a mensagem `Nao ha participantes` se vazia.
  - `static int somaPontos(Pontuavel... itens)`.

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Set;

public class Principal {
    public static void main(String[] args) {
        Jogador ana = new Jogador("Ana", 80);
        Jogador bia = new Jogador("Bia", 95);
        Jogador caio = new Jogador("Caio", 80);
        Jogador davi = new Jogador("Davi", 60);

        List<Jogador> jogadores = new ArrayList<>(List.of(ana, bia, caio, davi));
        System.out.println("Top 3 = " + Ranking.melhores(jogadores, 3));
        System.out.println("Top 10 = " + Ranking.melhores(jogadores, 10));
        System.out.println("Lista original intacta = " + jogadores);

        Equipe azul = new Equipe("Azul");
        azul.adicionar(ana);
        azul.adicionar(davi);
        System.out.println("Repetido entra? " + azul.adicionar(new Jogador("Ana", 1)));
        Equipe verde = new Equipe("Verde");
        verde.adicionar(bia);
        verde.adicionar(caio);
        Equipe vermelha = new Equipe("Vermelha");
        vermelha.adicionar(new Jogador("Eva", 200));

        Set<Equipe> equipes = new HashSet<>(List.of(azul, verde, vermelha));
        System.out.println("Equipes = " + Ranking.melhores(equipes, 2));
        System.out.println("Lider = " + Ranking.lider(equipes));
        System.out.println("Soma = " + Ranking.somaPontos(ana, bia, azul));
        try {
            Ranking.lider(new ArrayList<Jogador>());
        } catch (NoSuchElementException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Saída esperada

```
Top 3 = [Bia(95), Ana(80), Caio(80)]
Top 10 = [Bia(95), Ana(80), Caio(80), Davi(60)]
Lista original intacta = [Ana(80), Bia(95), Caio(80), Davi(60)]
Repetido entra? false
Equipes = [Vermelha[200], Verde[175]]
Lider = Vermelha[200]
Soma = 315
Erro: Nao ha participantes
```

Exercício 7: Regras de frete

Dificuldade: ★★★★☆ (intermediário a avançado) | **Conteúdos:** interface funcional, composição de interfaces, fábrica com retorno polimórfico, injeção de dependência, lambda

`Pacote` guarda `pesoKg`, `distanciaKm` e `valorDeclarado` (todos `double`, com getters).

- A interface funcional `RegraFrete` tem o método `double calcular(Pacote pacote)`.
- `FretePadrao`: `peso * 2.0 + distancia * 0.05`.
- `FreteExpresso`: **recebe outra `RegraFrete` no construtor** e devolve o valor dela multiplicado por 1,8.
- `FreteGratisAcimaDe`: recebe um limite e outra `RegraFrete`. Se o valor declarado for **maior ou igual** ao limite, o frete é 0. Senão, delega à regra recebida.
- `FreteFactory` (não instanciável): `static RegraFrete para(String tipo)`, que ignora maiúsculas/minúsculas e aceita `padrao`, `expresso` (expresso sobre o padrão), `gratis200` (grátis a partir de 200 sobre o padrão) e `retirada` (**sempre 0, implementada com lambda**). Tipo nulo ou desconhecido lança `IllegalArgumentException` (`Tipo de frete desconhecido: <tipo>`).
- `Pedido`: recebe `Pacote` e `RegraFrete` **no construtor** (injeção). `frete()` devolve o frete e `total()` devolve valor declarado mais frete.

```
import java.util.Locale;

public class Principal {
    public static void main(String[] args) {
        Pacote leve = new Pacote(2.0, 100.0, 80.0);
        Pacote caro = new Pacote(5.0, 400.0, 350.0);

        String[] tipos = {"padrao", "expresso", "gratis200", "retirada"};
        for (String tipo : tipos) {
            Pedido a = new Pedido(leve, FreteFactory.para(tipo));
            Pedido b = new Pedido(caro, FreteFactory.para(tipo));
            System.out.println(String.format(Locale.US, "%-10s leve: frete=%.2f total=%.2f | caro: frete=%.2f total=%.2f",
                    tipo, a.frete(), a.total(), b.frete(), b.total()));
        }

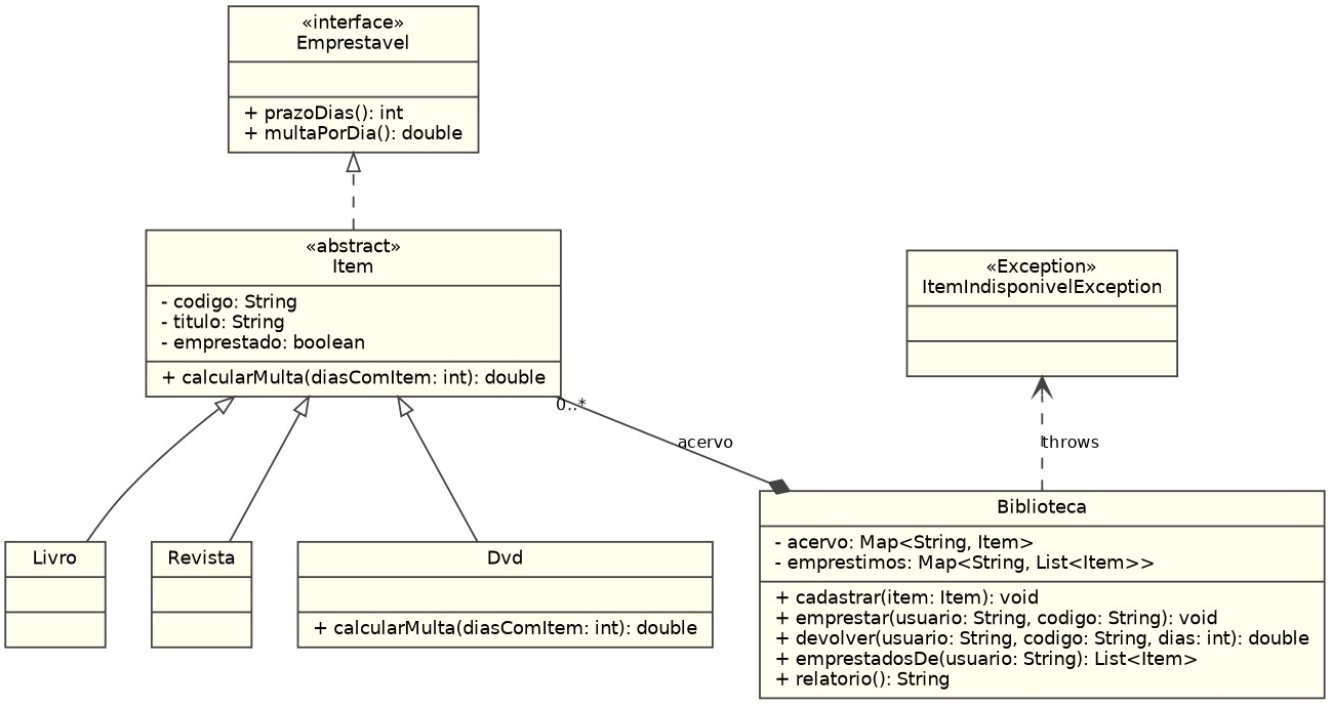
        RegraFrete promo = p -> Math.min(10.0, new FretePadrao().calcular(p));
        System.out.println(String.format(Locale.US, "promo caro: %.2f", new Pedido(caro, promo).frete()));
        RegraFrete combinada = new FreteExpresso(new FreteGratisAcimaDe(300.0, promo));
        System.out.println(String.format(Locale.US, "combinada leve: %.2f | caro: %.2f",
                combinada.calcular(leve), combinada.calcular(caro)));
        try {
            FreteFactory.para("drone");
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Saída esperada

```
padrao     leve: frete=9.00 total=89.00 | caro: frete=30.00 total=380.00
expresso   leve: frete=16.20 total=96.20 | caro: frete=54.00 total=404.00
gratis200  leve: frete=9.00 total=89.00 | caro: frete=0.00 total=350.00
retirada   leve: frete=0.00 total=80.00 | caro: frete=0.00 total=350.00
promo caro: 10.00
combinada leve: 16.20 | caro: 0.00
Erro: Tipo de frete desconhecido: drone
```

Exercício 8: Biblioteca

Dificuldade: ★★★★☆ (intermediário a avançado) | **Conteúdos:** herança, classe abstrata, interface, `Map<String, List<...>>`, exceção verificada, sobrescrita com `super`, `StringBuilder`



Regras

- `Item` implementa `Emprestavel` e guarda `codigo`, `titulo` e se está emprestado. `calcularMulta(int diasComItem)` devolve `(dias - prazoDias()) * multaPorDia()` se houver atraso, e `0` caso contrário. O método que altera o estado de empréstimo deve ser visível **apenas no pacote**.
- `Livro` (prazo 14 dias, multa 1,00/dia), `Revista` (7 dias, 0,50/dia) e `Dvd` (3 dias, 3,00/dia). O `Dvd` **sobrescreve** `calcularMulta`: se houver atraso, soma uma taxa fixa de 5,00 ao valor da superclasse.
- `ItemIndisponivelException` é verificada.

- `Biblioteca`:
  - `cadastrar(Item)` lança `IllegalArgumentException` se o código já existir.
  - `emprestar(usuario, codigo)` lança `ItemIndisponivelException` se o item não existir (`Item inexistente: <codigo>`) ou já estiver emprestado (`Item ja emprestado: <codigo>`). Um usuário pode ter **no máximo 3** itens: o quarto lança `IllegalStateException` (`Limite de 3 emprestimos atingido por <usuario>`).
  - `devolver(usuario, codigo, diasComItem)` devolve a multa. Se o usuário não tiver o item, lança `IllegalArgumentException` (`<usuario> nao possui o item <codigo>`).
  - `emprestadosDe(usuario)` devolve uma **cópia** (lista vazia se o usuário não tiver nada).
  - `relatorio()` usa `StringBuilder`, uma linha por item na ordem de cadastro: `String.format("%-6s | %-8s | %-22s | %s%n", ...)` com código, nome simples da classe, título e `EMPRESTADO`/`DISPONIVEL`.

```
import java.util.Locale;

public class Principal {
    public static void main(String[] args) {
        Biblioteca b = new Biblioteca();
        b.cadastrar(new Livro("L1", "Java: Como Programar"));
        b.cadastrar(new Livro("L2", "Clean Code"));
        b.cadastrar(new Revista("R1", "Superinteressante"));
        b.cadastrar(new Dvd("D1", "Matrix"));
        b.cadastrar(new Dvd("D2", "Interestelar"));
        try {
            b.cadastrar(new Livro("L1", "Duplicado"));
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }

        try {
            b.emprestar("ana", "L1");
            b.emprestar("ana", "R1");
            b.emprestar("ana", "D1");
            b.emprestar("ana", "L2");
        } catch (ItemIndisponivelException e) {
            System.out.println("Indisponivel: " + e.getMessage());
```

```
        } catch (IllegalStateException e) {
            System.out.println("Regra: " + e.getMessage());
        }
        try {
            b.emprestar("bia", "L1");
        } catch (ItemIndisponivelException e) {
            System.out.println("Indisponivel: " + e.getMessage());
        }
        try {
            b.emprestar("bia", "X9");
        } catch (ItemIndisponivelException e) {
            System.out.println("Indisponivel: " + e.getMessage());
        }

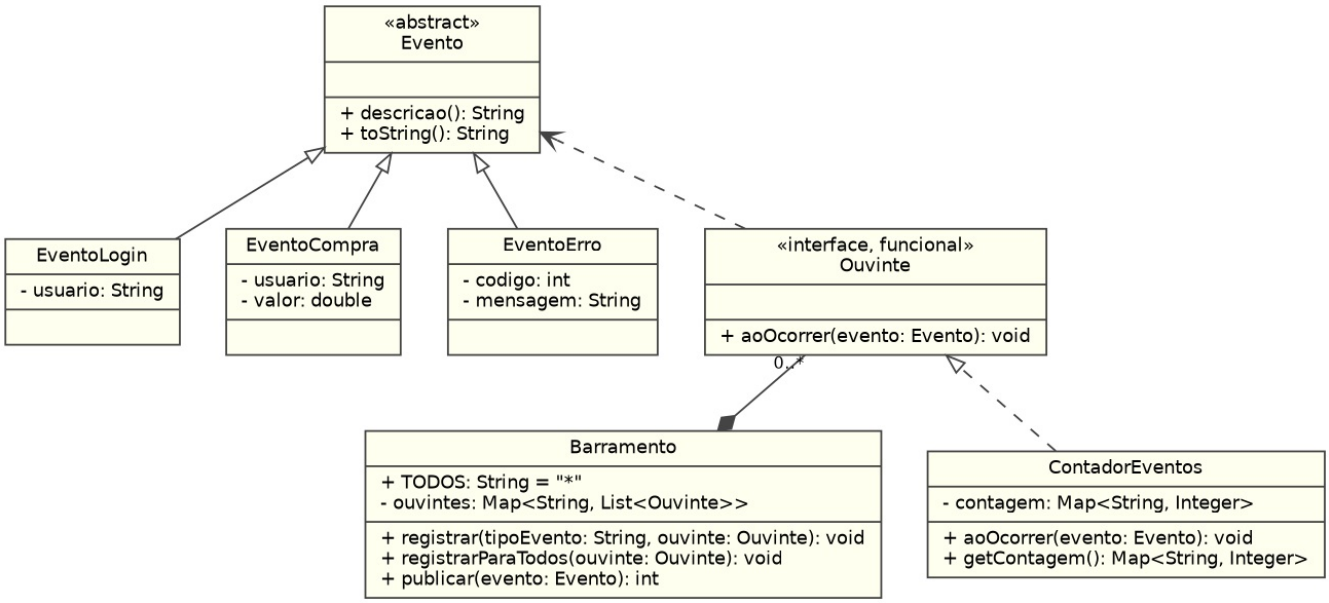
        System.out.print(b.relatorio());
        System.out.println("Emprestados de ana: " + b.emprestadosDe("ana"));
        System.out.println(String.format(Locale.US, "Multa L1 (20 dias): %.2f", b.devolver("ana", "L1", 20)));
        System.out.println(String.format(Locale.US, "Multa R1 (7 dias): %.2f", b.devolver("ana", "R1", 7)));
        System.out.println(String.format(Locale.US, "Multa D1 (5 dias): %.2f", b.devolver("ana", "D1", 5)));
        try {
            b.devolver("ana", "D1", 1);
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }
        System.out.print(b.relatorio());
    }
}
```

Saída esperada

```
Erro: Codigo ja cadastrado: L1
Regra: Limite de 3 emprestimos atingido por ana
Indisponivel: Item ja emprestado: L1
Indisponivel: Item inexistente: X9
L1     | Livro    | Java: Como Programar   | EMPRESTADO
L2     | Livro    | Clean Code             | DISPONIVEL
R1     | Revista  | Superinteressante      | EMPRESTADO
D1     | Dvd      | Matrix                 | EMPRESTADO
D2     | Dvd      | Interestelar           | DISPONIVEL
Emprestados de ana: [Livro[L1 - Java: Como Programar], Revista[R1 - Superinteressante], Dvd[D1 - Matrix]]
Multa L1 (20 dias): 6.00
Multa R1 (7 dias): 0.00
Multa D1 (5 dias): 11.00
Erro: ana nao possui o item D1
L1     | Livro    | Java: Como Programar   | DISPONIVEL
L2     | Livro    | Clean Code             | DISPONIVEL
R1     | Revista  | Superinteressante      | DISPONIVEL
D1     | Dvd      | Matrix                 | DISPONIVEL
D2     | Dvd      | Interestelar           | DISPONIVEL
```

Exercício 9: Barramento de eventos

Dificuldade: ★★★★★ (avançado) | **Conteúdos:** herança, interface funcional, lambda, classe anônima, `Map<String, List<...>>`, `instanceof` com padrão, tratamento de exceção



Regras

- `Evento` é abstrata. `toString()` devolve `<NomeDaClasse>: <descricao()>`. Sua descrição é: login → `<usuario> entrou`; compra → `<usuario> comprou <valor>`; erro → `erro <codigo> - <mensagem>`.
- `Ouvinte` é uma interface funcional.
- `Barramento` guarda ouvintes em `Map<String, List<Ouvinte>>`, cuja chave é o **nome simples da classe do evento** (`getClass().getSimpleName()`) ou `"*"` para ouvintes de qualquer evento.
- `publicar(Evento)` notifica **primeiro** os ouvintes de `"*"` e **depois** os do tipo do evento, cada grupo em ordem de registro, e devolve **quantos ouvintes concluíram sem exceção**. Sem ouvintes devolve 0, sem lançar nada. Se um ouvinte lançar `RuntimeException`, o barramento imprime `[falha em ouvinte: <mensagem>]` e **segue** para os demais.
- `ContadorEventos` é um `Ouvinte` que conta eventos por tipo e expõe uma **cópia ordenada por nome** da contagem.

> **Dica:** variáveis locais usadas dentro de lambdas precisam ser *efetivamente finais*. Para acumular um total, use um atributo estático ou um objeto auxiliar.

```
public class Principal {
    private static double totalVendas = 0.0;

    public static void main(String[] args) {
        Barramento barramento = new Barramento();
        ContadorEventos contador = new ContadorEventos();

        barramento.registrarParaTodos(contador);
        barramento.registrarParaTodos(e -> System.out.println("LOG  -> " + e));
        barramento.registrar("EventoCompra", e -> {
            EventoCompra compra = (EventoCompra) e;
            totalVendas += compra.getValor();
        });
        barramento.registrar("EventoErro", e -> {
            if (e instanceof EventoErro erro && erro.getCodigo() >= 500) {
                throw new IllegalStateException("alerta critico " + erro.getCodigo());
            }
        });
        barramento.registrar("EventoErro", new Ouvinte() {
            private int vistos = 0;

            @Override
            public void aoOcorrer(Evento evento) {
                vistos++;
                System.out.println("ERRO -> ja vi " + vistos + " erro(s)");
            }
        });

        System.out.println("notificados = " + barramento.publicar(new EventoLogin("ana")));
        System.out.println("notificados = " + barramento.publicar(new EventoCompra("ana", 120.5)));
        System.out.println("notificados = " + barramento.publicar(new EventoCompra("bia", 79.5)));
        System.out.println("notificados = " + barramento.publicar(new EventoErro(404, "pagina nao encontrada")));
        System.out.println("notificados = " + barramento.publicar(new EventoErro(500, "falha interna")));

        Barramento vazio = new Barramento();
```

```
        System.out.println("sem ouvintes = " + vazio.publicar(new EventoLogin("caio")));

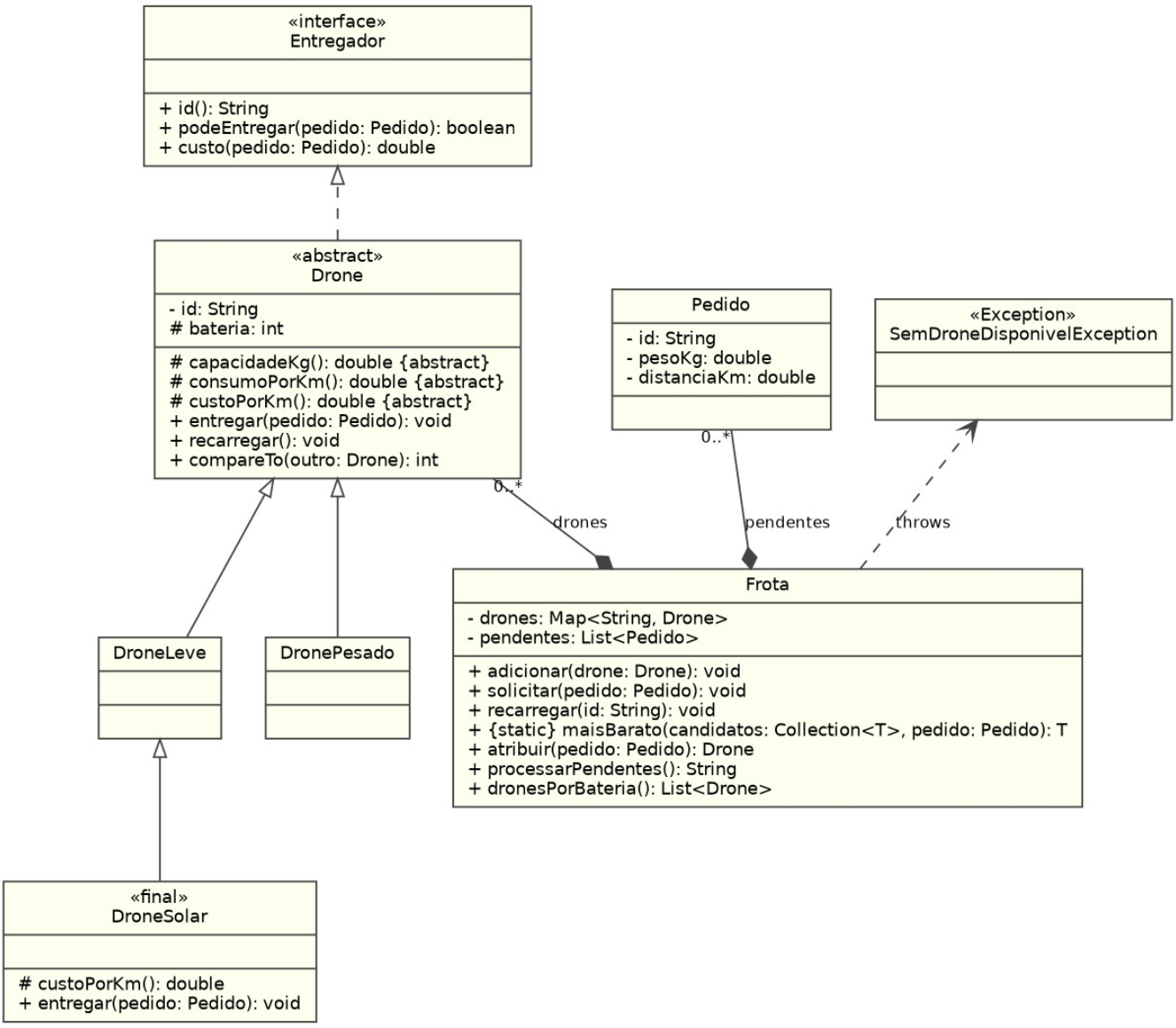
        System.out.println("Total de vendas = " + totalVendas);
        System.out.println("Contagem = " + contador.getContagem());
    }
}
```

Saída esperada

```
LOG  -> EventoLogin: ana entrou
notificados = 2
LOG  -> EventoCompra: ana comprou 120.5
notificados = 3
LOG  -> EventoCompra: bia comprou 79.5
notificados = 3
LOG  -> EventoErro: erro 404 - pagina nao encontrada
ERRO -> ja vi 1 erro(s)
notificados = 4
LOG  -> EventoErro: erro 500 - falha interna
  [falha em ouvinte: alerta critico 500]
ERRO -> ja vi 2 erro(s)
notificados = 3
sem ouvintes = 0
Total de vendas = 200.0
Contagem = {EventoCompra=2, EventoErro=2, EventoLogin=1}
```

Exercício 10: Frota de drones de entrega

Dificuldade: ★★★★★ (avançado, integra tudo) | **Conteúdos:** herança em três níveis, classe abstrata, interface, `Comparable`, método genérico com limite, `Map` ordenado, `Iterator.remove()`, exceção verificada, `StringBuilder`



Regras

- `Entregador` é a interface com `id()`, `podeEntregar(Pedido)` e `custo(Pedido)`.
- `Drone` (abstrata) implementa `Entregador` e `Comparable<Drone>`. A bateria vai de 0 a 100: fora disso o construtor lança `IllegalArgumentException`. Os métodos abstratos `capacidadeKg()`, `consumoPorKm()` (% de bateria por km) e `custoPorKm()` são implementados pelas subclasses.
  - **Bateria necessária** para um pedido: `consumoPorKm * distancia * 2` (ida e volta).
  - `podeEntregar`: peso menor ou igual à capacidade **e** bateria suficiente.
  - `custo`: `custoPorKm * distancia * 2`.
  - `entregar(Pedido)` lança `IllegalStateException` se não puder; senão desconta da bateria a necessária **arredondada para cima** (`Math.ceil`, convertida para `int`).
  - `recarregar()` leva a bateria a 100.
  - Ordem natural: **maior bateria primeiro**, empate por `id`. Dois drones são iguais se tiverem o mesmo `id`.
- `DroneLeve` (capacidade 2,0 kg; consumo 1,0; custo 0,5), `DronePesado` (10,0 kg; 2,5; 1,5) e `DroneSolar` (**não pode ser herdado**; estende `DroneLeve`; custo 0,3; depois de uma entrega **recarrega 5 pontos**, no máximo 100).
- `Frota`:
  - Drones em `Map` ordenado por id. Id repetido lança `IllegalArgumentException` (`Drone repetido: <id>`).

- `static <T extends Entregador> T mais Barato(Collection<T> candidatos, Pedido p)`: menor custo, empate pelo menor `id`.
- `Drone atribuir(Pedido)` escolhe, entre os drones que podem, o mais barato (usando o método genérico), manda entregar e o devolve. Se nenhum puder, lança a exceção verificada `SemDroneDisponivelException` (`nenhum drone pode atender <idPedido>`).
- `String processarPendentes()` percorre os pedidos pendentes com um `Iterator`. Cada pedido atendido é **removido com `iterator.remove()`** e registrado como `<pedido> -> <drone> (custo <valor %.2f>)`. Cada não atendido permanece pendente e é registrado como `<pedido> -> PENDENTE (<mensagem>)`. Use `StringBuilder`.
- `dronesPorBateria()` devolve uma **cópia ordenada pela ordem natural**.

```
public class Principal {
    public static void main(String[] args) {
        Frota frota = new Frota();
        frota.adicionar(new DroneLeve("L1", 100));
        frota.adicionar(new DroneSolar("S1", 40));
        frota.adicionar(new DronePesado("P1", 100));
        try {
            frota.adicionar(new DroneLeve("L1", 50));
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }

        frota.solicitar(new Pedido("A", 1.5, 10));
        frota.solicitar(new Pedido("B", 5.0, 20));
        frota.solicitar(new Pedido("C", 8.0, 5));
        frota.solicitar(new Pedido("D", 1.0, 15));
        frota.solicitar(new Pedido("E", 3.0, 1));

        System.out.println("== 1o processamento");
        System.out.print(frota.processarPendentes());
        System.out.println("Pendentes: " + frota.totalPendentes());
        System.out.println("Baterias: " + frota.dronesPorBateria());

        frota.recarregar("P1");
        System.out.println("== 2o processamento (apos recarregar P1)");
        System.out.print(frota.processarPendentes());
        System.out.println("Pendentes: " + frota.totalPendentes());
        System.out.println("Baterias: " + frota.dronesPorBateria());
    }
}
```

Saída esperada

```
Erro: Drone repetido: L1
== 1o processamento
A -> S1 (custo 6.00)
B -> P1 (custo 60.00)
C -> PENDENTE (nenhum drone pode atender C)
D -> L1 (custo 15.00)
E -> PENDENTE (nenhum drone pode atender E)
Pendentes: 2
Baterias: [DroneLeve[L1, bateria=70], DroneSolar[S1, bateria=25], DronePesado[P1, bateria=0]]
== 2o processamento (apos recarregar P1)
C -> P1 (custo 15.00)
E -> P1 (custo 3.00)
Pendentes: 0
Baterias: [DroneLeve[L1, bateria=70], DronePesado[P1, bateria=70], DroneSolar[S1, bateria=25]]
```

Parte II: Cinco provas simuladas

Cada prova é **autocontida** e segue o modelo das referências: contexto, diagrama UML, tabela de itens com pontuação (soma 10,0) e quadros de saída. Resolva cada uma em um **projeto novo**. Nos diagramas, construtores, getters e setters podem ter sido omitidos "por maior clareza", como nas provas originais. Quando o UML mostra `{abstract}`, `{static}`, `{default}` ou `{final}`, trata-se de modificadores Java.

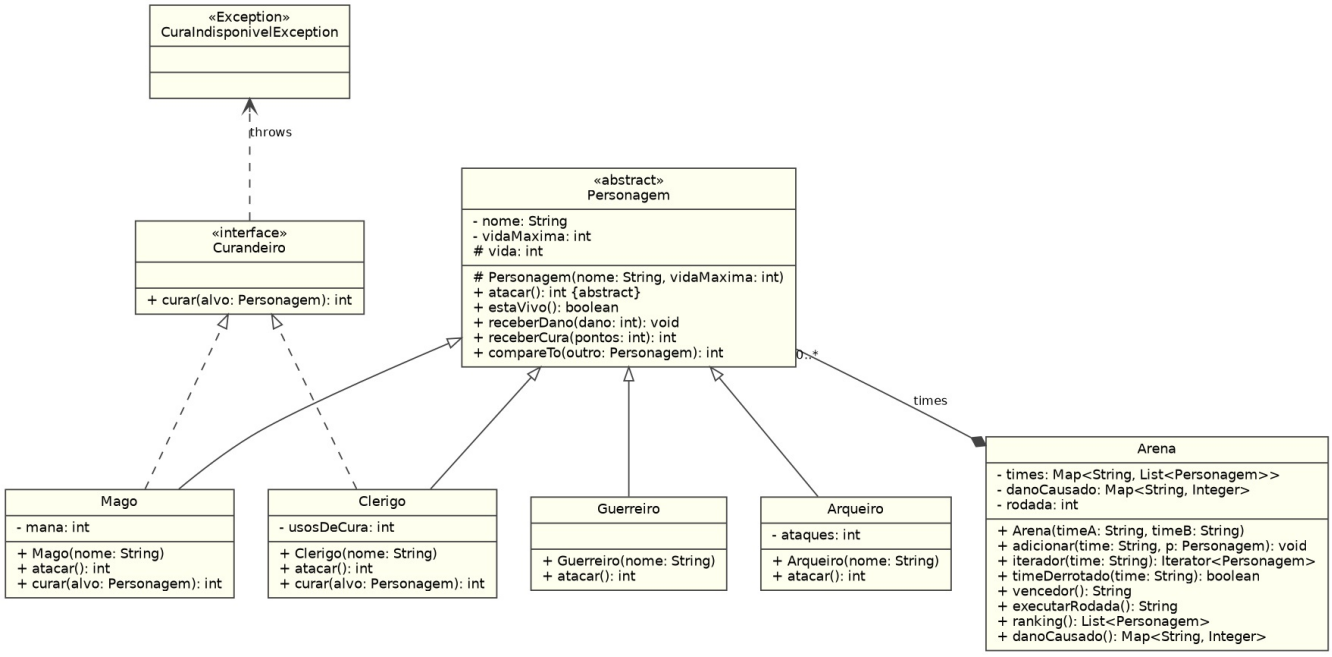
Legenda dos diagramas. Seta de ponta vazada contínua = herança. Seta de ponta vazada tracejada = implementação de interface. Losango cheio = composição/agregação (o losango fica no "todo"). Seta aberta tracejada = dependência (`throws` indica exceção lançada). Cada prova traz ainda os modificadores de acesso: `+` público, `-` privado, `#` protegido, `~` pacote.

Prova Simulada 1: Programação Orientada a Objetos

Tema: Arena Java (combate por turnos)

✔ A prova é individual e sem consulta.	✔ Nomeie o projeto como `PRONTUARIO_SIM1` e envie todo o projeto.
✔ Não é permitido usar código implementado por você anteriormente.	✔ Não coloque seu nome no projeto.
✔ Atribui-se nota zero à prova em desacordo com os itens acima.	✔ Tempo sugerido: 3h30.

Você foi contratado para criar o motor de combate de um jogo de RPG. Dois times se enfrentam em rodadas. Cada personagem tem uma **classe** (guerreiro, arqueiro, mago ou clérigo) que define sua vida máxima e como ele ataca. Magos e clérigos também são **curandeiros**: podem curar aliados feridos, mas a cura pode estar indisponível (falta de mana ou de usos). O comportamento específico de cada classe deve ser resolvido **polimorficamente**, sem `if` que teste a classe do personagem para atacar.



Para a atribuição da nota serão consideradas funcionalidade, qualidade, adequação e pertinência de cada solução. Bom senso faz parte da prova. Você pode criar métodos auxiliares, principalmente privados.

#	Descrição	Pontos
1	Crie o projeto com três pacotes: `arena.modelo`, `arena.jogo` e `arena.main`. A classe `Principal` (com o método `main`) fica em `main`, a `Arena` em `jogo` e todas as demais em `modelo`.	0,5
2	Crie a interface `Curandeiro` e a exceção **verificada** `CuraIndisponivelException`, que recebe o motivo (`String`) no construtor e o expõe por `getMessage()`. O método `curar` pode lançar essa exceção.	1,0
3	Crie a classe **abstrata** `Personagem`. `nome` e `vidaMaxima` são imutáveis e `vida` é visível às subclasses. O **construtor** é visível apenas à hierarquia e lança `IllegalArgumentException` se o nome for nulo ou em branco, ou se a vida máxima for ≤ 0. A vida começa igual à máxima. Implemente `estaVivo()` (vida > 0), `receberDano(int)` (dano negativo lança `IllegalArgumentException`; a vida **nunca fica abaixo de 0**) e `receberCura(int)` (personagem derrotado **não é curado** e a cura devolve 0; a vida **não passa da máxima**; devolve a cura **efetivamente aplicada**). Inclua os getters.	1,0
4	Em `Personagem`, implemente `equals/hashCode` (dois personagens são iguais se tiverem o **mesmo nome, independentemente da subclasse**: um `Guerreiro Thor` e um `Mago Thor` são o mesmo personagem), `compareTo` (**maior vida primeiro**; empate, nome em ordem alfabética) e `toString` no formato `Guerreiro Thor (120/120)`: nome simples da classe, nome e `(vida/vidaMaxima)`.	0,75
5	Implemente `Guerreiro` (vida máxima 120; dano 18; em **fúria**, isto é, com vida **menor que 30%** da máxima, o dano passa a 150%, calculado com inteiros como `18 * 3 / 2`) e `Arqueiro` (vida máxima 90; dano 14; **a cada terceiro ataque** feito pelo arqueiro, ou seja, o 3º, o 6º etc., o dano **dobra**).	1,0
6	Implemente `Mago` (vida máxima 70; mana inicial 40). Ao atacar, se tiver **pelo menos 10** de mana, gasta 10 e causa 25 de dano; senão causa 3 de dano e não gasta mana. O `Mago` é `Curandeiro`: curar custa 15 de mana. Com menos de 15, lança a exceção com o motivo `<nome> sem mana (<mana atual>)`. Em caso de sucesso, cura 30 pontos do alvo e devolve a cura efetiva.	1,0

7	Implemente `Clerigo` (vida máxima 100; dano 8). É `Curandeiro` com **2 usos** de cura por combate, cada um cura 35. Sem usos, lança a exceção com o motivo `<nome> sem usos de cura`.	0,5
8	Crie a classe `Arena`. O construtor recebe os nomes dos dois times e a arena os guarda em um **mapa que preserva a ordem de cadastro**, associado à lista de seus personagens. Implemente: `adicionar(String time, Personagem p)` (time inexistente lança `IllegalArgumentException` com `Time inexistente: <time>`; se **qualquer** personagem igual já estiver em **qualquer** time, lança `Personagem repetido: <nome>`); `Iterator<Personagem> iterador(String time)` (a lista **não pode ser exposta**); `timeDerrotado(String)`; e `String vencedor()`, que devolve o nome do time vencedor ou `null`.	1,0
9	Implemente `String executarRodada()` da `Arena` (veja as regras abaixo). A arena **não imprime nada**; ela devolve o log da rodada, montado com `StringBuilder`.	2,0
10	Implemente `List<Personagem> ranking()`, que devolve **todos** os personagens ordenados pela ordem natural **sem alterar a ordem interna** da arena, e `Map<String, Integer> danoCausado()`, uma **cópia** ordenada por nome do dano **nominal** causado por cada personagem. O dano nominal é a soma dos valores devolvidos por `atacar()`, **sem** descontar a vida que o alvo ainda tinha.	0,75
11	Crie `Principal`. Monte a arena com os times `Dragoes` e `Fenix`. Em `Dragoes`: `Guerreiro Thor`, `Mago Merlin` e `Clerigo Aria`. Em `Fenix`: `Arqueiro Legolas`, `Guerreiro Conan` e `Arqueiro Robin`, nessa ordem. Tente adicionar um `Mago Thor` a `Fenix` e trate o erro. Execute rodadas **enquanto não houver vencedor**, com no máximo `MAX_RODADAS` (constante, valor 40). Imprima o log de cada rodada, o vencedor com a quantidade de rodadas, o ranking e o dano causado.	0,5
😈	`Arena` expondo suas listas ou mapas internos por getters (retornando a estrutura original).	-1,0
😈	Imprimir qualquer coisa fora de `Principal`, ou não seguir as orientações de criação do projeto.	-1,0

Regras do item 9 (execução da rodada). A rodada começa incrementando um contador interno de rodadas (a primeira é a rodada 1) e escreve a linha `--- Rodada <n> ---`. Depois percorre os times **na ordem de cadastro** e, em cada time, os personagens **na ordem em que foram adicionados**. Para cada personagem **vivo**:

1. Se o time inimigo não tem mais ninguém vivo, a rodada **termina imediatamente** (devolva o log até aqui).
2. Se o personagem é `Curandeiro`, procure entre os aliados **vivos** (ele inclusive) o de **menor vida** cuja vida seja **menor que a metade** da vida máxima (em empate, o primeiro da lista). Se existir, tente curá-lo:
    - se conseguir, registre `<curandeiro> cura <alvo> (+<cura efetiva>) vida=<vida do alvo>` e passe ao próximo personagem;
    - se lançar `CuraIndisponivelException`, registre `<curandeiro> nao pode curar: <motivo>` e **siga para o ataque**.
3. **Ataque:** o alvo é o **primeiro personagem vivo** do time inimigo, na ordem de cadastro. Aplique o dano devolvido por `atacar()`, acumule-o em `danoCausado` e registre `<atacante> ataca <alvo> (-<dano>) vida=<vida do alvo>`, acrescentando `[DERROTADO]` se o alvo chegou a 0.

Cada registro ocupa uma linha, terminada por quebra de linha.

Quadro A: início e fim de uma execução esperada do `Principal` (as linhas centrais foram omitidas)

```
Erro: Personagem repetido: Thor
--- Rodada 1 ---
Thor ataca Legolas (-18) vida=72
Merlin ataca Legolas (-25) vida=47
Aria ataca Legolas (-8) vida=39
Legolas ataca Thor (-14) vida=106
Conan ataca Thor (-18) vida=88
Robin ataca Thor (-14) vida=74
--- Rodada 2 ---
Thor ataca Legolas (-18) vida=21
Merlin ataca Legolas (-25) vida=0 [DERROTADO]
Aria ataca Conan (-8) vida=112
Conan ataca Thor (-18) vida=56
Robin ataca Thor (-14) vida=42
--- Rodada 3 ---
Thor ataca Conan (-18) vida=94
Merlin cura Thor (+30) vida=72
Aria ataca Conan (-8) vida=86
Conan ataca Thor (-18) vida=54
Robin ataca Thor (-28) vida=26
--- Rodada 4 ---
Thor ataca Conan (-27) vida=59
...
Aria nao pode curar: Aria sem usos de cura
Aria ataca Robin (-8) vida=0 [DERROTADO]
=== Vencedor: Dragoes apos 15 rodadas
--- Ranking
Clerigo Aria (30/100)
Guerreiro Conan (0/120)
Arqueiro Legolas (0/90)
Mago Merlin (0/70)
Arqueiro Robin (0/90)
Guerreiro Thor (0/120)
--- Dano causado
Aria = 104
Conan = 99
Legolas = 14
Merlin = 71
```

```
Robin = 252
Thor = 135
```

Prova Simulada 2: Programação Orientada a Objetos

Tema: MelodIFy (streaming de áudio)

✔ A prova é individual e sem consulta.	✔ Nomeie o projeto como `PRONTUARIO_SIM2` e envie todo o projeto.
✔ Não é permitido usar código não fornecido junto com a prova.	✔ Não coloque seu nome no projeto.
✔ Atribui-se nota zero à prova em desacordo com os itens acima.	✔ Tempo sugerido: 3h30.

Sua equipe está criando o **MelodIFy**, um serviço de streaming de áudio. O catálogo contém **músicas** e **podcasts**, que são tipos de **mídia** e podem ser **reproduzidos**. Um **usuário** monta **playlists** (sem mídias repetidas, preservando a ordem em que foram adicionadas), toca mídias e consulta as mais ouvidas. Um colega já implementou a classe `Dados`, que devolve o catálogo como uma lista de linhas de texto. Você não deve alterá-la.

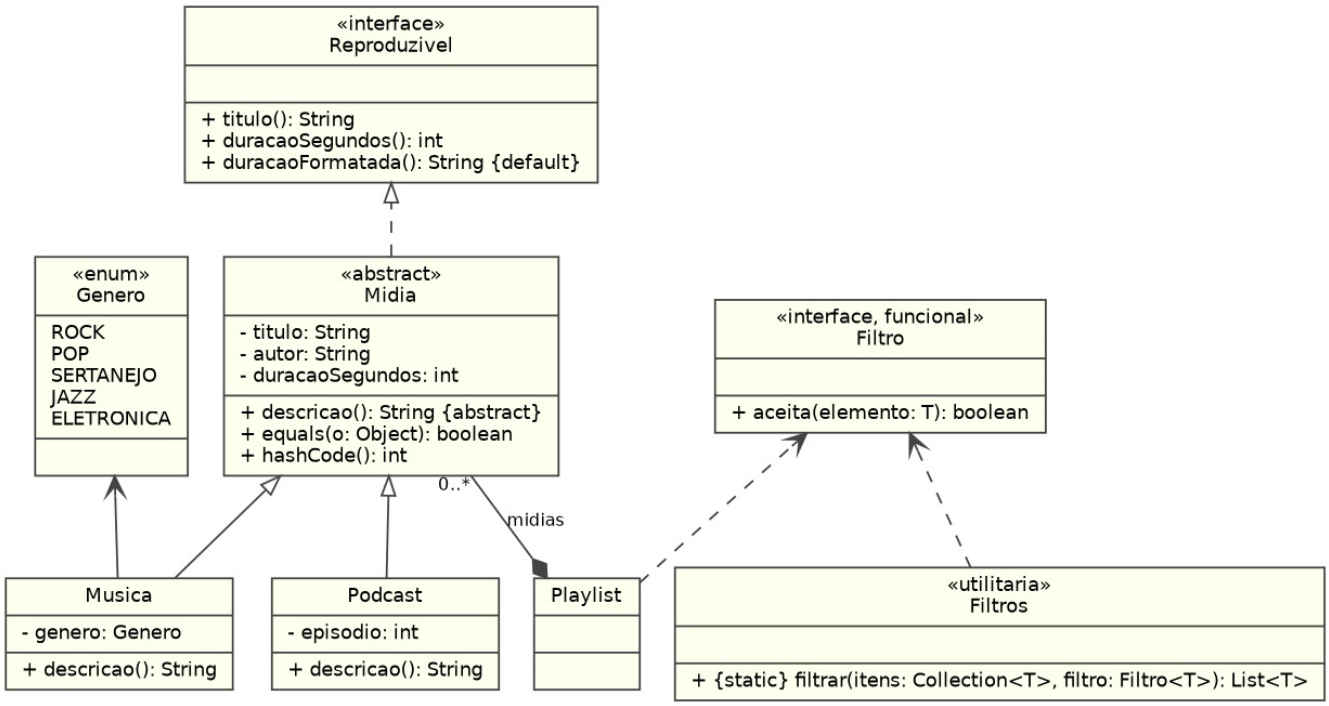
```java
package melodify.dados;

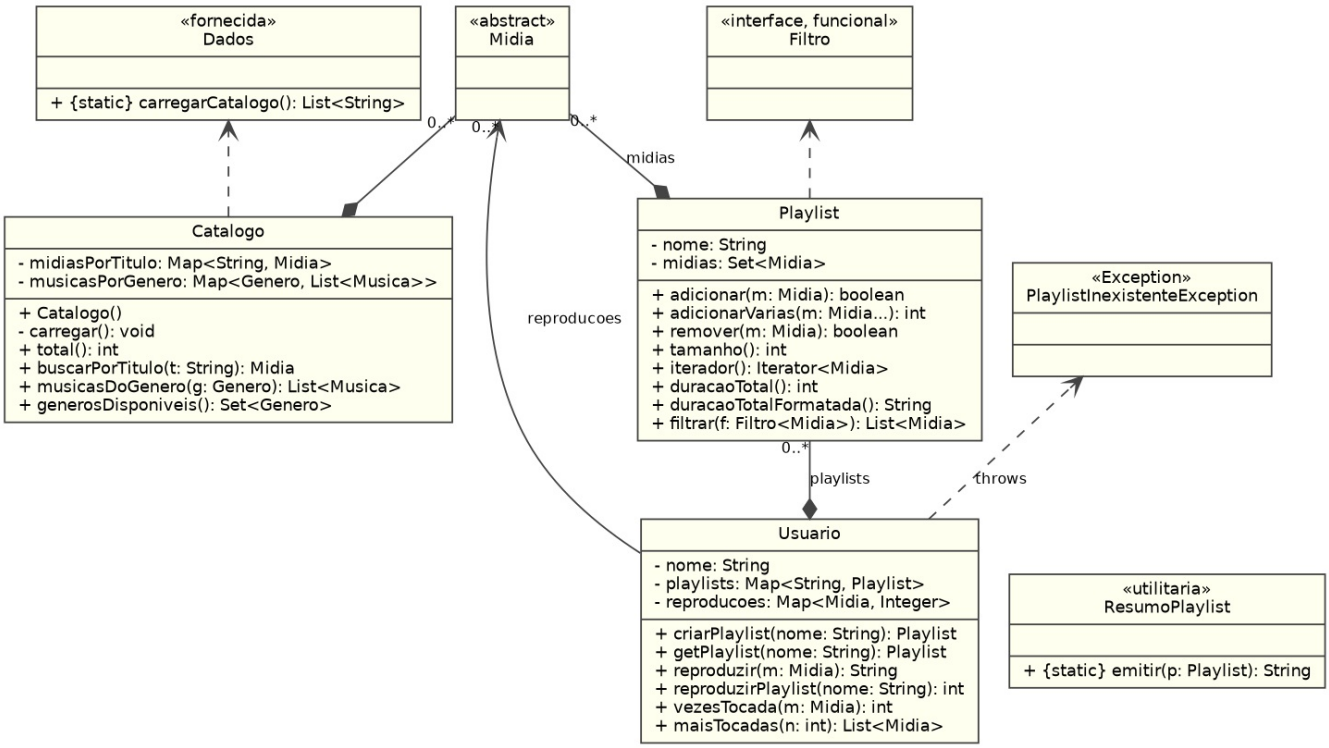
import java.util.List;

public final class Dados {
    private Dados() { }

    public static List<String> carregarCatalogo() {
        return List.of(
            "MUSICA;Bohemian Rhapsody;Queen;ROCK;355",
            "MUSICA;Billie Jean;Michael Jackson;POP;294",
            "MUSICA;Evidencias;Chitaozinho e Xororo;SERTANEJO;284",
            "MUSICA;Take Five;Dave Brubeck;JAZZ;324",
            "MUSICA;Another One Bites the Dust;Queen;ROCK;215",
            "MUSICA;Harder Better Faster;Daft Punk;ELETRONICA;224",
            "MUSICA;Thriller;Michael Jackson;POP;357",
            "PODCAST;Flow #42;Igor 3K;42;3600",
            "PODCAST;Nerdcast #10;Jovem Nerd;10;4200",
            "MUSICA;Hotel California;Eagles;ROCK;391"
        );
    }
}
```

Cada linha tem campos separados por `;`. O primeiro campo é o tipo. **Música:** `MUSICA;título;artista;GÊNERO;duração em segundos`. **Podcast:** `PODCAST;título;apresentador;episódio;duração em segundos`.





Para a atribuição da nota serão consideradas funcionalidade, qualidade, adequação e pertinência de cada solução. Você pode criar métodos auxiliares, principalmente privados.

#	Descrição	Pontos
1	Crie o projeto com os pacotes `melodify.modelo`, `melodify.dados`, `melodify.servico` e `melodify.main` e inclua a classe `Dados` em `dados`. Crie o `enum Genero` e a interface `Reproduzivel` com `titulo()`, `duracaoSegundos()` e o **método padrão** `duracaoFormatada()`, que devolve `mm:ss` com dois dígitos em cada parte (os minutos podem passar de 59: 3600 s vira `60:00`).	1,0
2	Crie a classe abstrata `Midia`, que implementa `Reproduzivel` (duração ≤ 0 lança `IllegalArgumentException`). Duas mídias são iguais se tiverem o **mesmo título e o mesmo autor** (`equals/hashCode`). `toString`: `Thriller - Michael Jackson [05:57]`. Implemente `Musica` (descrição `Musica de <autor> (<GENERO>)`) e `Podcast` (descrição `Podcast de <autor>, episodio <n>`).	1,0
3	Crie a interface funcional genérica `Filtro<T>` e a classe utilitária `Filtros` (não instanciável) com o **método genérico estático** `filtrar`, cujo tipo `T` só aceita `Reproduzivel`. Ele devolve uma nova lista com os itens aceitos pelo filtro.	1,0
4	Crie `Playlist`: guarda mídias em estrutura que **não aceita repetidas e preserva a ordem de inserção**. Implemente `adicionar` (devolve se entrou), `adicionarVarias(Midia...)` (devolve **quantas efetivamente entraram**), `remover`, `tamanho`, `iterador()` (**sem expor** a estrutura interna) e `filtrar(Filtro<Midia>)`, que reutiliza `Filtros`.	1,5
5	Em `Playlist`, implemente `duracaoTotal()` (soma em segundos, **usando streams**) e `duracaoTotalFormatada()` (`mm:ss`).	0,5
6	Crie `Catalogo`. Seu construtor chama `carregar()`, que lê `Dados`, separa os campos e cria `Musica` ou `Podcast`. Mantenha um mapa título → mídia (preservando a ordem de leitura) e um mapa gênero → lista de músicas. Implemente `total()`, `buscarPorTitulo` (`null` se não existir), `musicasDoGenero` (devolve **cópia**, lista vazia se o gênero não tiver músicas) e `generosDisponiveis()` (um conjunto **na ordem de declaração do enum**).	1,5
7	Crie `Usuario` e a exceção **verificada** `PlaylistInexistenteException` (`Playlist inexistente: <nome>`). `criarPlaylist(nome)` cria e devolve a playlist (se já existir, lança `IllegalArgumentException` com `Playlist ja existe: <nome>`). `getPlaylist(nome)` lança a exceção verificada se não existir. As playlists ficam em um mapa que preserva a ordem de criação.	1,0
8	Em `Usuario`: `reproduzir(Midia)` registra a reprodução (contador por mídia) e devolve `Tocando: <título> [<mm:ss>]`; `reproduzirPlaylist(nome)` toca, **em ordem**, todas as mídias da playlist e devolve o **total de segundos**; `vezesTocada(Midia)`; e `maisTocadas(int n)`, que devolve **até n** mídias, por número de reproduções **decrescente** e, em empate, por título **crescente**, considerando só as já tocadas.	1,5
9	Crie a classe utilitária `ResumoPlaylist` (não instanciável) com `static String emitir(Playlist)`, que monta o texto com `StringBuilder` no formato do **Quadro A**. Cada faixa usa `String.format("%2d. %s%n", posição, midia)`.	0,5
10	Crie `Principal` e execute este roteiro: (i) criar o catálogo e imprimir o total, os gêneros disponíveis, as músicas de ROCK e a quantidade de músicas de JAZZ e de SERTANEJO; (ii) criar o usuário `Ana` e a playlist `Treino` e adicionar de uma só vez `Harder Better Faster`, `Thriller`, `Thriller` e `Another One Bites the Dust`, imprimindo quantas entraram; tentar adicionar `Thriller` de novo e imprimir o resultado; imprimir o resumo; (iii) tentar criar `Treino` de novo e	0,5

	tratar o erro; reproduzir a playlist duas vezes imprimindo o total de segundos ouvidos; tentar reproduzir `Estudo` e tratar a exceção; (iv) reproduzir `Flow #42` e `Thriller` e imprimir as 3 mais tocadas; (v) filtrar da playlist as mídias com mais de 250 s e, usando `Filtros` direto, as músicas de ROCK com menos de 300 s (**ambos com lambda**); imprimir a descrição do `Nerdcast #10`.	
😈	`Playlist` ou `Usuario` expondo suas estruturas internas por getters, ou `Playlist` usando uma lista que admite mídias repetidas.	-1,0
😈	Não seguir as orientações de criação do projeto ou imprimir fora de `Principal` nas classes de domínio.	-1,0

Quadro A: formato do resumo da playlist (o traço tem 40 caracteres)

```
========== PLAYLIST: Treino ==========
 1. Harder Better Faster - Daft Punk [03:44]
 2. Thriller - Michael Jackson [05:57]
 3. Another One Bites the Dust - Queen [03:35]
----------------------------------------
Faixas: 3 | Duracao total: 13:16
```

Quadro B: execução esperada do roteiro do item 10 (o texto das suas linhas de `Principal` é livre; os **valores** devem coincidir)

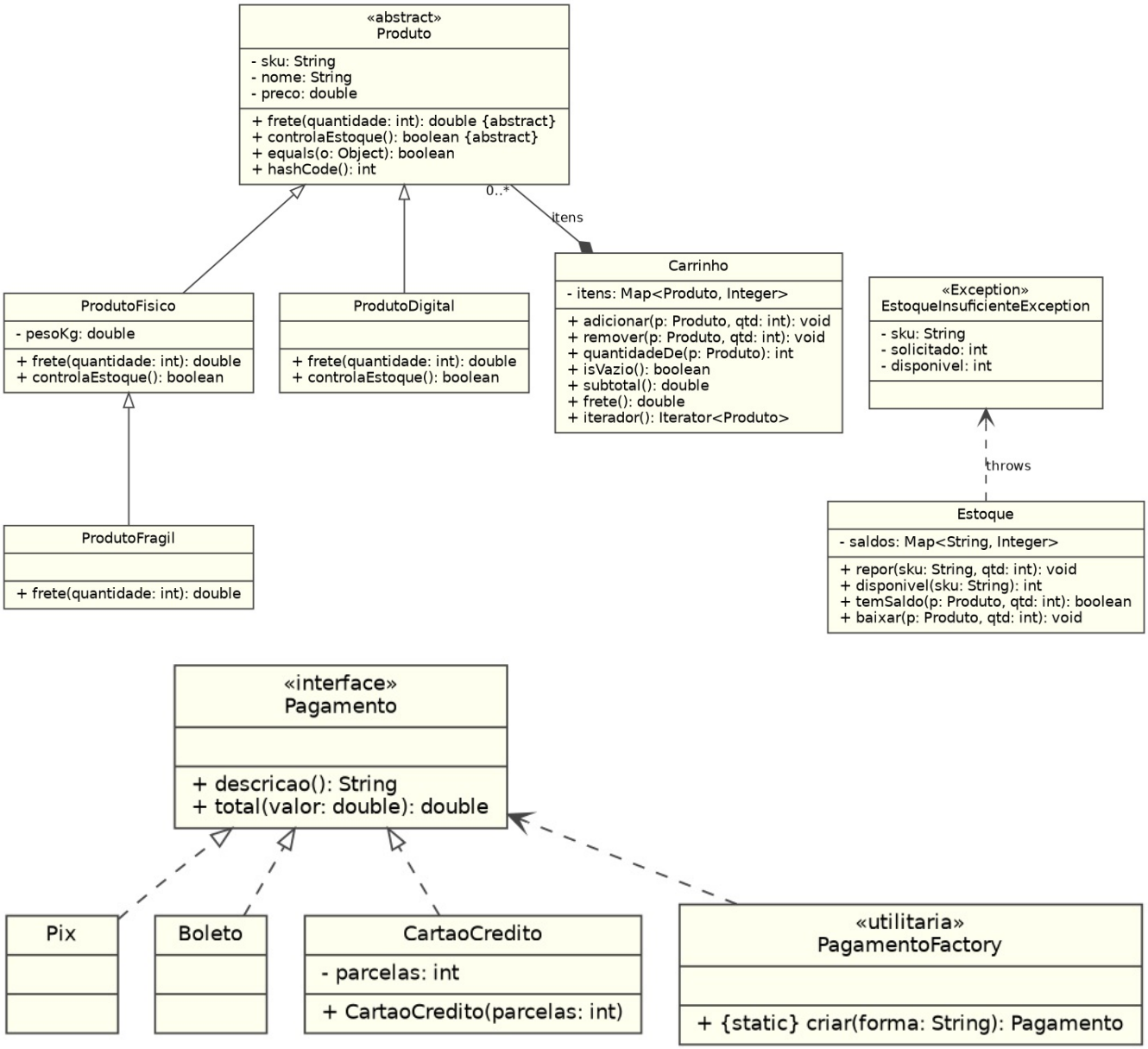
```
Midias no catalogo: 10
Generos: [ROCK, POP, SERTANEJO, JAZZ, ELETRONICA]
Rock: [Bohemian Rhapsody - Queen [05:55], Another One Bites the Dust - Queen [03:35], Hotel California - Eagles [06:31]]
Jazz: 1 | Sertanejo: 1
Adicionadas de uma vez: 3
Repetida entra? false
========== PLAYLIST: Treino ==========
 1. Harder Better Faster - Daft Punk [03:44]
 2. Thriller - Michael Jackson [05:57]
 3. Another One Bites the Dust - Queen [03:35]
----------------------------------------
Faixas: 3 | Duracao total: 13:16
Erro: Playlist ja existe: Treino
Segundos ouvidos: 1592
Erro: Playlist inexistente: Estudo
Tocando: Flow #42 [60:00]
Tocando: Thriller [05:57]
Mais tocadas: [Thriller - Michael Jackson [05:57], Another One Bites the Dust - Queen [03:35], Harder Better Faster - Daft Punk
[03:44]]
Longas da playlist: [Thriller - Michael Jackson [05:57]]
Rock curto: [Another One Bites the Dust - Queen [03:35]]
Podcast de Jovem Nerd, episodio 10
```

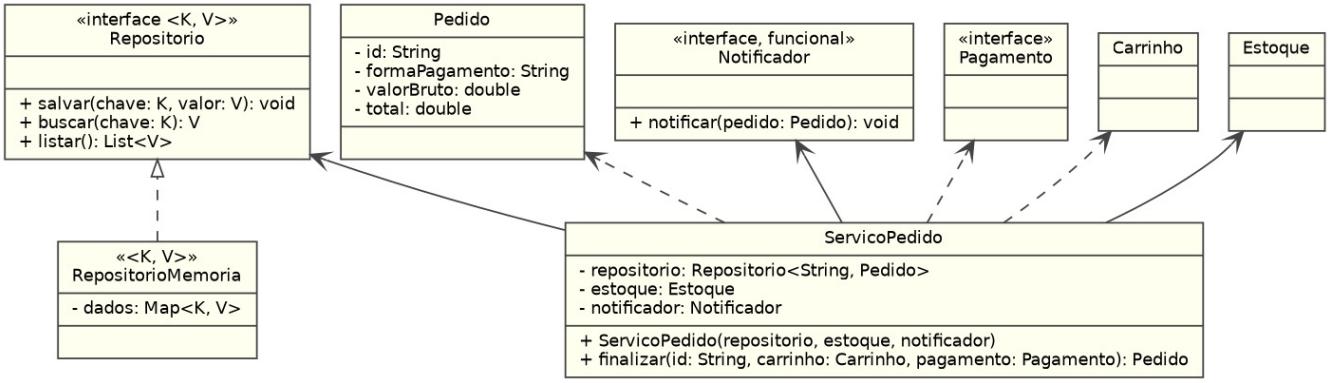
Prova Simulada 3: Programação Orientada a Objetos

Tema: Loja Virtual (estratégias de pagamento, fábrica e injeção de dependência)

✔ A prova é individual e sem consulta.	✔ Nomeie o projeto como `PRONTUARIO_SIM3` e envie todo o projeto.
✔ Não é permitido usar código implementado por você anteriormente.	✔ Não coloque seu nome no projeto.
✔ Atribui-se nota zero à prova em desacordo com os itens acima.	✔ Tempo sugerido: 3h30.

Uma loja virtual vende produtos **físicos**, **frágeis** e **digitais**. O cliente monta um **carrinho**, escolhe uma **forma de pagamento** e finaliza o **pedido**. A loja controla o estoque dos produtos que existem fisicamente, mas não o dos digitais. O serviço que finaliza o pedido contém a regra de negócio e **não pode depender de classes concretas** de pagamento, de persistência ou de notificação: tudo chega a ele por **interfaces**, injetadas pelo construtor. Isso permite trocar o repositório, a forma de pagamento ou o aviso sem alterar o serviço.





Para a atribuição da nota serão consideradas funcionalidade, qualidade, adequação e pertinência. Você pode criar métodos auxiliares, principalmente privados.

#	Descrição	Pontos
1	Crie o projeto com os pacotes `loja.modelo`, `loja.pagamento`, `loja.servico` e `loja.main`. Em `loja.modelo` crie `Produto` (abstrata): `sku`, `nome` e `preco` (preço negativo lança `IllegalArgumentException`); `frete(int quantidade)` e `controlaEstoque()` abstratos; `equals`/`hashCode` pelo **sku, qualquer que seja a subclasse**; `toString` no formato `Livro de Java (LIV-1)`. Crie `ProdutoFisico` (peso em kg; frete = `peso × quantidade × 3,00`; controla estoque), `ProdutoFragil` (herda de `ProdutoFisico`; frete = frete do físico **mais 15,00 fixos**, independentemente da quantidade, reutilizando o cálculo da superclasse) e `ProdutoDigital` (frete 0; **não** controla estoque).	1,5
2	Em `loja.pagamento`, crie a interface `Pagamento` (`descricao()` e `total(double valor)`, o valor a pagar para um valor bruto). Implemente `Pix` (5% de desconto; descrição `PIX`), `CartaoCredito` (parcelas de 1 a 12, senão `IllegalArgumentException` com `Parcelas devem estar entre 1 e 12: <n>`; total = `valor × (1 + 0,02 × (parcelas − 1))`; descrição `CARTAO em <n>x`) e `Boleto` (soma 3,50; descrição `BOLETO`).	1,0
3	Crie `PagamentoFactory` (classe utilitária não instanciável) com `static Pagamento criar(String forma)`. Ignora maiúsculas/minúsculas e aceita `pix`, `boleto` e `cartao:N` (por exemplo, `cartao:6`). O tipo de retorno deve ser a **interface**. Erros, sempre com `IllegalArgumentException`: forma nula (`Forma de pagamento nula`), desconhecida (`Forma de pagamento desconhecida: <forma>`), cartão sem parcelas (`Informe as parcelas: cartao:N`) e parcelas não numéricas (`Parcelas invalidas: <texto>`). Para o último, **capture** a `NumberFormatException` e lance a exceção do enunciado.	1,0
4	Crie `Carrinho`, com um **mapa** de produto para quantidade que preserva a ordem de inclusão. `adicionar(Produto, int)` **acumula** quantidades (quantidade ≤ 0 lança `IllegalArgumentException`: `Quantidade deve ser positiva: <q>`). `remover(Produto, int)` lança `IllegalArgumentException` (`Nao e possivel remover <q> de <produto>`) se a quantidade for ≤ 0 ou maior que a existente, e remove o produto do mapa quando a quantidade chega a zero. Implemente também `quantidadeDe`, `isVazio`, `subtotal()` (preço × quantidade), `frete()` (soma do frete de cada linha) e `Iterator<Produto> iterador()` sobre uma **cópia** das chaves.	1,5
5	Crie `EstoqueInsuficienteException` (**verificada**; guarda `sku`, `solicitado` e `disponivel`; mensagem `Estoque insuficiente de <sku>: pedido <q>, disponivel <d>`) e `Estoque`, que guarda saldos por sku: `repor(sku, qtd)` (qtd ≤ 0 lança `IllegalArgumentException`), `disponivel(sku)` (0 se desconhecido), `temSaldo(Produto, int)` (**produtos que não controlam estoque sempre têm saldo**) e `baixar(Produto, int)` (lança a exceção se faltar; para produto que não controla estoque, não faz nada).	1,0
6	Em `loja.servico`, crie a interface genérica `Repositorio<K, V>` com `void salvar(K, V)`, `V buscar(K)` (`null` se não existir) e `List<V> listar()`, e a implementação em memória `RepositorioMemoria<K, V>`: salvar com chave existente **substitui**; listar devolve uma **cópia** na ordem de inserção.	1,0
7	Crie `Pedido` (`id`, forma de pagamento, valor bruto e total; `toString` no formato `Pedido P-001 | CARTAO em 3x | bruto=585.50 | total=608.92`) e a **interface funcional** `Notificador`, com `void notificar(Pedido)`.	0,5
8	Crie `ServicoPedido`, cujo construtor recebe `Repositorio<String, Pedido>`, `Estoque` e `Notificador`. Implemente `Pedido finalizar(String id, Carrinho carrinho, Pagamento pagamento)`, que pode lançar `EstoqueInsuficienteException`, nesta ordem: (a) carrinho vazio lança `IllegalStateException` (`Carrinho vazio`); (b) pedido com o mesmo id já salvo lança `IllegalArgumentException` (`Pedido ja existe: <id>`); (c) **verifica o saldo de todos os itens antes de baixar qualquer um**: se algum faltar, lança a exceção **sem alterar o estoque**; (d) baixa todos; (e) valor bruto = subtotal + frete; total = `pagamento.total(bruto)`; (f) salva o pedido, notifica e o devolve. A classe **não pode mencionar** `Pix`, `Boleto`, `CartaoCredito` nem `RepositorioMemoria`.	2,0
9	Crie `Principal` e execute este roteiro: (i) criar um livro físico (`LIV-1`, 80,00, 0,5 kg), um vaso frágil (`VID-1`, 120,00, 2,0 kg) e um curso digital (`CUR-1`, 200,00); repor 5 livros e 1 vaso; criar o serviço com `RepositorioMemoria` e um **notificador por lambda** que imprime `[NOTIFICACAO] <id> confirmado`; (ii) montar um carrinho adicionando livro ×2, vaso ×1, curso ×1 e livro ×1, removendo o curso ×1 e adicionando-o de novo; imprimir a quantidade de livros (consultando com **outro objeto** de mesmo sku) e o subtotal e frete; (iii) para cada forma em `pix`, `cartao:6`, `boleto`, `cartao:abc`, `cartao:20` e `cripto`, imprimir a descrição e o total de 100,00 ou a mensagem de erro; (iv) finalizar `P-001` com `cartao:3` e imprimir o pedido; em seguida, tentar finalizar `P-002` com um carrinho de livro ×1 e vaso ×1 pagos com `pix`, tratando a falha; imprimir o estoque restante e a quantidade de pedidos salvos e se `P-002` existe; por fim, tentar finalizar `P-003` com carrinho vazio e tratar o erro.	0,5
😈	`ServicoPedido` mencionando classes concretas de pagamento, repositório ou notificador (violação do item 8).	-1,0
😈	Nas classes de domínio, imprimir fora de `Principal` ou não seguir as orientações de criação do projeto.	-1,0

Quadro A: execução esperada do roteiro do item 9 (o texto das suas linhas de `Principal` é livre; os **valores** devem coincidir)

```
livros no carrinho = 3
subtotal=560.00 frete=25.50
pix         -> PIX | total de 100.00 = 95.00
cartao:6    -> CARTAO em 6x | total de 100.00 = 110.00
boleto      -> BOLETO | total de 100.00 = 103.50
cartao:abc -> erro: Parcelas invalidas: abc
cartao:20 -> erro: Parcelas devem estar entre 1 e 12: 20
cripto -> erro: Forma de pagamento desconhecida: cripto
[NOTIFICACAO] P-001 confirmado
Pedido P-001 | CARTAO em 3x | bruto=585.50 | total=608.92
Falha: Estoque insuficiente de VID-1: pedido 1, disponivel 0
Estoque LIV-1 = 2, VID-1 = 0
Pedidos salvos = 1
P-002 existe? false
Erro: Carrinho vazio
```

Prova Simulada 4: Programação Orientada a Objetos

Tema: Secretaria Acadêmica (notas, turmas e boletins)

✔ A prova é individual e sem consulta.	✔ Nomeie o projeto como `PRONTUARIO_SIM4` e envie todo o projeto.
✔ Não é permitido usar código não fornecido junto com a prova.	✔ Não coloque seu nome no projeto.
✔ Atribui-se nota zero à prova em desacordo com os itens acima.	✔ Tempo sugerido: 3h30.

O sistema acadêmico de um instituto precisa carregar notas de uma planilha exportada, montar a turma, classificar os alunos e emitir boletins. **Pessoas** podem ser apenas **alunos** ou **professores**, e nenhuma outra classe pode herdar de `Pessoa` (use herança controlada). **Alunos** e **turmas** são **avaliáveis**: ambos têm uma média. A planilha exportada é imperfeita: algumas linhas estão incompletas, com notas inválidas ou fora do intervalo. O importador deve **aproveitar as linhas boas e relatar as ruins**, sem interromper o processamento. Um colega já escreveu `Utilitario`, que devolve as linhas da planilha. Você não deve alterá-la.

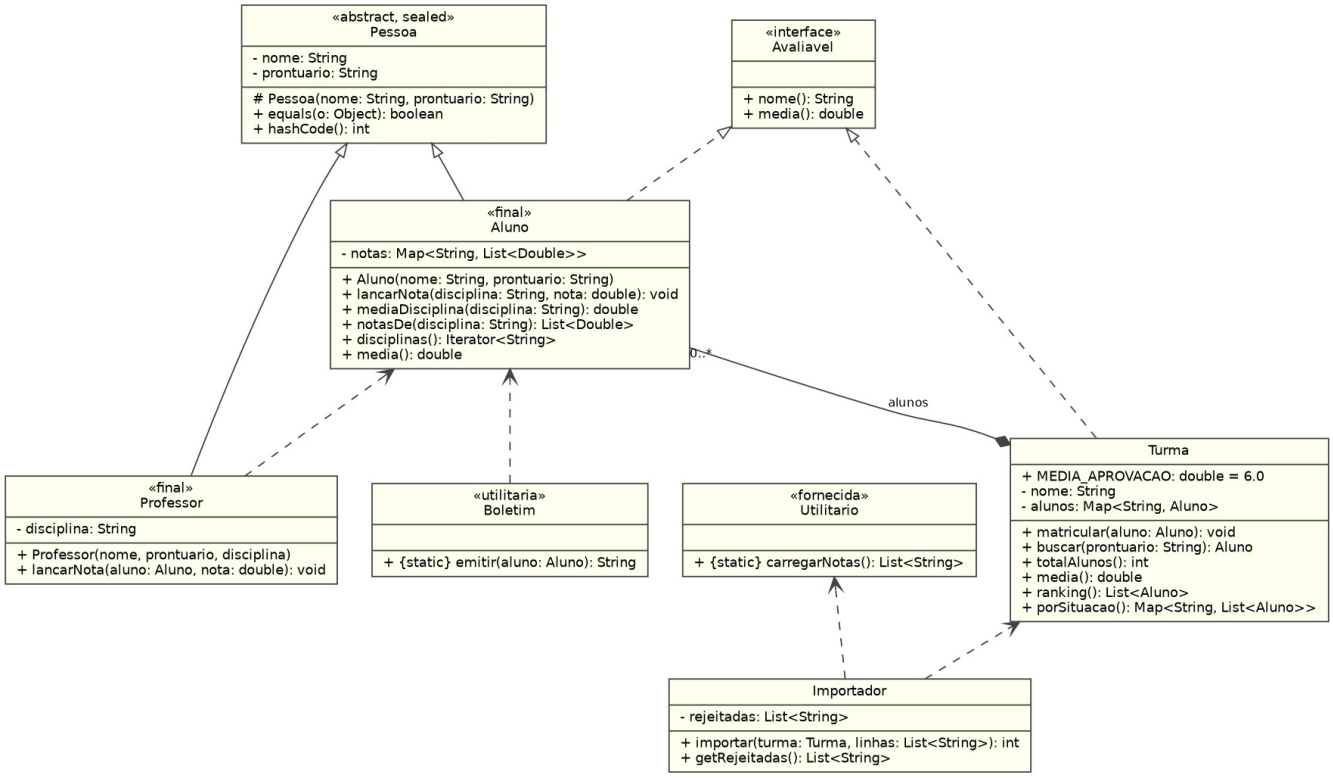
```
package sia.dados;

import java.util.List;

public final class Utilitario {
    private Utilitario() { }

    public static List<String> carregarNotas() {
        return List.of(
            "SC3001;Ana Souza;POO;8.5",
            "SC3001;Ana Souza;POO;7.0",
            "SC3001;Ana Souza;BD;5.0",
            "SC3002;Bruno Lima;POO;4.0",
            "SC3002;Bruno Lima;POO;6.5",
            "SC3002;Bruno Lima;BD;9.0",
            "SC3003;Carla Dias;POO;10.0",
            "SC3003;Carla Dias;BD;abc",
            "SC3004;Davi Reis;POO",
            "SC3003;Carla Dias;BD;11.5",
            "SC3001;Ana Souza;BD;6.0",
            "SC3003;Carla Dias;BD;7.5",
            "",
            "SC3005;Eva Melo;POO;3.0",
            "SC3005;Eva Melo;BD;4.5"
        );
    }
}
```

Cada linha válida tem **4 campos** separados por `;`: `PRONTUÁRIO;NOME;DISCIPLINA;NOTA`.



Para a atribuição da nota serão consideradas funcionalidade, qualidade, adequação e pertinência. Você pode criar métodos auxiliares, principalmente privados.

#	Descrição	Pontos
1	Crie o projeto com os pacotes `sia.modelo`, `sia.dados`, `sia.servico` e `sia.main` e inclua `Utilitario` em `dados`. Em `modelo`, crie `Pessoa` (abstrata e **selada**, permitindo somente `Aluno` e `Professor`, ambas `final`). `Pessoa` guarda `nome` e `prontuario` (nulos ou em branco lançam `IllegalArgumentException` com `Nome e prontuario sao obrigatorios`). Duas pessoas são iguais se tiverem o **mesmo prontuário, qualquer que seja a subclasse**. `toString`: `Aluno Ana Souza (SC3001)`. `Professor` tem uma disciplina e o método `lancarNota(Aluno, double)`, que lança a nota **na disciplina do professor**.	1,5
2	Crie a interface `Avaliavel` com `String nome()` e `double media()`.	0,5
3	Crie `Aluno`, que implementa `Avaliavel`. As notas ficam em um mapa **ordenado pelo nome da disciplina**, que associa cada disciplina à sua lista de notas. `lancarNota(String, double)` lança `IllegalArgumentException` (`Nota fora do intervalo 0..10: <nota>`) se a nota não estiver entre 0 e 10. `mediaDisciplina(String)` lança `IllegalArgumentException` (`Sem notas em <disciplina>`) se não houver notas nela. `notasDe(String)` devolve uma **cópia** (lista vazia se não houver). `disciplinas()` devolve um `Iterator<String>` sobre uma **cópia**. `media()` é a **média das médias por disciplina** (0,0 se não houver notas).	1,5
4	Crie `Turma`, que implementa `Avaliavel`. Os alunos ficam em um mapa **ordenado por prontuário**. `matricular(Aluno)` lança `IllegalArgumentException` (`Aluno ja matriculado: <prontuario>`) se o prontuário já existir. Implemente `buscar` (`null` se não existir), `totalAlunos()` e `media()` (média das médias dos alunos, **usando streams**, 0,0 se a turma for vazia). Declare a constante pública `MEDIA_APROVACAO = 6.0`.	1,0
5	Em `Turma`, implemente `List<Aluno> ranking()`: **cópia** ordenada por média **decrescente** e, em empate, por nome.	1,0
6	Em `Turma`, implemente `Map<String, List<Aluno>> porSituacao()`, cujas chaves `"APROVADO"` (média ≥ `MEDIA_APROVACAO`) e `"REPROVADO"` sempre existem, **nessa ordem de iteração**, mesmo vazias. Cada lista segue a ordem do ranking.	1,0
7	Em `sia.servico`, crie `Importador` com `int importar(Turma turma, List<String> linhas)`, que devolve quantas linhas foram **aceitas**. Linhas inválidas **não interrompem** o processo e são registradas, com numeração iniciada em 1 e contando **todas** as linhas: `Linha <n>: esperados 4 campos, encontrados <k>`; `Linha <n>: nota nao numerica`; ou `Linha <n>: <mensagem da exceção>` quando a nota estiver fora do intervalo. **Uma linha rejeitada nunca pode matricular um aluno novo.** Trate cada situação com o `catch` da exceção apropriada. `getRejeitadas()` devolve uma **cópia**.	2,0
8	Crie `Boletim` (classe utilitária não instanciável) com `static String emitir(Aluno)`, que usa `StringBuilder` e o formato do **Quadro A**: cada disciplina com suas notas (uma casa decimal, separadas por espaço, montadas com `StringJoiner`) e média da disciplina; no fim, a média geral e a situação (`APROVADO` se ≥ 6,0, senão `REPROVADO`).	1,0
9	Crie `Principal` e execute este roteiro: (i) criar a turma `3A`, importar `Utilitario.carregarNotas()` e imprimir quantas linhas foram aceitas, quantos alunos foram matriculados e cada linha rejeitada; (ii) criar o professor `Lucas` (prontuário `PF001`, disciplina `POO`), lançar 9,0 e depois -1 em `Eva` (`SC3005`), tratando o erro, e imprimir a média dela em POO; (iii) imprimir a média da turma, o ranking (`%-12s %.2f`) e a situação (nomes separados por `;`); (iv) imprimir o boletim de `SC3001`; (v) tentar matricular um `Aluno("Outra Ana", "SC3001")` e tratar o erro.	0,5
😈	`Aluno` ou `Turma` expondo suas coleções internas por getters.	-1,0
😈	Imprimir fora de `Principal` nas classes de domínio, ou não seguir as orientações de criação do projeto.	-1,0

Quadro A: formato do boletim (a linha de traços tem 43 caracteres)

```
=========== BOLETIM: Ana Souza (SC3001) ===========
Disciplina   Notas                   Media
BD           5.0 6.0                  5.50
POO          8.5 7.0                  7.75
-------------------------------------------
Media geral: 6.63 | APROVADO
```

Quadro B: execução esperada do roteiro do item 9

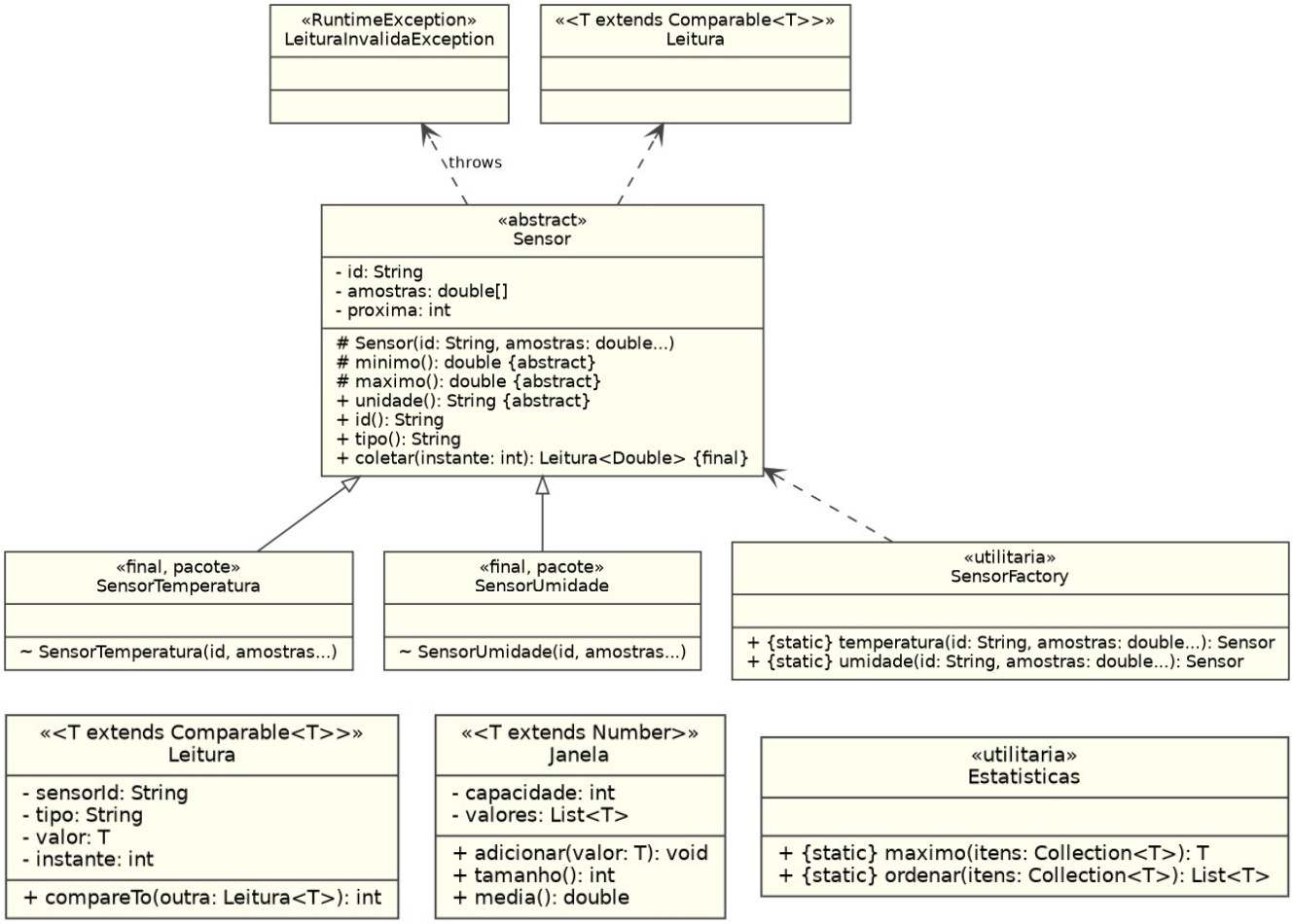
```
Linhas aceitas: 11
Alunos matriculados: 4
  rejeitada -> Linha 8: nota nao numerica
  rejeitada -> Linha 9: esperados 4 campos, encontrados 3
  rejeitada -> Linha 10: Nota fora do intervalo 0..10: 11.5
  rejeitada -> Linha 13: esperados 4 campos, encontrados 1
Erro: Nota fora do intervalo 0..10: -1.0
Media de Eva em POO: 6.0
Media da turma: 6.94
--- Ranking
Carla Dias   8.75
Bruno Lima   7.13
Ana Souza    6.63
Eva Melo     5.25
--- Situacao
APROVADO: Carla Dias; Bruno Lima; Ana Souza;
REPROVADO: Eva Melo;
=========== BOLETIM: Ana Souza (SC3001) ===========
Disciplina   Notas                   Media
BD           5.0 6.0                  5.50
POO          8.5 7.0                  7.75
-------------------------------------------
Media geral: 6.63 | APROVADO
Erro: Aluno ja matriculado: SC3001
```

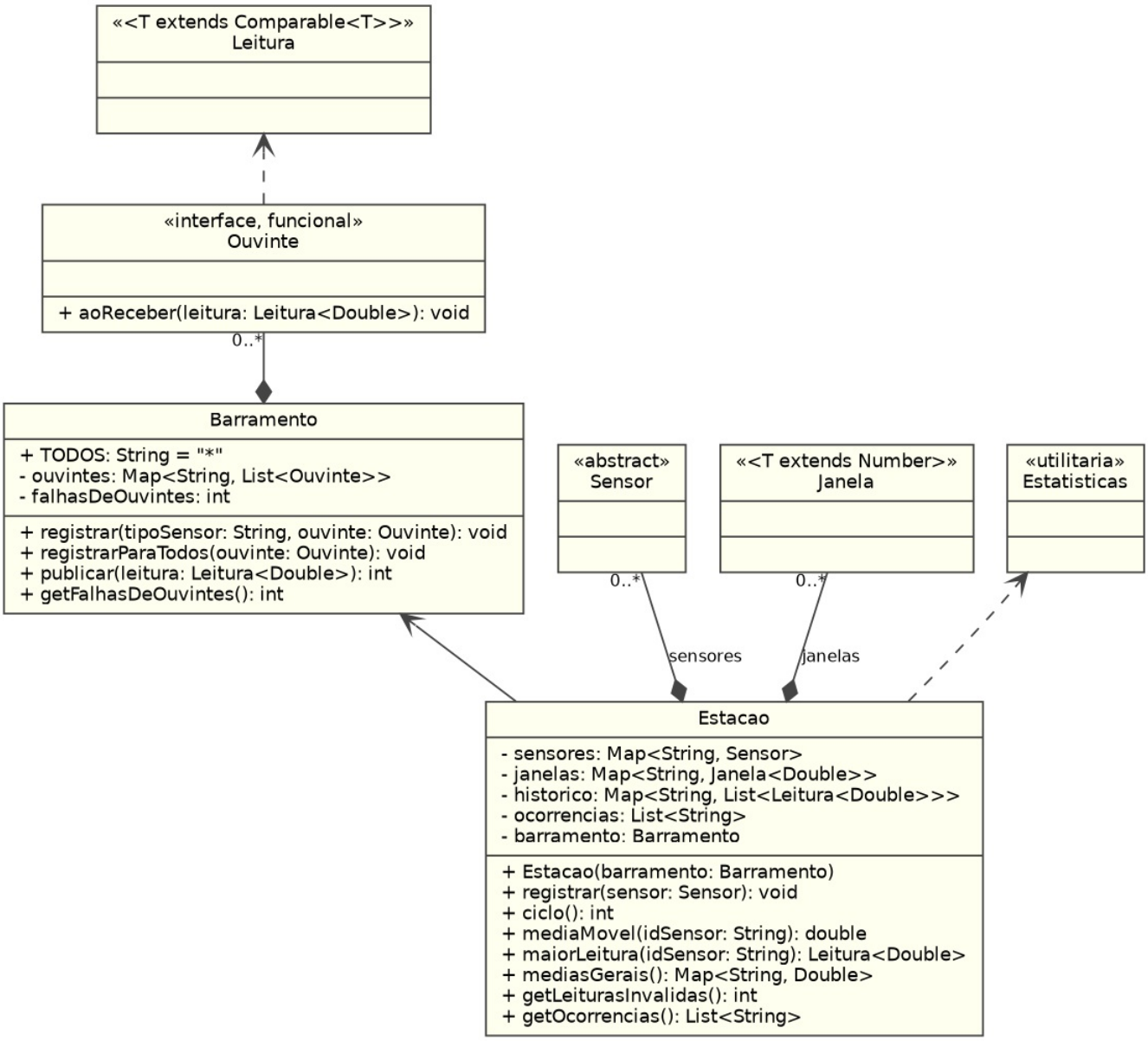
Prova Simulada 5: Programação Orientada a Objetos

Tema: Estação Meteorológica IoT (sensores, eventos e estatísticas)

✔ A prova é individual e sem consulta.	✔ Nomeie o projeto como `PRONTUARIO_SIM5` e envie todo o projeto.
✔ Não é permitido usar código implementado por você anteriormente.	✔ Não coloque seu nome no projeto.
✔ Atribui-se nota zero à prova em desacordo com os itens acima.	✔ Tempo sugerido: 3h30.

Uma estação meteorológica recebe dados de **sensores** de temperatura e umidade. A cada **ciclo**, cada sensor produz uma **leitura**. Leituras fora da faixa física do sensor são **inválidas**. A estação mantém o histórico de cada sensor e uma **janela móvel** com as últimas leituras, e **publica** cada leitura válida em um **barramento de eventos**, ao qual quaisquer interessados (um log, um alarme, um contador) podem se inscrever **sem que a estação os conheça**. Uma falha em um ouvinte não pode impedir que os demais sejam notificados. Para simular o hardware, cada sensor devolve, circularmente, os valores de um vetor de amostras recebido na construção.





Para a atribuição da nota serão consideradas funcionalidade, qualidade, adequação e pertinência. **Não utilize tipos genéricos sem parâmetro** (raw types). Você pode criar métodos auxiliares, principalmente privados.

#	Descrição	Pontos
1	Crie o projeto com os pacotes `estacao.modelo`, `estacao.eventos` e `estacao.main`. Em `modelo`, crie a classe genérica `Leitura<T extends Comparable<T>>`, que implementa `Comparable<Leitura<T>>`. Ela guarda `sensorId`, `tipo`, `valor` (do tipo `T`) e `instante`. Ordem natural: pelo valor e, em empate, pelo instante. `toString`: `SensorTemperatura[T1]=36.5 @t4`.	1,0
2	Crie `LeituraInvalidaException` (**não verificada**; mensagem `Leitura invalida de <id>: <valor>`) e a classe abstrata `Sensor`. O construtor (visível à hierarquia) recebe o `id` e as amostras como **varargs** de `double` e guarda uma **cópia** (sem amostras lança `IllegalArgumentException`: `Sensor sem amostras: <id>`). São abstratos `minimo()`, `maximo()` e `unidade()`. `tipo()` devolve o **nome simples da classe**. Implemente `public final Leitura<Double> coletar(int instante)` (**método-modelo**): usa a próxima amostra, **circularmente**, **avançando a posição mesmo se a amostra for inválida**. Se estiver fora de `[minimo, maximo]`, lança a exceção; senão devolve a leitura.	1,25
3	Crie `SensorTemperatura` (faixa −50 a 60; unidade `C`) e `SensorUmidade` (0 a 100; unidade `%`). Ambas são `final` e **só podem ser vistas e construídas dentro do pacote**. Crie `SensorFactory` (pública, não instanciável) com `temperatura(String id, double... amostras)` e `umidade(String id, double... amostras)`, ambos devolvendo `Sensor`.	1,0
4	Crie `Janela<T extends Number>`: capacidade fixa (≤ 0 lança `IllegalArgumentException`: `Capacidade deve ser positiva: <n>`). `adicionar(T)` **descarta o valor mais antigo** quando a capacidade é excedida. Implemente `tamanho()` e `media()` (0,0 se vazia), usando `doubleValue()`.	1,25
5	Crie `Estatisticas` (utilitária, não instanciável) com dois **métodos genéricos**: `<T extends Comparable<T>> T maximo(Collection<T>)` (coleção vazia lança `NoSuchElementException`: `Colecao vazia`) e `<T extends Comparable<T>> List<T> ordenar(Collection<T>)`, que devolve uma cópia ordenada.	0,75
6	Em `estacao.eventos`, crie a **interface funcional** `Ouvinte` (`void aoReceber(Leitura<Double>)`) e `Barramento`, que guarda ouvintes em um mapa cuja chave é o **tipo do sensor** (`tipo()`) ou `"*"` (todos). Implemente `registrar`, `registrarParaTodos` e `int publicar(Leitura<Double>)`. Ele notifica **primeiro** os ouvintes de `"*"` e **depois** os do tipo da	1,5

	leitura, cada grupo na ordem de registro, e devolve **quantos concluíram sem exceção**. Se um ouvinte lançar `RuntimeException`, o barramento **não propaga**, conta a falha (`getFalhasDeOuvintes()`) e segue para os demais.	
7	Em `modelo`, crie `Estacao`, cujo construtor recebe o `Barramento` (**injeção de dependência**). `registrar(Sensor)` lança `IllegalArgumentException` (`Sensor repetido: <id>`) se o id já existir; mantém os sensores em um mapa **ordenado por id** e cria, para cada um, uma `Janela<Double>` de tamanho 3 e um histórico de leituras. `int ciclo()` incrementa o instante (o primeiro ciclo tem instante 1), percorre os sensores **em ordem de id** e tenta coletar de cada um. Leitura válida vai para a janela, para o histórico e é **publicada**. Leitura inválida incrementa o contador de inválidas e registra a ocorrência `t<instante>: <mensagem da exceção>`. Devolve o número de leituras válidas do ciclo. A estação **não imprime nada**.	1,5
8	Em `Estacao`, implemente `mediaMovel(String id)` e `maiorLeitura(String id)` (que usa `Estatisticas.maximo`), ambos lançando `IllegalArgumentException` (`Sensor desconhecido: <id>`) para id inexistente; `Map<String, Double> mediasGerais()`, um `TreeMap` id → média de **todo o histórico**, calculada com **streams** (0,0 se não houver leituras); `getLeiturasInvalidas()`; e `getOcorrencias()`, que devolve uma cópia.	1,0
9	Crie `Principal` com `CICLOS = 6` e este roteiro: (i) registrar `temperatura("T1", 22.0, 25.5, 31.0, 36.5, 99.0, 28.0)` e `umidade("U1", 40, 55, 70, 105, 60, 45)`; (ii) inscrever: um ouvinte de **todos** os tipos (lambda que imprime `LOG <leitura>`); um ouvinte de `SensorTemperatura` (lambda que imprime `ALERTA calor: <valor>` se o valor passar de 35,0); outro ouvinte de `SensorTemperatura` (lambda que lança `IllegalStateException("ouvinte com defeito")` se o valor for 31,0); e um ouvinte de `SensorUmidade` implementado por **classe anônima**, que conta leituras por id de sensor em um `Map` local `final`; (iii) executar os ciclos, imprimindo `-- ciclo <i>: <n> leitura(s) valida(s)`; (iv) imprimir as médias móveis (`%.2f`), a maior leitura de cada sensor, as médias gerais (`%.2f`), as leituras inválidas com suas ocorrências, as falhas de ouvintes e a contagem de umidade; (v) pedir a média móvel de `X9` e tratar o erro.	0,75
😈	Usar tipos genéricos sem parâmetro (raw types) ou deixar `SensorTemperatura`, `SensorUmidade` ou seus construtores públicos.	-1,0
😈	`Estacao` ou `Barramento` imprimindo qualquer coisa, ou não seguir as orientações de criação do projeto.	-1,0

Quadro A: execução esperada do roteiro do item 9 (o texto das suas linhas de `Principal` é livre; os **valores** devem coincidir)

```
LOG    SensorTemperatura[T1]=22.0 @t1
LOG    SensorUmidade[U1]=40.0 @t1
-- ciclo 1: 2 leitura(s) valida(s)
LOG    SensorTemperatura[T1]=25.5 @t2
LOG    SensorUmidade[U1]=55.0 @t2
-- ciclo 2: 2 leitura(s) valida(s)
LOG    SensorTemperatura[T1]=31.0 @t3
LOG    SensorUmidade[U1]=70.0 @t3
-- ciclo 3: 2 leitura(s) valida(s)
LOG    SensorTemperatura[T1]=36.5 @t4
ALERTA calor: 36.5
-- ciclo 4: 1 leitura(s) valida(s)
LOG    SensorUmidade[U1]=60.0 @t5
-- ciclo 5: 1 leitura(s) valida(s)
LOG    SensorTemperatura[T1]=28.0 @t6
LOG    SensorUmidade[U1]=45.0 @t6
-- ciclo 6: 2 leitura(s) valida(s)
Media movel T1 = 31.83
Media movel U1 = 58.33
Maior T1 = SensorTemperatura[T1]=36.5 @t4
Maior U1 = SensorUmidade[U1]=70.0 @t3
Media geral T1 = 28.60
Media geral U1 = 54.00
Invalidas = 2 [t4: Leitura invalida de U1: 105.0, t5: Leitura invalida de T1: 99.0]
Falhas de ouvintes = 1
Contagem umidade = {U1=5}
Erro: Sensor desconhecido: X9
```

Parte III: Gabarito comentado dos exercícios

Os códigos abaixo foram compilados (`javac --release 17`) e executados, e as saídas conferem com as mostradas nos enunciados. Os `Principal` já constam nos enunciados e não são repetidos aqui. Existem outras soluções corretas; o que importa é atender aos requisitos.

Gabarito do Exercício 1: Ingressos de show

`Ingresso.java`

```
import java.util.Locale;

public class Ingresso {
    private final String evento;
    private final double valorBase;

    public Ingresso(String evento, double valorBase) {
        this.evento = evento;
        this.valorBase = valorBase;
    }

    public final String getEvento() {
        return evento;
    }

    public double getValorBase() {
        return valorBase;
    }

    public double valorFinal() {
        return valorBase;
    }

    @Override
    public String toString() {
        return String.format(Locale.US, "Ingresso[evento=%s, valorBase=%.2f]", evento, valorBase);
    }
}
```

`IngressoCamarote.java`

```
public final class IngressoCamarote extends IngressoVip {

    public IngressoCamarote(String evento, double valorBase, double taxaServico) {
        super(evento, valorBase, taxaServico);
    }

    @Override
    public double valorFinal() {
        return super.valorFinal() * 1.2;
    }

    @Override
    public String toString() {
        return super.toString() + " [CAMAROTE]";
    }
}
```

`IngressoMeia.java`

```
public class IngressoMeia extends Ingresso {
    private final String documento;

    public IngressoMeia(String evento, double valorBase, String documento) {
        super(evento, valorBase);
        this.documento = documento;
    }

    @Override
    public double valorFinal() {
        return super.valorFinal() / 2;
    }

    @Override
    public String toString() {
        return super.toString() + " [MEIA, doc=" + documento + "]";
    }
}
```

`IngressoVip.java`

```
public class IngressoVip extends Ingresso {
    private final double taxaServico;

    public IngressoVip(String evento, double valorBase, double taxaServico) {
```

```
        super(evento, valorBase);
        this.taxaServico = taxaServico;
    }

    public double getTaxaServico() {
        return taxaServico;
    }

    @Override
    public double valorFinal() {
        return super.valorFinal() + taxaServico;
    }

    @Override
    public String toString() {
        return super.toString() + " [VIP, taxa=" + taxaServico + "]";
    }
}
```

Decisões e comentários

- **Reuso por `super`.** `IngressoCamarote` não repete a fórmula do VIP. Ele chama `super.valorFinal()`, que por sua vez chama `Ingresso.valorFinal()`. O cálculo é (300 + 50) × 1,2 = 420. Se a taxa do VIP mudar, o camarote acompanha.
- **`final` com propósito.** `getEvento()` é `final` porque não foi projetado para ser sobrescrito. `IngressoCamarote` é `final` porque encerra a hierarquia (slides: "classes não projetadas para herança não devem permitir herança").
- **Polimorfismo.** No `Principal`, o vetor é `Ingresso[]`, mas `valorFinal()` e `toString()` executam a versão da **classe real** de cada objeto (despacho dinâmico).
- **Erro comum.** Esquecer `super(evento, valorBase)` nas subclasses: como `Ingresso` não tem construtor sem parâmetros, o código **não compila**.

Gabarito do Exercício 2: Central de notificações

`CentralNotificacoes.java`

```
public class CentralNotificacoes {
    private final Notificador[] canais;

    public CentralNotificacoes(Notificador... canais) {
        this.canais = canais.clone();
    }

    public int enviarParaTodos(String destinatario, String mensagem) {
        int enviados = 0;
        for (Notificador canal : canais) {
            if (canal.enviar(destinatario, mensagem)) {
                enviados++;
            }
        }
        return enviados;
    }

    public Notificador buscarCanal(String nome) {
        for (Notificador canal : canais) {
            if (canal.canal().equalsIgnoreCase(nome)) {
                return canal;
            }
        }
        return null;
    }
}
```

`Notificador.java`

```
public interface Notificador {
    String canal();

    boolean enviar(String destinatario, String mensagem);
}
```

`NotificadorConsole.java`

```
public class NotificadorConsole implements Notificador {

    @Override
    public String canal() {
        return "CONSOLE";
    }

    @Override
    public boolean enviar(String destinatario, String mensagem) {
        System.out.println("[CONSOLE] " + destinatario + " <- " + mensagem);
        return true;
    }
}
```

`NotificadorEmail.java`

```java
public class NotificadorEmail implements Notificador {

    @Override
    public String canal() {
        return "EMAIL";
    }

    @Override
    public boolean enviar(String destinatario, String mensagem) {
        if (destinatario == null || !destinatario.contains("@")) {
            return false;
        }
        System.out.println("[EMAIL] para " + destinatario + ": " + mensagem);
        return true;
    }
}
```

`NotificadorSms.java`

```java
public class NotificadorSms implements Notificador {

    @Override
    public String canal() {
        return "SMS";
    }

    @Override
    public boolean enviar(String destinatario, String mensagem) {
        if (destinatario == null || destinatario.length() != 11) {
            return false;
        }
        for (int i = 0; i < destinatario.length(); i++) {
            if (!Character.isDigit(destinatario.charAt(i))) {
                return false;
            }
        }
        System.out.println("[SMS] para " + destinatario + ": " + mensagem);
        return true;
    }
}
```

Decisões e comentários

- A interface define o **contrato**; cada canal o cumpre à sua maneira. `CentralNotificacoes` só conhece `Notificador`. Para criar um novo canal basta implementar a interface, como o `PUSH` anônimo do `Principal`.
- `canais.clone()` no construtor é uma **cópia defensiva**: se o chamador alterar o vetor que passou, a central não é afetada.
- **Varargs** (`Notificador...`) é só um vetor dentro do método. O chamador pode passar quantos argumentos quiser.
- As validações vêm **antes** de qualquer efeito (impressão), e `null` é testado antes de `contains`/`length`, para evitar `NullPointerException`.

Gabarito do Exercício 3: Agenda de tarefas

`Agenda.java`

```java
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Iterator;
import java.util.List;

public class Agenda {
    private final List<Tarefa> tarefas = new ArrayList<>();

    public boolean adicionar(Tarefa tarefa) {
        if (tarefas.contains(tarefa)) {
            return false;
        }
        return tarefas.add(tarefa);
    }

    public boolean remover(String titulo) {
        for (Iterator<Tarefa> it = tarefas.iterator(); it.hasNext(); ) {
            if (it.next().getTitulo().equals(titulo)) {
                it.remove();
                return true;
            }
        }
        return false;
    }

    public boolean concluir(String titulo) {
        for (Tarefa t : tarefas) {
            if (t.getTitulo().equals(titulo)) {
                t.concluir();
```

```
                return true;
            }
        }
        return false;
    }

    public Iterator<Tarefa> iterador() {
        return tarefas.iterator();
    }

    public List<Tarefa> ordenadas() {
        List<Tarefa> copia = new ArrayList<>(tarefas);
        Collections.sort(copia);
        return copia;
    }

    public List<Tarefa> porPrioridade() {
        List<Tarefa> copia = new ArrayList<>(tarefas);
        copia.sort((a, b) -> {
            int cmp = Integer.compare(b.getPrioridade(), a.getPrioridade());
            return cmp != 0 ? cmp : a.getPrazo().compareTo(b.getPrazo());
        });
        return copia;
    }

    public List<Tarefa> atrasadas(LocalDate hoje) {
        List<Tarefa> resultado = new ArrayList<>();
        for (Tarefa t : tarefas) {
            if (!t.isConcluida() && t.getPrazo().isBefore(hoje)) {
                resultado.add(t);
            }
        }
        return resultado;
    }
}
```

`Tarefa.java`

```
import java.time.LocalDate;
import java.util.Objects;

public class Tarefa implements Comparable<Tarefa> {
    private final String titulo;
    private final int prioridade;
    private final LocalDate prazo;
    private boolean concluida;

    public Tarefa(String titulo, int prioridade, LocalDate prazo) {
        this.titulo = titulo;
        this.prioridade = prioridade;
        this.prazo = prazo;
    }

    public String getTitulo() { return titulo; }
    public int getPrioridade() { return prioridade; }
    public LocalDate getPrazo() { return prazo; }
    public boolean isConcluida() { return concluida; }
    public void concluir() { this.concluida = true; }

    @Override
    public int compareTo(Tarefa outra) {
        int cmp = prazo.compareTo(outra.prazo);
        if (cmp != 0) return cmp;
        cmp = Integer.compare(outra.prioridade, prioridade);
        if (cmp != 0) return cmp;
        return titulo.compareTo(outra.titulo);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Tarefa tarefa = (Tarefa) o;
        return Objects.equals(titulo, tarefa.titulo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(titulo);
    }

    @Override
    public String toString() {
        return titulo + " (p" + prioridade + ", " + prazo + (concluida ? ", ok)" : ")");
    }
}
```

Decisões e comentários

- **Ordem do `compareTo`.** Prazo ascendente; em empate, `Integer.compare(outra.prioridade, prioridade)`, com os operandos **invertidos** para obter prioridade decrescente; por fim, o título.
- **`equals` por título × `compareTo` por prazo.** Aqui, duas tarefas com o mesmo título mas prazos diferentes são `equals`, porém `compareTo` não dá 0. Os slides recomendam consistência entre os dois. Neste exercício isso é aceitável porque só usamos `List` (`contains` usa `equals`). **Num `TreeSet` ou `TreeMap` isso causaria comportamento inesperado**, pois

eles decidem a igualdade por `compareTo`.
- **Encapsulamento.** Não há getter da lista. `ordenadas()` e `porPrioridade()` ordenam uma **cópia** (`new ArrayList<>(tarefas)`), e a ordem interna se preserva.
- **Remoção segura.** `remover` usa `Iterator.remove()`. Remover com `for-each` + `list.remove` lança `ConcurrentModificationException`.

Gabarito do Exercício 4: Contas e comandos inválidos

`Conta.java`

```
public class Conta {
    private final String titular;
    private double saldo;

    public Conta(String titular, double saldoInicial) {
        if (titular == null || titular.isBlank()) {
            throw new IllegalArgumentException("Titular obrigatorio");
        }
        if (saldoInicial < 0) {
            throw new IllegalArgumentException("Saldo inicial negativo");
        }
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public String getTitular() { return titular; }
    public double getSaldo() { return saldo; }

    public void depositar(double valor) {
        if (valor <= 0) {
            throw new IllegalArgumentException("Valor de deposito deve ser positivo: " + valor);
        }
        saldo += valor;
    }

    public void sacar(double valor) throws SaldoInsuficienteException {
        if (valor <= 0) {
            throw new IllegalArgumentException("Valor de saque deve ser positivo: " + valor);
        }
        if (valor > saldo) {
            throw new SaldoInsuficienteException(saldo, valor);
        }
        saldo -= valor;
    }

    public void transferir(Conta destino, double valor) throws SaldoInsuficienteException {
        if (destino == null) {
            throw new IllegalArgumentException("Destino nulo");
        }
        sacar(valor);          // se falhar, nada foi alterado
        destino.depositar(valor);
    }
}
```

`Processador.java`

```
import java.util.ArrayList;
import java.util.List;

public class Processador {
    private static int comandosProcessados = 0;

    public static int getComandosProcessados() { return comandosProcessados; }

    public static List<String> processar(Conta conta, String[] comandos) {
        List<String> erros = new ArrayList<>();
        for (String comando : comandos) {
            try {
                String[] partes = comando.split(":");
                if (partes.length != 2) {
                    throw new IllegalArgumentException("Formato invalido: " + comando);
                }
                double valor = Double.parseDouble(partes[1]);
                if (partes[0].equals("D")) {
                    conta.depositar(valor);
                } else if (partes[0].equals("S")) {
                    conta.sacar(valor);
                } else {
                    throw new IllegalArgumentException("Operacao desconhecida: " + partes[0]);
                }
            } catch (NumberFormatException e) {
                erros.add("Numero invalido em '" + comando + "'");
            } catch (SaldoInsuficienteException e) {
                erros.add(e.getMessage());
            } catch (IllegalArgumentException e) {
                erros.add(e.getMessage());
            } finally {
                comandosProcessados++;
            }
        }
        return erros;
```

```
    }
}
```

SaldoInsuficienteException.java

```
public class SaldoInsuficienteException extends Exception {
    private final double saldoAtual;
    private final double valorSolicitado;

    public SaldoInsuficienteException(double saldoAtual, double valorSolicitado) {
        super("Saldo insuficiente: saldo=" + saldoAtual + ", solicitado=" + valorSolicitado);
        this.saldoAtual = saldoAtual;
        this.valorSolicitado = valorSolicitado;
    }

    public double getSaldoAtual() { return saldoAtual; }
    public double getValorSolicitado() { return valorSolicitado; }
}
```

Decisões e comentários

- **Ordem dos `catch`.** `NumberFormatException` é **subclasse** de `IllegalArgumentException`. Por isso seu `catch` precisa vir **antes**. Invertendo, o código não compila ("exception already caught").
- **Verificada × não verificada.** `SaldoInsuficienteException extends Exception` é verificada: `sacar` e `transferir` declaram `throws`, e o chamador é obrigado a tratar. `IllegalArgumentException` é não verificada e indica uso incorreto.
- **`finally`** roda com sucesso ou falha, então `comandosProcessados` vale 8 mesmo havendo 5 falhas.
- **`transferir` atômico.** Chamar `sacar` primeiro garante que, se faltar saldo, a exceção sai **antes** do depósito, e nenhuma conta muda (a saída mostra os saldos `80.0 / 0.0`).
- `"D:20:30".split(":")` produz 3 partes, por isso a verificação `partes.length != 2` antes de converter o número.

Gabarito do Exercício 5: Corrigir e estender o estoque

Defeitos encontrados e correções

#	Defeito	Correção
1	`ProdutoEletronico` acessa `preco`, que é `private` em `Produto`: **erro de compilação**.	Usar `getPreco()`.
2	`codigo == outro.codigo` compara **referências**, não o texto. Strings montadas em tempo de execução são objetos distintos.	`Objects.equals(codigo, outro.codigo)`.
3	Não há `hashCode`. Num `HashMap`, o *hash* é consultado antes do `equals`. Dois produtos "iguais" caíam em baldes diferentes e viravam chaves distintas.	`hashCode` coerente com o `equals` (`Objects.hash(codigo)`).
4	`equals` usa `getClass()`: dois produtos de mesmo código e **subclasses diferentes** nunca seriam iguais, contra o requisito.	`instanceof Produto`.
5	`put(produto, quantidade)` **substitui** a quantidade anterior.	`getOrDefault(produto, 0) + quantidade`.
6	Remover dentro de um `for-each` do `keySet()` provoca `ConcurrentModificationException`.	`Iterator` sobre `entrySet()` e `it.remove()`.
7	`getQuantidades()` entrega o mapa interno a qualquer um.	Removido; criados `quantidadeDe` e `tiposDeProduto`.

Estoque.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

public class Estoque {
    private final Map<Produto, Integer> quantidades = new HashMap<>();

    public void adicionar(Produto produto, int quantidade) {
        int atual = quantidades.getOrDefault(produto, 0);
        quantidades.put(produto, atual + quantidade);
    }

    public int quantidadeDe(Produto produto) {
        return quantidades.getOrDefault(produto, 0);
    }

    public int tiposDeProduto() {
        return quantidades.size();
    }

    public void removerEsgotados() {
        Iterator<Map.Entry<Produto, Integer>> it = quantidades.entrySet().iterator();
        while (it.hasNext()) {
```

```
            if (it.next().getValue() == 0) {
                it.remove();
            }
        }
    }

    public double valorTotal() {
        double total = 0;
        for (Map.Entry<Produto, Integer> e : quantidades.entrySet()) {
            total += e.getKey().getPreco() * e.getValue();
        }
        return total;
    }

    public Produto buscarPorCodigo(String codigo) {
        for (Produto p : quantidades.keySet()) {
            if (p.getCodigo().equals(codigo)) {
                return p;
            }
        }
        return null;
    }

    public List<Produto> maisBaratosQue(double limite) {
        List<Produto> resultado = new ArrayList<>();
        for (Produto p : quantidades.keySet()) {
            if (p.getPreco() < limite) {
                resultado.add(p);
            }
        }
        resultado.sort(Comparator.comparingDouble(Produto::getPreco));
        return resultado;
    }

    public double freteTotal() {
        double total = 0;
        for (Produto p : quantidades.keySet()) {
            if (quantidades.get(p) > 0) {
                total += p.frete();
            }
        }
        return total;
    }

    public Iterator<Produto> iterador() {
        return new ArrayList<>(quantidades.keySet()).iterator();
    }
}
```

Produto.java

```
import java.util.Objects;

public abstract class Produto {
    private final String codigo;
    private final String nome;
    private final double preco;

    public Produto(String codigo, String nome, double preco) {
        this.codigo = codigo;
        this.nome = nome;
        this.preco = preco;
    }

    public abstract double frete();

    public String getCodigo() { return codigo; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Produto outro)) return false;
        return Objects.equals(codigo, outro.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo);
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + "[" + codigo + ", " + nome + ", " + preco + "]";
    }
}
```

ProdutoEletronico.java

```
public class ProdutoEletronico extends Produto {
    private int garantiaMeses;

    public ProdutoEletronico(String codigo, String nome, double preco, int garantiaMeses) {
        super(codigo, nome, preco);
        this.garantiaMeses = garantiaMeses;
```

```
    }

    public int getGarantiaMeses() { return garantiaMeses; }

    @Override
    public double frete() {
        return getPreco() * 0.02;
    }
}
```

ProdutoPerecivel.java

```
import java.time.LocalDate;

public class ProdutoPerecivel extends Produto {
    private LocalDate validade;

    public ProdutoPerecivel(String codigo, String nome, double preco, LocalDate validade) {
        super(codigo, nome, preco);
        this.validade = validade;
    }

    public LocalDate getValidade() { return validade; }

    @Override
    public double frete() {
        return 12.0;
    }
}
```

Comentários

- `maisBaratosQue` usa `Comparator.comparingDouble(Produto::getPreco)` (referência a método, Aula 07).
- `iterador()` percorre uma **cópia**: o chamador não consegue remover itens do estoque por ele.
- `freteTotal()` mostra polimorfismo: `p.frete()` executa a versão do perecível (12,00) ou do eletrônico (2% do preço).

Gabarito do Exercício 6: Ranking genérico

Equipe.java

```
import java.util.HashSet;
import java.util.Set;

public class Equipe implements Pontuavel {
    private final String nome;
    private final Set<Jogador> jogadores = new HashSet<>();

    public Equipe(String nome) {
        this.nome = nome;
    }

    public boolean adicionar(Jogador jogador) {
        return jogadores.add(jogador);
    }

    public int tamanho() {
        return jogadores.size();
    }

    @Override
    public String nome() { return nome; }

    @Override
    public int pontos() {
        int soma = 0;
        for (Jogador j : jogadores) {
            soma += j.pontos();
        }
        return soma;
    }

    @Override
    public String toString() { return nome + "[" + pontos() + "]"; }
}
```

Jogador.java

```
import java.util.Objects;

public class Jogador implements Pontuavel {
    private final String nome;
    private final int pontos;

    public Jogador(String nome, int pontos) {
        this.nome = nome;
        this.pontos = pontos;
    }

    @Override
```

```
    public String nome() { return nome; }

    @Override
    public int pontos() { return pontos; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return nome.equals(((Jogador) o).nome);
    }

    @Override
    public int hashCode() { return Objects.hash(nome); }

    @Override
    public String toString() { return nome + "(" + pontos + ")"; }
}
```

Pontuavel.java

```
public interface Pontuavel {
    String nome();

    int pontos();
}
```

Ranking.java

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.NoSuchElementException;

public final class Ranking {

    private Ranking() { }

    public static <T extends Pontuavel> List<T> melhores(Collection<T> itens, int n) {
        List<T> copia = new ArrayList<>(itens);
        copia.sort(Comparator.comparingInt((T t) -> t.pontos()).reversed()
                .thenComparing(t -> t.nome()));
        return new ArrayList<>(copia.subList(0, Math.min(n, copia.size())));
    }

    public static <T extends Pontuavel> T lider(Collection<T> itens) {
        if (itens.isEmpty()) {
            throw new NoSuchElementException("Nao ha participantes");
        }
        return melhores(itens, 1).get(0);
    }

    public static int somaPontos(Pontuavel... itens) {
        int soma = 0;
        for (Pontuavel p : itens) {
            soma += p.pontos();
        }
        return soma;
    }
}
```

Decisões e comentários

- `<T extends Pontuavel>` é um **limite superior**: só entram tipos que cumprem o contrato. Dentro do método, chamamos `t.pontos()` sem *cast*. `Jogador` e `Equipe` convivem com o mesmo `Ranking`.
- `Comparator.comparingInt((T t) -> t.pontos())` precisa de `(T t)` **explícito** porque, ao encadear `.reversed()`, o compilador perde a inferência do tipo do lambda.
- `subList` devolve uma **visão** da lista, não uma cópia independente. Por isso a envolvemos em `new ArrayList<>(...)`. A coleção original nunca é alterada.
- `Equipe` usa `Set<Jogador>`. `Jogador` define `equals`/`hashCode` por nome, então o segundo "Ana" é rejeitado (`adicionar` devolve `false`).
- `somaPontos(Pontuavel...)` aceita um `Jogador` e uma `Equipe` na mesma chamada, graças à interface.

Gabarito do Exercício 7: Regras de frete

FreteExpresso.java

```
public class FreteExpresso implements RegraFrete {
    private final RegraFrete base;

    public FreteExpresso(RegraFrete base) {
        this.base = base;
    }

    @Override
```

```
    public double calcular(Pacote pacote) {
        return base.calcular(pacote) * 1.8;
    }
}
```

FreteFactory.java

```
public final class FreteFactory {

    private FreteFactory() { }

    public static RegraFrete para(String tipo) {
        if (tipo == null) {
            throw new IllegalArgumentException("Tipo de frete nulo");
        }
        switch (tipo.toLowerCase()) {
            case "padrao":
                return new FretePadrao();
            case "expresso":
                return new FreteExpresso(new FretePadrao());
            case "gratis200":
                return new FreteGratisAcimaDe(200.0, new FretePadrao());
            case "retirada":
                return pacote -> 0.0;
            default:
                throw new IllegalArgumentException("Tipo de frete desconhecido: " + tipo);
        }
    }
}
```

FreteGratisAcimaDe.java

```
public class FreteGratisAcimaDe implements RegraFrete {
    private final double limite;
    private final RegraFrete base;

    public FreteGratisAcimaDe(double limite, RegraFrete base) {
        this.limite = limite;
        this.base = base;
    }

    @Override
    public double calcular(Pacote pacote) {
        if (pacote.getValorDeclarado() >= limite) {
            return 0.0;
        }
        return base.calcular(pacote);
    }
}
```

FretePadrao.java

```
public class FretePadrao implements RegraFrete {

    @Override
    public double calcular(Pacote pacote) {
        return pacote.getPesoKg() * 2.0 + pacote.getDistanciaKm() * 0.05;
    }
}
```

Pacote.java

```
public class Pacote {
    private final double pesoKg;
    private final double distanciaKm;
    private final double valorDeclarado;

    public Pacote(double pesoKg, double distanciaKm, double valorDeclarado) {
        this.pesoKg = pesoKg;
        this.distanciaKm = distanciaKm;
        this.valorDeclarado = valorDeclarado;
    }

    public double getPesoKg() { return pesoKg; }
    public double getDistanciaKm() { return distanciaKm; }
    public double getValorDeclarado() { return valorDeclarado; }
}
```

Pedido.java

```
public class Pedido {
    private final Pacote pacote;
    private final RegraFrete regra;

    public Pedido(Pacote pacote, RegraFrete regra) {
        this.pacote = pacote;
        this.regra = regra;
    }

    public double frete() {
```

```
        return regra.calcular(pacote);
    }

    public double total() {
        return pacote.getValorDeclarado() + frete();
    }
}
```

RegraFrete.java

```
@FunctionalInterface
public interface RegraFrete {
    double calcular(Pacote pacote);
}
```

Decisões e comentários

- **Composição de interfaces.** `FreteExpresso` e `FreteGratisAcimaDe` recebem outra `RegraFrete` e **delegam** a ela. Assim, regras se combinam sem herança: `new FreteExpresso(new FreteGratisAcimaDe(300.0, promo))`. Isso segue o **princípio da inversão de dependência** da Aula 06.
- `Pedido` depende apenas da interface. Trocar a regra não exige mudar `Pedido`.
- `FreteFactory` retorna a **interface** (retorno polimórfico). `retirada` é uma **lambda**, pois `RegraFrete` tem um único método abstrato (interface funcional).
- Conferência: na regra combinada para o pacote leve (valor 80 < 300), a promoção dá `min(10, 9,00) = 9,00` e o expresso multiplica por 1,8, resultando 16,20. No pacote caro (350 ≥ 300), o frete é 0.

Gabarito do Exercício 8: Biblioteca

Biblioteca.java

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class Biblioteca {
    private static final int LIMITE_EMPRESTIMOS = 3;

    private final Map<String, Item> acervo = new LinkedHashMap<>();
    private final Map<String, List<Item>> emprestimos = new LinkedHashMap<>();

    public void cadastrar(Item item) {
        if (acervo.containsKey(item.getCodigo())) {
            throw new IllegalArgumentException("Codigo ja cadastrado: " + item.getCodigo());
        }
        acervo.put(item.getCodigo(), item);
    }

    public void emprestar(String usuario, String codigo) throws ItemIndisponivelException {
        Item item = acervo.get(codigo);
        if (item == null) {
            throw new ItemIndisponivelException("Item inexistente: " + codigo);
        }
        if (item.isEmprestado()) {
            throw new ItemIndisponivelException("Item ja emprestado: " + codigo);
        }
        List<Item> doUsuario = emprestimos.get(usuario);
        if (doUsuario == null) {
            doUsuario = new ArrayList<>();
            emprestimos.put(usuario, doUsuario);
        }
        if (doUsuario.size() >= LIMITE_EMPRESTIMOS) {
            throw new IllegalStateException("Limite de " + LIMITE_EMPRESTIMOS + " emprestimos atingido por " + usuario);
        }
        item.setEmprestado(true);
        doUsuario.add(item);
    }

    public double devolver(String usuario, String codigo, int diasComItem) {
        List<Item> doUsuario = emprestimos.get(usuario);
        Item item = acervo.get(codigo);
        if (doUsuario == null || item == null || !doUsuario.remove(item)) {
            throw new IllegalArgumentException(usuario + " nao possui o item " + codigo);
        }
        item.setEmprestado(false);
        return item.calcularMulta(diasComItem);
    }

    public List<Item> emprestadosDe(String usuario) {
        List<Item> lista = emprestimos.get(usuario);
        return lista == null ? new ArrayList<>() : new ArrayList<>(lista);
    }

    public String relatorio() {
        StringBuilder sb = new StringBuilder();
        for (Item item : acervo.values()) {
            sb.append(String.format("%-6s | %-8s | %-22s | %s%n", item.getCodigo(),
                    item.getClass().getSimpleName(), item.getTitulo(),
```

```
                    item.isEmprestado() ? "EMPRESTADO" : "DISPONIVEL"));
        }
        return sb.toString();
    }
}
```

Dvd.java

```
public class Dvd extends Item {
    public Dvd(String codigo, String titulo) { super(codigo, titulo); }

    @Override public int prazoDias() { return 3; }
    @Override public double multaPorDia() { return 3.0; }

    @Override
    public double calcularMulta(int diasComItem) {
        double base = super.calcularMulta(diasComItem);
        return base > 0 ? base + 5.0 : 0.0;
    }
}
```

Emprestavel.java

```
public interface Emprestavel {
    int prazoDias();

    double multaPorDia();
}
```

Item.java

```
public abstract class Item implements Emprestavel {
    private final String codigo;
    private final String titulo;
    private boolean emprestado;

    protected Item(String codigo, String titulo) {
        this.codigo = codigo;
        this.titulo = titulo;
    }

    public String getCodigo() { return codigo; }
    public String getTitulo() { return titulo; }
    public boolean isEmprestado() { return emprestado; }
    void setEmprestado(boolean emprestado) { this.emprestado = emprestado; }

    public double calcularMulta(int diasComItem) {
        int atraso = diasComItem - prazoDias();
        return atraso > 0 ? atraso * multaPorDia() : 0.0;
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + "[" + codigo + " - " + titulo + "]";
    }
}
```

ItemIndisponivelException.java

```
public class ItemIndisponivelException extends Exception {
    public ItemIndisponivelException(String mensagem) {
        super(mensagem);
    }
}
```

Livro.java

```
public class Livro extends Item {
    public Livro(String codigo, String titulo) { super(codigo, titulo); }

    @Override public int prazoDias() { return 14; }
    @Override public double multaPorDia() { return 1.0; }
}
```

Revista.java

```
public class Revista extends Item {
    public Revista(String codigo, String titulo) { super(codigo, titulo); }

    @Override public int prazoDias() { return 7; }
    @Override public double multaPorDia() { return 0.5; }
}
```

Decisões e comentários

- `setEmprestado` **sem modificador** (visibilidade de pacote): só `Biblioteca`, no mesmo pacote, muda o estado de empréstimo. Código de fora não pode "devolver" um item sem passar pela biblioteca.

- `Dvd.calcularMulta` **sobrescreve** e reaproveita `super.calcularMulta(...)`. Só soma a taxa de 5,00 se houve atraso: 5 dias com prazo de 3 dão `2 × 3,00 + 5,00 = 11,00`.
- O mapa `emprestimos` é `Map<String, List<Item>>`. A lista do usuário é criada **sob demanda** (`get` devolve `null` na primeira vez).
- **Ordem das verificações** em `emprestar`: existência, disponibilidade e só então o limite. Nenhuma alteração é feita antes de todas passarem.
- `emprestadosDe` devolve **cópia**. `relatorio()` usa `StringBuilder` e `%n`.
- `ItemIndisponivelException` é verificada (situação esperada do negócio). O limite de 3 usa `IllegalStateException` (estado do usuário incompatível com a operação).

Gabarito do Exercício 9: Barramento de eventos

`Barramento.java`

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Barramento {
    public static final String TODOS = "*";

    private final Map<String, List<Ouvinte>> ouvintes = new HashMap<>();

    public void registrar(String tipoEvento, Ouvinte ouvinte) {
        if (!ouvintes.containsKey(tipoEvento)) {
            ouvintes.put(tipoEvento, new ArrayList<>());
        }
        ouvintes.get(tipoEvento).add(ouvinte);
    }

    public void registrarParaTodos(Ouvinte ouvinte) {
        registrar(TODOS, ouvinte);
    }

    public int publicar(Evento evento) {
        List<Ouvinte> destinatarios = new ArrayList<>();
        if (ouvintes.containsKey(TODOS)) {
            destinatarios.addAll(ouvintes.get(TODOS));
        }
        String tipo = evento.getClass().getSimpleName();
        if (ouvintes.containsKey(tipo)) {
            destinatarios.addAll(ouvintes.get(tipo));
        }
        int notificados = 0;
        for (Ouvinte o : destinatarios) {
            try {
                o.aoOcorrer(evento);
                notificados++;
            } catch (RuntimeException e) {
                System.out.println("  [falha em ouvinte: " + e.getMessage() + "]");
            }
        }
        return notificados;
    }
}
```

`ContadorEventos.java`

```java
import java.util.Map;
import java.util.TreeMap;

public class ContadorEventos implements Ouvinte {
    private final Map<String, Integer> contagem = new TreeMap<>();

    @Override
    public void aoOcorrer(Evento evento) {
        String tipo = evento.getClass().getSimpleName();
        contagem.put(tipo, contagem.getOrDefault(tipo, 0) + 1);
    }

    public Map<String, Integer> getContagem() {
        return new TreeMap<>(contagem);
    }
}
```

`Evento.java`

```java
public abstract class Evento {
    public abstract String descricao();

    @Override
    public String toString() {
        return getClass().getSimpleName() + ": " + descricao();
    }
}
```

`EventoCompra.java`

```java
public class EventoCompra extends Evento {
    private final String usuario;
    private final double valor;

    public EventoCompra(String usuario, double valor) {
        this.usuario = usuario;
        this.valor = valor;
    }

    public String getUsuario() { return usuario; }
    public double getValor() { return valor; }

    @Override
    public String descricao() { return usuario + " comprou " + valor; }
}
```

`EventoErro.java`

```java
public class EventoErro extends Evento {
    private final int codigo;
    private final String mensagem;

    public EventoErro(int codigo, String mensagem) {
        this.codigo = codigo;
        this.mensagem = mensagem;
    }

    public int getCodigo() { return codigo; }

    @Override
    public String descricao() { return "erro " + codigo + " - " + mensagem; }
}
```

`EventoLogin.java`

```java
public class EventoLogin extends Evento {
    private final String usuario;

    public EventoLogin(String usuario) { this.usuario = usuario; }

    public String getUsuario() { return usuario; }

    @Override
    public String descricao() { return usuario + " entrou"; }
}
```

`Ouvinte.java`

```java
@FunctionalInterface
public interface Ouvinte {
    void aoOcorrer(Evento evento);
}
```

Decisões e comentários

- A **chave** do mapa é `evento.getClass().getSimpleName()`: o tipo real do objeto é descoberto em tempo de execução, não pelo tipo declarado da variável.
- `publicar` monta primeiro a lista de destinatários (`"*"` e depois o tipo) e só então dispara. Assim, mesmo que um ouvinte registre outro durante o disparo, não ocorre `ConcurrentModificationException`.
- **Isolamento de falhas.** O `try/catch (RuntimeException)` fica **dentro** do laço: a falha de um ouvinte não impede os seguintes. O retorno conta só os que terminaram sem erro (no evento de erro 500 são 3 de 4).
- Lambdas e **classe anônima**: a anônima mantém **estado próprio** (`vistos`), o que uma lambda não faria. A lambda de compras só consegue acumular em `totalVendas` porque ele é **atributo estático**, e não variável local.
- `e instanceof EventoErro erro && erro.getCodigo() >= 500` combina teste de tipo, conversão segura e condição.

Gabarito do Exercício 10: Frota de drones

`Drone.java`

```java
import java.util.Objects;

public abstract class Drone implements Entregador, Comparable<Drone> {
    private final String id;
    protected int bateria;

    protected Drone(String id, int bateria) {
        if (bateria < 0 || bateria > 100) {
            throw new IllegalArgumentException("Bateria fora de 0..100: " + bateria);
        }
        this.id = id;
        this.bateria = bateria;
```

```
    }

    protected abstract double capacidadeKg();
    protected abstract double consumoPorKm();
    protected abstract double custoPorKm();

    @Override
    public String id() { return id; }

    public int bateria() { return bateria; }

    private double bateriaNecessaria(Pedido pedido) {
        return consumoPorKm() * pedido.getDistanciaKm() * 2;
    }

    @Override
    public boolean podeEntregar(Pedido pedido) {
        return pedido.getPesoKg() <= capacidadeKg() && bateriaNecessaria(pedido) <= bateria;
    }

    @Override
    public double custo(Pedido pedido) {
        return custoPorKm() * pedido.getDistanciaKm() * 2;
    }

    public void entregar(Pedido pedido) {
        if (!podeEntregar(pedido)) {
            throw new IllegalStateException(id + " nao pode entregar " + pedido.getId());
        }
        bateria -= (int) Math.ceil(bateriaNecessaria(pedido));
    }

    public void recarregar() {
        bateria = 100;
    }

    @Override
    public int compareTo(Drone outro) {
        int cmp = Integer.compare(outro.bateria, bateria);
        return cmp != 0 ? cmp : id.compareTo(outro.id);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return id.equals(((Drone) o).id);
    }

    @Override
    public int hashCode() { return Objects.hash(id); }

    @Override
    public String toString() {
        return getClass().getSimpleName() + "[" + id + ", bateria=" + bateria + "]";
    }
}
```

DroneLeve.java

```
public class DroneLeve extends Drone {
    public DroneLeve(String id, int bateria) { super(id, bateria); }

    @Override protected double capacidadeKg() { return 2.0; }
    @Override protected double consumoPorKm() { return 1.0; }
    @Override protected double custoPorKm() { return 0.5; }
}
```

DronePesado.java

```
public class DronePesado extends Drone {
    public DronePesado(String id, int bateria) { super(id, bateria); }

    @Override protected double capacidadeKg() { return 10.0; }
    @Override protected double consumoPorKm() { return 2.5; }
    @Override protected double custoPorKm() { return 1.5; }
}
```

DroneSolar.java

```
public final class DroneSolar extends DroneLeve {
    public DroneSolar(String id, int bateria) { super(id, bateria); }

    @Override protected double custoPorKm() { return 0.3; }

    @Override
    public void entregar(Pedido pedido) {
        super.entregar(pedido);
        bateria = Math.min(100, bateria + 5);
    }
}
```

Entregador.java

```java
public interface Entregador {
    String id();

    boolean podeEntregar(Pedido pedido);

    double custo(Pedido pedido);
}
```

Frota.java

```java
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class Frota {
    private final Map<String, Drone> drones = new TreeMap<>();
    private final List<Pedido> pendentes = new ArrayList<>();

    public void adicionar(Drone drone) {
        if (drones.containsKey(drone.id())) {
            throw new IllegalArgumentException("Drone repetido: " + drone.id());
        }
        drones.put(drone.id(), drone);
    }

    public void solicitar(Pedido pedido) {
        pendentes.add(pedido);
    }

    public int totalPendentes() {
        return pendentes.size();
    }

    public void recarregar(String id) {
        Drone d = drones.get(id);
        if (d == null) {
            throw new IllegalArgumentException("Drone inexistente: " + id);
        }
        d.recarregar();
    }

    public static <T extends Entregador> T mais Barato(Collection<T> candidatos, Pedido pedido) {
        T melhor = null;
        for (T c : candidatos) {
            if (melhor == null
                    || c.custo(pedido) < melhor.custo(pedido)
                    || (c.custo(pedido) == melhor.custo(pedido) && c.id().compareTo(melhor.id()) < 0)) {
                melhor = c;
            }
        }
        return melhor;
    }

    public Drone atribuir(Pedido pedido) throws SemDroneDisponivelException {
        List<Drone> candidatos = new ArrayList<>();
        for (Drone d : drones.values()) {
            if (d.podeEntregar(pedido)) {
                candidatos.add(d);
            }
        }
        if (candidatos.isEmpty()) {
            throw new SemDroneDisponivelException(pedido);
        }
        Drone escolhido = maisBarato(candidatos, pedido);
        escolhido.entregar(pedido);
        return escolhido;
    }

    public String processarPendentes() {
        StringBuilder log = new StringBuilder();
        Iterator<Pedido> it = pendentes.iterator();
        while (it.hasNext()) {
            Pedido p = it.next();
            try {
                Drone d = atribuir(p);
                log.append(String.format("%s -> %s (custo %.2f)%n", p.getId(), d.id(), d.custo(p)));
                it.remove();
            } catch (SemDroneDisponivelException e) {
                log.append(String.format("%s -> PENDENTE (%s)%n", p.getId(), e.getMessage()));
            }
        }
        return log.toString();
    }

    public List<Drone> dronesPorBateria() {
        List<Drone> copia = new ArrayList<>(drones.values());
        Collections.sort(copia);
        return copia;
    }
}
```

Pedido.java

```java
import java.util.Objects;

public class Pedido {
    private final String id;
    private final double pesoKg;
    private final double distanciaKm;

    public Pedido(String id, double pesoKg, double distanciaKm) {
        this.id = id;
        this.pesoKg = pesoKg;
        this.distanciaKm = distanciaKm;
    }

    public String getId() { return id; }
    public double getPesoKg() { return pesoKg; }
    public double getDistanciaKm() { return distanciaKm; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return id.equals(((Pedido) o).id);
    }

    @Override
    public int hashCode() { return Objects.hash(id); }

    @Override
    public String toString() { return "Pedido[" + id + ", " + pesoKg + "kg, " + distanciaKm + "km]"; }
}
```

`SemDroneDisponivelException.java`

```java
public class SemDroneDisponivelException extends Exception {
    public SemDroneDisponivelException(Pedido pedido) {
        super("nenhum drone pode atender " + pedido.getId());
    }
}
```

Decisões e comentários

- **Pontos de extensão.** `capacidadeKg`, `consumoPorKm` e `custoPorKm` são abstratos: cada classe de drone fornece só seus números. `podeEntregar`, `custo` e `entregar` ficam em `Drone`, e `bateriaNecessaria` é privado.
- **Três níveis.** `DroneSolar extends DroneLeve extends Drone`. `DroneSolar` sobrescreve `custoPorKm` (0,3) e `entregar`: chama `super.entregar(...)` e depois recarrega 5, no máximo 100. Por isso `bateria` é `protected`.
- **Método genérico.** `maisBarato(Collection<T>, Pedido)` com `T extends Entregador` não conhece `Drone`. Funcionaria com qualquer implementação do contrato. O desempate pelo menor `id` torna o resultado determinístico.
- **`TreeMap`** mantém os drones ordenados por id. `Collections.sort(copia)` usa a ordem natural (`Comparable`): maior bateria primeiro.
- **`Iterator.remove()`** em `processarPendentes` retira apenas os pedidos atendidos. A `SemDroneDisponivelException` (verificada) é capturada **dentro do laço**, de modo que um pedido impossível não interrompe os demais.
- **Conferência da saída.** O pedido A (1,5 kg, 10 km) pode ser atendido por `L1` (custo 10,00), `S1` (0,3 × 20 = 6,00) e `P1` (30,00). Vence `S1`, e sua bateria vai a 40 − 20 + 5 = 25. O pedido B (5 kg) só cabe em `P1`, que gasta 100 e fica com 0. Os pedidos C e E ficam pendentes, e voltam a ser atendidos depois de `recarregar("P1")`.

Parte IV: Gabaritos comentados das provas

Cada gabarito traz o **projeto completo** (compilado com `javac --release 17` e executado, com saída idêntica aos quadros dos enunciados), uma **tabela de critérios de correção** com distribuição sugerida de pontos e comentários sobre as decisões e armadilhas. Outras soluções são válidas se atenderem ao enunciado. Para itens com crédito parcial, os critérios indicam o que vale cada parte.

Gabarito da Prova Simulada 1: Arena Java

`src/arena/jogo/Arena.java`

```java
package arena.jogo;

import arena.modelo.CuraIndisponivelException;
import arena.modelo.Curandeiro;
import arena.modelo.Personagem;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class Arena {
    private final Map<String, List<Personagem>> times = new LinkedHashMap<>();
    private final Map<String, Integer> danoCausado = new TreeMap<>();
    private int rodada = 0;

    public Arena(String timeA, String timeB) {
        times.put(timeA, new ArrayList<>());
        times.put(timeB, new ArrayList<>());
    }

    public void adicionar(String time, Personagem personagem) {
        List<Personagem> lista = times.get(time);
        if (lista == null) {
            throw new IllegalArgumentException("Time inexistente: " + time);
        }
        for (List<Personagem> outra : times.values()) {
            if (outra.contains(personagem)) {
                throw new IllegalArgumentException("Personagem repetido: " + personagem.getNome());
            }
        }
        lista.add(personagem);
        danoCausado.put(personagem.getNome(), 0);
    }

    public Iterator<Personagem> iterador(String time) {
        List<Personagem> lista = times.get(time);
        if (lista == null) {
            throw new IllegalArgumentException("Time inexistente: " + time);
        }
        return lista.iterator();
    }

    private String oponente(String time) {
        for (String nome : times.keySet()) {
            if (!nome.equals(time)) {
                return nome;
            }
        }
        throw new IllegalStateException("Arena sem oponente");
    }

    private Personagem primeiroVivo(List<Personagem> lista) {
        for (Personagem p : lista) {
            if (p.estaVivo()) {
                return p;
            }
        }
        return null;
    }

    private Personagem maisFerido(List<Personagem> aliados) {
        Personagem ferido = null;
        for (Personagem p : aliados) {
            boolean precisaCura = p.estaVivo() && p.getVida() * 2 < p.getVidaMaxima();
            if (precisaCura && (ferido == null || p.getVida() < ferido.getVida())) {
                ferido = p;
            }
        }
        return ferido;
    }

    public boolean timeDerrotado(String time) {
        return primeiroVivo(times.get(time)) == null;
    }
```

```
    public String vencedor() {
        for (String time : times.keySet()) {
            if (timeDerrotado(oponente(time))) {
                return time;
            }
        }
        return null;
    }

    public String executarRodada() {
        rodada++;
        StringBuilder sb = new StringBuilder();
        sb.append("--- Rodada ").append(rodada).append(" ---\n");
        for (String nomeTime : times.keySet()) {
            List<Personagem> aliados = times.get(nomeTime);
            List<Personagem> inimigos = times.get(oponente(nomeTime));
            for (Personagem p : aliados) {
                if (!p.estaVivo()) {
                    continue;
                }
                if (primeiroVivo(inimigos) == null) {
                    return sb.toString();
                }
                if (p instanceof Curandeiro curandeiro) {
                    Personagem ferido = maisFerido(aliados);
                    if (ferido != null) {
                        try {
                            int cura = curandeiro.curar(ferido);
                            sb.append(String.format("%s cura %s (+%d) vida=%d%n",
                                    p.getNome(), ferido.getNome(), cura, ferido.getVida()));
                            continue;
                        } catch (CuraIndisponivelException e) {
                            sb.append(String.format("%s nao pode curar: %s%n", p.getNome(), e.getMessage()));
                        }
                    }
                }
                Personagem alvo = primeiroVivo(inimigos);
                int dano = p.atacar();
                alvo.receberDano(dano);
                danoCausado.put(p.getNome(), danoCausado.get(p.getNome()) + dano);
                sb.append(String.format("%s ataca %s (-%d) vida=%d%s%n", p.getNome(), alvo.getNome(),
                        dano, alvo.getVida(), alvo.estaVivo() ? "" : " [DERROTADO]"));
            }
        }
        return sb.toString();
    }

    public List<Personagem> ranking() {
        List<Personagem> todos = new ArrayList<>();
        for (List<Personagem> lista : times.values()) {
            todos.addAll(lista);
        }
        Collections.sort(todos);
        return todos;
    }

    public Map<String, Integer> danoCausado() {
        return new TreeMap<>(danoCausado);
    }
}
```

src/arena/modelo/Arqueiro.java

```
package arena.modelo;

public class Arqueiro extends Personagem {
    private static final int BASE = 14;
    private int ataques = 0;

    public Arqueiro(String nome) {
        super(nome, 90);
    }

    @Override
    public int atacar() {
        ataques++;
        return ataques % 3 == 0 ? BASE * 2 : BASE;
    }
}
```

src/arena/modelo/Clerigo.java

```
package arena.modelo;

public class Clerigo extends Personagem implements Curandeiro {
    private int usosDeCura = 2;

    public Clerigo(String nome) {
        super(nome, 100);
    }

    @Override
    public int atacar() {
        return 8;
    }
```

```
    @Override
    public int curar(Personagem alvo) throws CuraIndisponivelException {
        if (usosDeCura == 0) {
            throw new CuraIndisponivelException(getNome() + " sem usos de cura");
        }
        usosDeCura--;
        return alvo.receberCura(35);
    }
}
```

src/arena/modelo/CuraIndisponivelException.java

```
package arena.modelo;

public class CuraIndisponivelException extends Exception {
    public CuraIndisponivelException(String motivo) {
        super(motivo);
    }
}
```

src/arena/modelo/Curandeiro.java

```
package arena.modelo;

public interface Curandeiro {
    int curar(Personagem alvo) throws CuraIndisponivelException;
}
```

src/arena/modelo/Guerreiro.java

```
package arena.modelo;

public class Guerreiro extends Personagem {
    private static final int FORCA = 18;

    public Guerreiro(String nome) {
        super(nome, 120);
    }

    @Override
    public int atacar() {
        boolean furia = vida * 10 < getVidaMaxima() * 3;
        return furia ? FORCA * 3 / 2 : FORCA;
    }
}
```

src/arena/modelo/Mago.java

```
package arena.modelo;

public class Mago extends Personagem implements Curandeiro {
    private int mana = 40;

    public Mago(String nome) {
        super(nome, 70);
    }

    @Override
    public int atacar() {
        if (mana >= 10) {
            mana -= 10;
            return 25;
        }
        return 3;
    }

    @Override
    public int curar(Personagem alvo) throws CuraIndisponivelException {
        if (mana < 15) {
            throw new CuraIndisponivelException(getNome() + " sem mana (" + mana + ")");
        }
        mana -= 15;
        return alvo.receberCura(30);
    }

    public int getMana() { return mana; }
}
```

src/arena/modelo/Personagem.java

```
package arena.modelo;

import java.util.Objects;

public abstract class Personagem implements Comparable<Personagem> {
    private final String nome;
    private final int vidaMaxima;
    protected int vida;

    protected Personagem(String nome, int vidaMaxima) {
```

```
        if (nome == null || nome.isBlank()) {
            throw new IllegalArgumentException("Nome obrigatorio");
        }
        if (vidaMaxima <= 0) {
            throw new IllegalArgumentException("Vida maxima deve ser positiva");
        }
        this.nome = nome;
        this.vidaMaxima = vidaMaxima;
        this.vida = vidaMaxima;
    }

    public abstract int atacar();

    public String getNome() { return nome; }
    public int getVida() { return vida; }
    public int getVidaMaxima() { return vidaMaxima; }

    public boolean estaVivo() {
        return vida > 0;
    }

    public void receberDano(int dano) {
        if (dano < 0) {
            throw new IllegalArgumentException("Dano negativo: " + dano);
        }
        vida = Math.max(0, vida - dano);
    }

    public int receberCura(int pontos) {
        if (!estaVivo()) {
            return 0;
        }
        int antes = vida;
        vida = Math.min(vidaMaxima, vida + pontos);
        return vida - antes;
    }

    @Override
    public int compareTo(Personagem outro) {
        int cmp = Integer.compare(outro.vida, vida);
        return cmp != 0 ? cmp : nome.compareTo(outro.nome);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Personagem outro)) return false;
        return nome.equals(outro.nome);
    }

    @Override
    public int hashCode() {
        return Objects.hash(nome);
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + " " + nome + " (" + vida + "/" + vidaMaxima + ")";
    }
}
```

src/arena/main/Principal.java

```
package arena.main;

import arena.jogo.Arena;
import arena.modelo.Arqueiro;
import arena.modelo.Clerigo;
import arena.modelo.Guerreiro;
import arena.modelo.Mago;
import arena.modelo.Personagem;

import java.util.Map;

public class Principal {
    private static final int MAX_RODADAS = 40;

    public static void main(String[] args) {
        Arena arena = new Arena("Dragoes", "Fenix");
        arena.adicionar("Dragoes", new Guerreiro("Thor"));
        arena.adicionar("Dragoes", new Mago("Merlin"));
        arena.adicionar("Dragoes", new Clerigo("Aria"));
        arena.adicionar("Fenix", new Arqueiro("Legolas"));
        arena.adicionar("Fenix", new Guerreiro("Conan"));
        arena.adicionar("Fenix", new Arqueiro("Robin"));

        try {
            arena.adicionar("Fenix", new Mago("Thor"));
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }

        int rodadas = 0;
        while (arena.vencedor() == null && rodadas < MAX_RODADAS) {
            System.out.print(arena.executarRodada());
            rodadas++;
        }
```

```
        System.out.println("=== Vencedor: " + arena.vencedor() + " apos " + rodadas + " rodadas");
        System.out.println("--- Ranking");
        for (Personagem p : arena.ranking()) {
            System.out.println(p);
        }
        System.out.println("--- Dano causado");
        for (Map.Entry<String, Integer> e : arena.danoCausado().entrySet()) {
            System.out.println(e.getKey() + " = " + e.getValue());
        }
    }
}
```

Critérios de correção

#	Pts	Distribuição sugerida
1	0,5	Três pacotes com as classes nos lugares pedidos (0,5).
2	1,0	Interface com `throws` (0,4); exceção verificada, isto é, `extends Exception` e não `RuntimeException` (0,4); mensagem via construtor (0,2).
3	1,0	Classe abstrata e método abstrato (0,2); atributos e construtor `protected` com validações (0,3); `receberDano` sem vida negativa e rejeitando dano negativo (0,2); `receberCura` com os três comportamentos pedidos (0,3).
4	0,75	`equals`/`hashCode` por nome **sem** `getClass()` (0,35); `compareTo` correto (0,25); `toString` (0,15).
5	1,0	Fúria com aritmética inteira e limiar correto (0,5); contador de ataques e crítico a cada 3º (0,5).
6	1,0	`atacar` gasta mana só quando pode (0,4); `curar` lança a exceção com a mensagem pedida (0,4); cura efetiva devolvida (0,2).
7	0,5	Número de usos controlado e exceção correta.
8	1,0	Mapa com ordem preservada (0,2); validações com as mensagens (0,4); iterador sem expor a lista (0,2); `timeDerrotado`/`vencedor` (0,2).
9	2,0	Percurso por times e personagens na ordem (0,4); término imediato quando o inimigo cai (0,2); uso de `instanceof Curandeiro` (interface, não classe concreta) e escolha do aliado mais ferido (0,5); `try/catch` com fallback para ataque (0,4); ataque polimórfico, contagem do dano e formato do log (0,5).
10	0,75	`ranking()` por cópia (0,35); `danoCausado()` cópia ordenada (0,4).
11	0,5	Montagem, laço com `MAX_RODADAS` e impressões pedidas.

Comentários

- **A armadilha do `equals`.** O `equals` das aulas usa `getClass() != o.getClass()`. Aqui isso **quebraria** a regra: `Guerreiro Thor` e `Mago Thor` seriam "diferentes" e o `Mago Thor` entraria no outro time. Como a identidade vale para toda a hierarquia, o correto é `o instanceof Personagem`. Sem isso, o erro `Personagem repetido: Thor` nunca aparece.
- **Fúria sem `double`.** `vida * 10 < vidaMaxima * 3` evita conversão para real e é exato (vida < 30%). O dano em fúria, `18 * 3 / 2`, vale 27.
- **Estado nas subclasses.** `Arqueiro.ataques` e `Mago.mana` são o estado que muda a cada chamada de `atacar()`. É por isso que `atacar()` é um método (comportamento) e não uma constante.
- **Interface como tipo de decisão.** A `Arena` pergunta `p instanceof Curandeiro curandeiro` e **nunca** `instanceof Mago`. Se amanhã surgir um `Druida implements Curandeiro`, a arena não muda.
- **Fluxo do `try/catch`.** Cura bem-sucedida usa `continue` (o personagem gastou o turno). Se a cura falhar, o `catch` registra o motivo e o código **cai no ataque**: o curandeiro que não pode curar ataca.
- **Conferência com o quadro.** Na rodada 3, `Thor` está com 42 (< 60, metade de 120), e `Merlin` o cura (+30 → 72). Na rodada 5, `Thor` ataca com 27 porque sua vida (29) está abaixo de 36, em fúria. `Robin` causa 28 no seu 3º ataque (crítico). Na rodada 6, `Aria` já gastou os dois usos e vemos `Aria nao pode curar: Aria sem usos de cura`, seguido do ataque.
- **Encapsulamento.** `iterador` devolve o `Iterator` da lista (como na Aula 07), e `ranking`/`danoCausado` devolvem **cópias**. A `Arena` retorna `String` em vez de imprimir.

Gabarito da Prova Simulada 2: MelodIFy

`src/melodify/dados/Dados.java`

```
package melodify.dados;

import java.util.List;

public final class Dados {

    private Dados() { }

    public static List<String> carregarCatalogo() {
        return List.of(
            "MUSICA;Bohemian Rhapsody;Queen;ROCK;355",
            "MUSICA;Billie Jean;Michael Jackson;POP;294",
            "MUSICA;Evidencias;Chitaozinho e Xororo;SERTANEJO;284",
```

```
            "MUSICA;Take Five;Dave Brubeck;JAZZ;324",
            "MUSICA;Another One Bites the Dust;Queen;ROCK;215",
            "MUSICA;Harder Better Faster;Daft Punk;ELETRONICA;224",
            "MUSICA;Thriller;Michael Jackson;POP;357",
            "PODCAST;Flow #42;Igor 3K;42;3600",
            "PODCAST;Nerdcast #10;Jovem Nerd;10;4200",
            "MUSICA;Hotel California;Eagles;ROCK;391"
        );
    }
}
```

src/melodify/modelo/Filtro.java

```
package melodify.modelo;

@FunctionalInterface
public interface Filtro<T> {
    boolean aceita(T elemento);
}
```

src/melodify/modelo/Filtros.java

```
package melodify.modelo;

import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

public final class Filtros {

    private Filtros() { }

    public static <T extends Reproduzivel> List<T> filtrar(Collection<T> itens, Filtro<T> filtro) {
        List<T> resultado = new ArrayList<>();
        for (T item : itens) {
            if (filtro.aceita(item)) {
                resultado.add(item);
            }
        }
        return resultado;
    }
}
```

src/melodify/modelo/Genero.java

```
package melodify.modelo;

public enum Genero {
    ROCK, POP, SERTANEJO, JAZZ, ELETRONICA
}
```

src/melodify/modelo/Midia.java

```
package melodify.modelo;

import java.util.Objects;

public abstract class Midia implements Reproduzivel {
    private final String titulo;
    private final String autor;
    private final int duracaoSegundos;

    protected Midia(String titulo, String autor, int duracaoSegundos) {
        if (duracaoSegundos <= 0) {
            throw new IllegalArgumentException("Duracao deve ser positiva");
        }
        this.titulo = titulo;
        this.autor = autor;
        this.duracaoSegundos = duracaoSegundos;
    }

    public abstract String descricao();

    @Override
    public String titulo() { return titulo; }

    public String autor() { return autor; }

    @Override
    public int duracaoSegundos() { return duracaoSegundos; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Midia outra)) return false;
        return titulo.equals(outra.titulo) && autor.equals(outra.autor);
    }

    @Override
    public int hashCode() {
        return Objects.hash(titulo, autor);
    }
```

```
    @Override
    public String toString() {
        return titulo + " - " + autor + " [" + duracaoFormatada() + "]";
    }
}
```

src/melodify/modelo/Musica.java

```
package melodify.modelo;

public class Musica extends Midia {
    private final Genero genero;

    public Musica(String titulo, String artista, Genero genero, int duracaoSegundos) {
        super(titulo, artista, duracaoSegundos);
        this.genero = genero;
    }

    public Genero getGenero() { return genero; }

    @Override
    public String descricao() {
        return "Musica de " + autor() + " (" + genero + ")";
    }
}
```

src/melodify/modelo/Playlist.java

```
package melodify.modelo;

import java.util.Iterator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public class Playlist {
    private final String nome;
    private final Set<Midia> midias = new LinkedHashSet<>();

    public Playlist(String nome) {
        this.nome = nome;
    }

    public String getNome() { return nome; }

    public boolean adicionar(Midia midia) {
        return midias.add(midia);
    }

    public int adicionarVarias(Midia... novas) {
        int adicionadas = 0;
        for (Midia m : novas) {
            if (midias.add(m)) {
                adicionadas++;
            }
        }
        return adicionadas;
    }

    public boolean remover(Midia midia) {
        return midias.remove(midia);
    }

    public int tamanho() {
        return midias.size();
    }

    public Iterator<Midia> iterador() {
        return midias.iterator();
    }

    public int duracaoTotal() {
        return midias.stream().mapToInt(Midia::duracaoSegundos).sum();
    }

    public String duracaoTotalFormatada() {
        int total = duracaoTotal();
        return String.format("%02d:%02d", total / 60, total % 60);
    }

    public List<Midia> filtrar(Filtro<Midia> filtro) {
        return Filtros.filtrar(midias, filtro);
    }
}
```

src/melodify/modelo/Podcast.java

```
package melodify.modelo;

public class Podcast extends Midia {
    private final int episodio;

    public Podcast(String titulo, String apresentador, int episodio, int duracaoSegundos) {
```

```
        super(titulo, apresentador, duracaoSegundos);
        this.episodio = episodio;
    }

    public int getEpisodio() { return episodio; }

    @Override
    public String descricao() {
        return "Podcast de " + autor() + ", episodio " + episodio;
    }
}
```

src/melodify/modelo/Reproduzivel.java

```
package melodify.modelo;

public interface Reproduzivel {
    String titulo();

    int duracaoSegundos();

    default String duracaoFormatada() {
        int segundos = duracaoSegundos();
        return String.format("%02d:%02d", segundos / 60, segundos % 60);
    }
}
```

src/melodify/servico/Catalogo.java

```
package melodify.servico;

import melodify.dados.Dados;
import melodify.modelo.Genero;
import melodify.modelo.Midia;
import melodify.modelo.Musica;
import melodify.modelo.Podcast;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeSet;

public class Catalogo {
    private final Map<String, Midia> midiasPorTitulo = new LinkedHashMap<>();
    private final Map<Genero, List<Musica>> musicasPorGenero = new HashMap<>();

    public Catalogo() {
        carregar();
    }

    private void carregar() {
        for (String linha : Dados.carregarCatalogo()) {
            String[] campos = linha.split(";");
            Midia midia;
            if (campos[0].equals("MUSICA")) {
                Genero genero = Genero.valueOf(campos[3]);
                Musica musica = new Musica(campos[1], campos[2], genero, Integer.parseInt(campos[4]));
                if (!musicasPorGenero.containsKey(genero)) {
                    musicasPorGenero.put(genero, new ArrayList<>());
                }
                musicasPorGenero.get(genero).add(musica);
                midia = musica;
            } else {
                midia = new Podcast(campos[1], campos[2], Integer.parseInt(campos[3]), Integer.parseInt(campos[4]));
            }
            midiasPorTitulo.put(midia.titulo(), midia);
        }
    }

    public int total() {
        return midiasPorTitulo.size();
    }

    public Midia buscarPorTitulo(String titulo) {
        return midiasPorTitulo.get(titulo);
    }

    public List<Musica> musicasDoGenero(Genero genero) {
        List<Musica> lista = musicasPorGenero.get(genero);
        return lista == null ? new ArrayList<>() : new ArrayList<>(lista);
    }

    public Set<Genero> generosDisponiveis() {
        return new TreeSet<>(musicasPorGenero.keySet());
    }
}
```

src/melodify/servico/PlaylistInexistenteException.java

```
package melodify.servico;
```

```java
public class PlaylistInexistenteException extends Exception {
    public PlaylistInexistenteException(String nome) {
        super("Playlist inexistente: " + nome);
    }
}
```

src/melodify/servico/ResumoPlaylist.java

```java
package melodify.servico;

import melodify.modelo.Midia;
import melodify.modelo.Playlist;

import java.util.Iterator;

public final class ResumoPlaylist {

    private ResumoPlaylist() { }

    public static String emitir(Playlist playlist) {
        StringBuilder sb = new StringBuilder();
        sb.append("========== PLAYLIST: ").append(playlist.getNome()).append(" ==========\n");
        int posicao = 1;
        Iterator<Midia> it = playlist.iterador();
        while (it.hasNext()) {
            Midia m = it.next();
            sb.append(String.format("%2d. %s%n", posicao++, m));
        }
        sb.append("----------------------------------------\n");
        sb.append("Faixas: ").append(playlist.tamanho())
          .append(" | Duracao total: ").append(playlist.duracaoTotalFormatada()).append('\n');
        return sb.toString();
    }
}
```

src/melodify/servico/Usuario.java

```java
package melodify.servico;

import melodify.modelo.Midia;
import melodify.modelo.Playlist;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class Usuario {
    private final String nome;
    private final Map<String, Playlist> playlists = new LinkedHashMap<>();
    private final Map<Midia, Integer> reproducoes = new HashMap<>();

    public Usuario(String nome) {
        this.nome = nome;
    }

    public String getNome() { return nome; }

    public Playlist criarPlaylist(String nomePlaylist) {
        if (playlists.containsKey(nomePlaylist)) {
            throw new IllegalArgumentException("Playlist ja existe: " + nomePlaylist);
        }
        Playlist playlist = new Playlist(nomePlaylist);
        playlists.put(nomePlaylist, playlist);
        return playlist;
    }

    public Playlist getPlaylist(String nomePlaylist) throws PlaylistInexistenteException {
        Playlist playlist = playlists.get(nomePlaylist);
        if (playlist == null) {
            throw new PlaylistInexistenteException(nomePlaylist);
        }
        return playlist;
    }

    public String reproduzir(Midia midia) {
        reproducoes.put(midia, reproducoes.getOrDefault(midia, 0) + 1);
        return "Tocando: " + midia.titulo() + " [" + midia.duracaoFormatada() + "]";
    }

    public int reproduzirPlaylist(String nomePlaylist) throws PlaylistInexistenteException {
        Playlist playlist = getPlaylist(nomePlaylist);
        int segundos = 0;
        Iterator<Midia> it = playlist.iterador();
        while (it.hasNext()) {
            Midia m = it.next();
            reproduzir(m);
            segundos += m.duracaoSegundos();
        }
        return segundos;
    }

    public int vezesTocada(Midia midia) {
        return reproducoes.getOrDefault(midia, 0);
```

```
    }

    public List<Midia> maisTocadas(int n) {
        List<Map.Entry<Midia, Integer>> entradas = new ArrayList<>(reproducoes.entrySet());
        entradas.sort((a, b) -> {
            int cmp = Integer.compare(b.getValue(), a.getValue());
            return cmp != 0 ? cmp : a.getKey().titulo().compareTo(b.getKey().titulo());
        });
        List<Midia> top = new ArrayList<>();
        for (int i = 0; i < Math.min(n, entradas.size()); i++) {
            top.add(entradas.get(i).getKey());
        }
        return top;
    }
}
```

`src/melodify/main/Principal.java`

```
package melodify.main;

import melodify.modelo.Filtros;
import melodify.modelo.Genero;
import melodify.modelo.Midia;
import melodify.modelo.Musica;
import melodify.modelo.Playlist;
import melodify.servico.Catalogo;
import melodify.servico.PlaylistInexistenteException;
import melodify.servico.ResumoPlaylist;
import melodify.servico.Usuario;

import java.util.List;

public class Principal {
    public static void main(String[] args) {
        Catalogo catalogo = new Catalogo();
        System.out.println("Midias no catalogo: " + catalogo.total());
        System.out.println("Generos: " + catalogo.generosDisponiveis());
        System.out.println("Rock: " + catalogo.musicasDoGenero(Genero.ROCK));
        System.out.println("Jazz: " + catalogo.musicasDoGenero(Genero.JAZZ).size()
                + " | Sertanejo: " + catalogo.musicasDoGenero(Genero.SERTANEJO).size());

        Usuario ana = new Usuario("Ana");
        Playlist treino = ana.criarPlaylist("Treino");
        int novas = treino.adicionarVarias(
                catalogo.buscarPorTitulo("Harder Better Faster"),
                catalogo.buscarPorTitulo("Thriller"),
                catalogo.buscarPorTitulo("Thriller"),
                catalogo.buscarPorTitulo("Another One Bites the Dust"));
        System.out.println("Adicionadas de uma vez: " + novas);
        System.out.println("Repetida entra? " + treino.adicionar(catalogo.buscarPorTitulo("Thriller")));
        System.out.print(ResumoPlaylist.emitir(treino));

        try {
            ana.criarPlaylist("Treino");
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }

        try {
            int segundos = ana.reproduzirPlaylist("Treino");
            segundos += ana.reproduzirPlaylist("Treino");
            System.out.println("Segundos ouvidos: " + segundos);
            ana.reproduzirPlaylist("Estudo");
        } catch (PlaylistInexistenteException e) {
            System.out.println("Erro: " + e.getMessage());
        }
        System.out.println(ana.reproduzir(catalogo.buscarPorTitulo("Flow #42")));
        System.out.println(ana.reproduzir(catalogo.buscarPorTitulo("Thriller")));
        System.out.println("Mais tocadas: " + ana.maisTocadas(3));

        List<Midia> longas = treino.filtrar(m -> m.duracaoSegundos() > 250);
        System.out.println("Longas da playlist: " + longas);
        List<Musica> rockCurtas = Filtros.filtrar(catalogo.musicasDoGenero(Genero.ROCK), m -> m.duracaoSegundos() < 300);
        System.out.println("Rock curto: " + rockCurtas);
        System.out.println(catalogo.buscarPorTitulo("Nerdcast #10").descricao());
    }
}
```

Critérios de correção

#	Pts	Distribuição sugerida
1	1,0	Pacotes e `Dados` (0,2); enum (0,1); interface com método **`default`** correto em `mm:ss` (0,7).
2	1,0	`Midia` abstrata com validação (0,3); `equals`/`hashCode` por título e autor (0,3); `toString` (0,1); `Musica` e `Podcast` com as descrições (0,3).
3	1,0	`Filtro<T>` funcional (0,3); método genérico com **limite** (0,4); resultado correto e classe não instanciável (0,3).
4	1,5	Estrutura sem repetição e com ordem (0,5); `adicionar`/`remover`/`tamanho` (0,3); `adicionarVarias` contando só as novas (0,3); `iterador` sem expor (0,2); `filtrar` reutilizando `Filtros` (0,2).
5	0,5	Soma com stream (0,3); formatação (0,2).
6	1,5	`split` e criação dos dois tipos (0,5); dois mapas corretos (0,4); `musicasDoGenero` por cópia e vazia se não houver (0,3);

		`generosDisponiveis` ordenado pelo enum (0,3).
7	1,0	Mapa com ordem (0,2); `criarPlaylist` com exceção não verificada (0,4); `getPlaylist` com exceção verificada (0,4).
8	1,5	Contagem com `getOrDefault` e retorno do texto (0,4); `reproduzirPlaylist` em ordem, somando segundos (0,4); `maisTocadas` com ordenação correta e desempate (0,5); só mídias tocadas e limite `n` (0,2).
9	0,5	`StringBuilder`, formato e classe não instanciável.
10	0,5	Roteiro completo, com lambdas nos filtros e tratamento das exceções.

Comentários

- **Método `default`.** `duracaoFormatada()` fica na interface porque depende apenas de `duracaoSegundos()`: todas as mídias herdam o comportamento sem duplicar código (Aula 06). Para 3600 s o resultado é `60:00`, já que não há conversão para horas.
- **`equals`/`hashCode` e `Set`/`Map`.** `LinkedHashSet` rejeita a repetição **porque** `Midia` define `equals`/`hashCode`. A chave `Map<Midia, Integer>` de `reproducoes` também depende disso: sem esses métodos, cada objeto contaria como mídia diferente.
- **Varargs.** `adicionarVarias(Midia...)` devolve **quantas entraram** usando o retorno booleano de `Set.add` (a 2ª `Thriller` é rejeitada, daí 3 e não 4).
- **Genéricos.** `Filtros.filtrar(Collection<T>, Filtro<T>)` é chamado com `Set<Midia>` (`T = Midia`) na playlist e com `List<Musica>` (`T = Musica`) direto. O limite `T extends Reproduzivel` garante que os itens têm `duracaoSegundos()`.
- **`TreeSet<Genero>`.** `Enum` já implementa `Comparable` pela ordem de declaração. Por isso `generosDisponiveis()` devolve `[ROCK, POP, SERTANEJO, JAZZ, ELETRONICA]` sem comparador.
- **`maisTocadas`.** Ordena-se uma lista de `Map.Entry`. `Thriller` tem 3 execuções (2 pela playlist e 1 avulsa). `Another One Bites the Dust` e `Harder Better Faster` empatam com 2 e saem por **título crescente**. `Flow #42` tem só 1 e fica de fora do top 3. Reproduzir a playlist duas vezes soma 2 × 796 = 1592 s.

Gabarito da Prova Simulada 3: Loja Virtual

`src/loja/modelo/Carrinho.java`

```java
package loja.modelo;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.Map;

public class Carrinho {
    private final Map<Produto, Integer> itens = new LinkedHashMap<>();

    public void adicionar(Produto produto, int quantidade) {
        if (quantidade <= 0) {
            throw new IllegalArgumentException("Quantidade deve ser positiva: " + quantidade);
        }
        itens.put(produto, itens.getOrDefault(produto, 0) + quantidade);
    }

    public void remover(Produto produto, int quantidade) {
        int atual = itens.getOrDefault(produto, 0);
        if (quantidade <= 0 || quantidade > atual) {
            throw new IllegalArgumentException("Nao e possivel remover " + quantidade + " de " + produto);
        }
        if (quantidade == atual) {
            itens.remove(produto);
        } else {
            itens.put(produto, atual - quantidade);
        }
    }

    public int quantidadeDe(Produto produto) {
        return itens.getOrDefault(produto, 0);
    }

    public boolean isVazio() {
        return itens.isEmpty();
    }

    public double subtotal() {
        double total = 0;
        for (Map.Entry<Produto, Integer> e : itens.entrySet()) {
            total += e.getKey().getPreco() * e.getValue();
        }
        return total;
    }

    public double frete() {
        double total = 0;
        for (Map.Entry<Produto, Integer> e : itens.entrySet()) {
            total += e.getKey().frete(e.getValue());
        }
        return total;
    }

    public Iterator<Produto> iterador() {
```

```
        return new ArrayList<>(itens.keySet()).iterator();
    }
}
```

src/loja/modelo/Estoque.java

```
package loja.modelo;

import java.util.HashMap;
import java.util.Map;

public class Estoque {
    private final Map<String, Integer> saldos = new HashMap<>();

    public void repor(String sku, int quantidade) {
        if (quantidade <= 0) {
            throw new IllegalArgumentException("Quantidade de reposicao deve ser positiva");
        }
        saldos.put(sku, disponivel(sku) + quantidade);
    }

    public int disponivel(String sku) {
        return saldos.getOrDefault(sku, 0);
    }

    public boolean temSaldo(Produto produto, int quantidade) {
        return !produto.controlaEstoque() || disponivel(produto.getSku()) >= quantidade;
    }

    public void baixar(Produto produto, int quantidade) throws EstoqueInsuficienteException {
        if (!produto.controlaEstoque()) {
            return;
        }
        int atual = disponivel(produto.getSku());
        if (atual < quantidade) {
            throw new EstoqueInsuficienteException(produto.getSku(), quantidade, atual);
        }
        saldos.put(produto.getSku(), atual - quantidade);
    }
}
```

src/loja/modelo/EstoqueInsuficienteException.java

```
package loja.modelo;

public class EstoqueInsuficienteException extends Exception {
    private final String sku;
    private final int solicitado;
    private final int disponivel;

    public EstoqueInsuficienteException(String sku, int solicitado, int disponivel) {
        super("Estoque insuficiente de " + sku + ": pedido " + solicitado + ", disponivel " + disponivel);
        this.sku = sku;
        this.solicitado = solicitado;
        this.disponivel = disponivel;
    }

    public String getSku() { return sku; }
    public int getSolicitado() { return solicitado; }
    public int getDisponivel() { return disponivel; }
}
```

src/loja/modelo/Produto.java

```
package loja.modelo;

import java.util.Objects;

public abstract class Produto {
    private final String sku;
    private final String nome;
    private final double preco;

    protected Produto(String sku, String nome, double preco) {
        if (preco < 0) {
            throw new IllegalArgumentException("Preco negativo");
        }
        this.sku = sku;
        this.nome = nome;
        this.preco = preco;
    }

    public abstract double frete(int quantidade);

    public abstract boolean controlaEstoque();

    public String getSku() { return sku; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Produto outro)) return false;
```

```
        return sku.equals(outro.sku);
    }

    @Override
    public int hashCode() {
        return Objects.hash(sku);
    }

    @Override
    public String toString() {
        return nome + " (" + sku + ")";
    }
}
```

src/loja/modelo/ProdutoDigital.java

```
package loja.modelo;

public class ProdutoDigital extends Produto {

    public ProdutoDigital(String sku, String nome, double preco) {
        super(sku, nome, preco);
    }

    @Override
    public double frete(int quantidade) {
        return 0.0;
    }

    @Override
    public boolean controlaEstoque() {
        return false;
    }
}
```

src/loja/modelo/ProdutoFisico.java

```
package loja.modelo;

public class ProdutoFisico extends Produto {
    private final double pesoKg;

    public ProdutoFisico(String sku, String nome, double preco, double pesoKg) {
        super(sku, nome, preco);
        this.pesoKg = pesoKg;
    }

    public double getPesoKg() { return pesoKg; }

    @Override
    public double frete(int quantidade) {
        return pesoKg * quantidade * 3.0;
    }

    @Override
    public boolean controlaEstoque() {
        return true;
    }
}
```

src/loja/modelo/ProdutoFragil.java

```
package loja.modelo;

public class ProdutoFragil extends ProdutoFisico {

    public ProdutoFragil(String sku, String nome, double preco, double pesoKg) {
        super(sku, nome, preco, pesoKg);
    }

    @Override
    public double frete(int quantidade) {
        return super.frete(quantidade) + 15.0;
    }
}
```

src/loja/pagamento/Boleto.java

```
package loja.pagamento;

public class Boleto implements Pagamento {

    @Override
    public String descricao() {
        return "BOLETO";
    }

    @Override
    public double total(double valor) {
        return valor + 3.50;
    }
}
```

`src/loja/pagamento/CartaoCredito.java`

```java
package loja.pagamento;

public class CartaoCredito implements Pagamento {
    private final int parcelas;

    public CartaoCredito(int parcelas) {
        if (parcelas < 1 || parcelas > 12) {
            throw new IllegalArgumentException("Parcelas devem estar entre 1 e 12: " + parcelas);
        }
        this.parcelas = parcelas;
    }

    @Override
    public String descricao() {
        return "CARTAO em " + parcelas + "x";
    }

    @Override
    public double total(double valor) {
        return valor * (1 + 0.02 * (parcelas - 1));
    }
}
```

`src/loja/pagamento/Pagamento.java`

```java
package loja.pagamento;

public interface Pagamento {
    String descricao();

    double total(double valor);
}
```

`src/loja/pagamento/PagamentoFactory.java`

```java
package loja.pagamento;

public final class PagamentoFactory {

    private PagamentoFactory() { }

    public static Pagamento criar(String forma) {
        if (forma == null) {
            throw new IllegalArgumentException("Forma de pagamento nula");
        }
        String[] partes = forma.toLowerCase().split(":");
        switch (partes[0]) {
            case "pix":
                return new Pix();
            case "boleto":
                return new Boleto();
            case "cartao":
                if (partes.length != 2) {
                    throw new IllegalArgumentException("Informe as parcelas: cartao:N");
                }
                try {
                    return new CartaoCredito(Integer.parseInt(partes[1]));
                } catch (NumberFormatException e) {
                    throw new IllegalArgumentException("Parcelas invalidas: " + partes[1]);
                }
            default:
                throw new IllegalArgumentException("Forma de pagamento desconhecida: " + forma);
        }
    }
}
```

`src/loja/pagamento/Pix.java`

```java
package loja.pagamento;

public class Pix implements Pagamento {

    @Override
    public String descricao() {
        return "PIX";
    }

    @Override
    public double total(double valor) {
        return valor * 0.95;
    }
}
```

`src/loja/servico/Notificador.java`

```java
package loja.servico;

@FunctionalInterface
public interface Notificador {
```

```
    void notificar(Pedido pedido);
}
```

src/loja/servico/Pedido.java

```
package loja.servico;

public class Pedido {
    private final String id;
    private final String formaPagamento;
    private final double valorBruto;
    private final double total;

    public Pedido(String id, String formaPagamento, double valorBruto, double total) {
        this.id = id;
        this.formaPagamento = formaPagamento;
        this.valorBruto = valorBruto;
        this.total = total;
    }

    public String getId() { return id; }
    public double getValorBruto() { return valorBruto; }
    public double getTotal() { return total; }

    @Override
    public String toString() {
        return String.format("Pedido %s | %s | bruto=%.2f | total=%.2f", id, formaPagamento, valorBruto, total);
    }
}
```

src/loja/servico/Repositorio.java

```
package loja.servico;

import java.util.List;

public interface Repositorio<K, V> {
    void salvar(K chave, V valor);

    V buscar(K chave);

    List<V> listar();
}
```

src/loja/servico/RepositorioMemoria.java

```
package loja.servico;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class RepositorioMemoria<K, V> implements Repositorio<K, V> {
    private final Map<K, V> dados = new LinkedHashMap<>();

    @Override
    public void salvar(K chave, V valor) {
        dados.put(chave, valor);
    }

    @Override
    public V buscar(K chave) {
        return dados.get(chave);
    }

    @Override
    public List<V> listar() {
        return new ArrayList<>(dados.values());
    }
}
```

src/loja/servico/ServicoPedido.java

```
package loja.servico;

import loja.modelo.Carrinho;
import loja.modelo.Estoque;
import loja.modelo.EstoqueInsuficienteException;
import loja.modelo.Produto;
import loja.pagamento.Pagamento;

import java.util.Iterator;

public class ServicoPedido {
    private final Repositorio<String, Pedido> repositorio;
    private final Estoque estoque;
    private final Notificador notificador;

    public ServicoPedido(Repositorio<String, Pedido> repositorio, Estoque estoque, Notificador notificador) {
        this.repositorio = repositorio;
        this.estoque = estoque;
```

```
        this.notificador = notificador;
    }

    public Pedido finalizar(String id, Carrinho carrinho, Pagamento pagamento)
            throws EstoqueInsuficienteException {
        if (carrinho.isVazio()) {
            throw new IllegalStateException("Carrinho vazio");
        }
        if (repositorio.buscar(id) != null) {
            throw new IllegalArgumentException("Pedido ja existe: " + id);
        }
        Iterator<Produto> it = carrinho.iterador();
        while (it.hasNext()) {
            Produto p = it.next();
            int quantidade = carrinho.quantidadeDe(p);
            if (!estoque.temSaldo(p, quantidade)) {
                throw new EstoqueInsuficienteException(p.getSku(), quantidade, estoque.disponivel(p.getSku()));
            }
        }
        it = carrinho.iterador();
        while (it.hasNext()) {
            Produto p = it.next();
            estoque.baixar(p, carrinho.quantidadeDe(p));
        }
        double bruto = carrinho.subtotal() + carrinho.frete();
        Pedido pedido = new Pedido(id, pagamento.descricao(), bruto, pagamento.total(bruto));
        repositorio.salvar(id, pedido);
        notificador.notificar(pedido);
        return pedido;
    }
}
```

src/loja/main/Principal.java

```
package loja.main;

import loja.modelo.Carrinho;
import loja.modelo.Estoque;
import loja.modelo.EstoqueInsuficienteException;
import loja.modelo.Produto;
import loja.modelo.ProdutoDigital;
import loja.modelo.ProdutoFisico;
import loja.modelo.ProdutoFragil;
import loja.pagamento.PagamentoFactory;
import loja.servico.Pedido;
import loja.servico.Repositorio;
import loja.servico.RepositorioMemoria;
import loja.servico.ServicoPedido;

public class Principal {
    public static void main(String[] args) {
        Produto livro = new ProdutoFisico("LIV-1", "Livro de Java", 80.0, 0.5);
        Produto vidro = new ProdutoFragil("VID-1", "Vaso de vidro", 120.0, 2.0);
        Produto curso = new ProdutoDigital("CUR-1", "Curso online", 200.0);

        Estoque estoque = new Estoque();
        estoque.repor("LIV-1", 5);
        estoque.repor("VID-1", 1);

        Repositorio<String, Pedido> repositorio = new RepositorioMemoria<>();
        ServicoPedido servico = new ServicoPedido(repositorio, estoque,
                pedido -> System.out.println("[NOTIFICACAO] " + pedido.getId() + " confirmado"));

        Carrinho carrinho = new Carrinho();
        carrinho.adicionar(livro, 2);
        carrinho.adicionar(vidro, 1);
        carrinho.adicionar(curso, 1);
        carrinho.adicionar(livro, 1);
        carrinho.remover(curso, 1);
        carrinho.adicionar(curso, 1);
        System.out.println("livros no carrinho = " + carrinho.quantidadeDe(new ProdutoFisico("LIV-1", "x", 0, 0)));
        System.out.printf("subtotal=%.2f frete=%.2f%n", carrinho.subtotal(), carrinho.frete());

        String[] formas = {"pix", "cartao:6", "boleto", "cartao:abc", "cartao:20", "cripto"};
        for (String forma : formas) {
            try {
                System.out.printf("%-10s -> %s | total de 100.00 = %.2f%n", forma,
                        PagamentoFactory.criar(forma).descricao(), PagamentoFactory.criar(forma).total(100.0));
            } catch (IllegalArgumentException e) {
                System.out.println(forma + " -> erro: " + e.getMessage());
            }
        }

        try {
            Pedido p1 = servico.finalizar("P-001", carrinho, PagamentoFactory.criar("cartao:3"));
            System.out.println(p1);
            Carrinho outro = new Carrinho();
            outro.adicionar(livro, 1);
            outro.adicionar(vidro, 1);
            servico.finalizar("P-002", outro, PagamentoFactory.criar("pix"));
        } catch (EstoqueInsuficienteException e) {
            System.out.println("Falha: " + e.getMessage());
        }
        System.out.println("Estoque LIV-1 = " + estoque.disponivel("LIV-1") + ", VID-1 = " + estoque.disponivel("VID-1"));
        System.out.println("Pedidos salvos = " + repositorio.listar().size());
        System.out.println("P-002 existe? " + (repositorio.buscar("P-002") != null));
        try {
```

```
            servico.finalizar("P-003", new Carrinho(), PagamentoFactory.criar("pix"));
        } catch (IllegalStateException | EstoqueInsuficienteException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Critérios de correção

#	Pts	Distribuição sugerida
1	1,5	`Produto` com validação, abstratos e `equals`/`hashCode` por sku **via** `instanceof` (0,6); `ProdutoFisico` (0,3); `ProdutoFragil` reutilizando o frete do pai (0,3); `ProdutoDigital` (0,3).
2	1,0	Interface (0,2); `Pix` e `Boleto` (0,3); `CartaoCredito` com validação e fórmula (0,5).
3	1,0	Retorno pela interface (0,2); tratamento de `cartao:N` com conversão de exceção (0,5); demais erros com as mensagens (0,3).
4	1,5	Mapa com ordem e acúmulo (0,5); `remover` com as três situações (0,4); `subtotal`/`frete` (0,3); iterador sobre cópia (0,3).
5	1,0	Exceção verificada com os campos (0,3); `temSaldo`/`baixar` respeitando `controlaEstoque()` (0,5); `repor`/`disponivel` (0,2).
6	1,0	Interface genérica (0,4); implementação genérica com `Map` (0,4); cópia no `listar` (0,2).
7	0,5	`Pedido` e `toString` (0,3); `Notificador` funcional (0,2).
8	2,0	Injeção só por interfaces (0,5); validações (a) e (b) (0,3); **verificação de todos os itens antes de baixar** (0,6); cálculo do total e persistência/notificação (0,4); declaração de `throws` (0,2).
9	0,5	Roteiro com lambda de notificação e tratamento das falhas.

Comentários

- `equals` **por sku e** `Map<Produto, Integer>`**.** O carrinho depende de `equals`/`hashCode` para somar o livro adicionado duas vezes. No `Principal`, consultamos com **outro objeto** de mesmo sku e obtemos 3. Usamos `instanceof Produto` para que o sku identifique o produto em qualquer subclasse.
- **Inversão de dependência (item 8).** `ServicoPedido` importa só `Repositorio`, `Estoque`, `Notificador` e `Pagamento`. O `Principal` decide as implementações concretas (`RepositorioMemoria`, lambda, `PagamentoFactory.criar("pix")`). Trocar `Pix` por `Boleto`, ou a memória por um banco de dados, não exige tocar no serviço.
- **Atomicidade.** O serviço percorre o carrinho **duas vezes**: primeiro `temSaldo` em todos os itens, depois `baixar`. Se `VID-1` falhar, `LIV-1` **não** é baixado: a saída mostra `Estoque LIV-1 = 2` depois da falha, e `P-002` não existe no repositório. Baixar item a item, sem a verificação prévia, deixaria o estoque pela metade.
- **Polimorfismo no estoque.** `temSaldo` e `baixar` perguntam `produto.controlaEstoque()`, sem testar o tipo do produto. Um novo tipo "assinatura" só precisa devolver `false` ali.
- **Conferência dos números.** Subtotal = 3 × 80 + 120 + 200 = 560. Frete: livros 0,5 × 3 × 3 = 4,50; vaso (2 × 1 × 3) + 15 = 21,00; total 25,50. Bruto de `P-001` = 585,50. Cartão em 3x: 585,50 × 1,04 = 608,92.
- **Lambda como** `Notificador`**.** Funciona porque a interface tem um único método abstrato. O notificador "de mentira" do `Principal` seria trocado por e-mail em produção, sem alterar o serviço.

Gabarito da Prova Simulada 4: Secretaria Acadêmica

`src/sia/dados/Utilitario.java`

```
package sia.dados;

import java.util.List;

public final class Utilitario {

    private Utilitario() { }

    public static List<String> carregarNotas() {
        return List.of(
            "SC3001;Ana Souza;POO;8.5",
            "SC3001;Ana Souza;POO;7.0",
            "SC3001;Ana Souza;BD;5.0",
            "SC3002;Bruno Lima;POO;4.0",
            "SC3002;Bruno Lima;POO;6.5",
            "SC3002;Bruno Lima;BD;9.0",
            "SC3003;Carla Dias;POO;10.0",
            "SC3003;Carla Dias;BD;abc",
            "SC3004;Davi Reis;POO",
            "SC3003;Carla Dias;BD;11.5",
            "SC3001;Ana Souza;BD;6.0",
            "SC3003;Carla Dias;BD;7.5",
            "",
            "SC3005;Eva Melo;POO;3.0",
            "SC3005;Eva Melo;BD;4.5"
        );
    }
}
```

`src/sia/modelo/Aluno.java`

```
package sia.modelo;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public final class Aluno extends Pessoa implements Avaliavel {
    private final Map<String, List<Double>> notas = new TreeMap<>();

    public Aluno(String nome, String prontuario) {
        super(nome, prontuario);
    }

    public void lancarNota(String disciplina, double nota) {
        if (nota < 0 || nota > 10) {
            throw new IllegalArgumentException("Nota fora do intervalo 0..10: " + nota);
        }
        if (!notas.containsKey(disciplina)) {
            notas.put(disciplina, new ArrayList<>());
        }
        notas.get(disciplina).add(nota);
    }

    public double mediaDisciplina(String disciplina) {
        List<Double> lista = notas.get(disciplina);
        if (lista == null) {
            throw new IllegalArgumentException("Sem notas em " + disciplina);
        }
        double soma = 0;
        for (double n : lista) {
            soma += n;
        }
        return soma / lista.size();
    }

    public List<Double> notasDe(String disciplina) {
        List<Double> lista = notas.get(disciplina);
        return lista == null ? new ArrayList<>() : new ArrayList<>(lista);
    }

    public Iterator<String> disciplinas() {
        return new ArrayList<>(notas.keySet()).iterator();
    }

    @Override
    public String nome() { return getNome(); }

    @Override
    public double media() {
        if (notas.isEmpty()) {
            return 0.0;
        }
        double soma = 0;
        for (String disciplina : notas.keySet()) {
            soma += mediaDisciplina(disciplina);
        }
        return soma / notas.size();
    }
}
```

`src/sia/modelo/Avaliavel.java`

```
package sia.modelo;

public interface Avaliavel {
    String nome();

    double media();
}
```

`src/sia/modelo/Pessoa.java`

```
package sia.modelo;

import java.util.Objects;

public abstract sealed class Pessoa permits Aluno, Professor {
    private final String nome;
    private final String prontuario;

    protected Pessoa(String nome, String prontuario) {
        if (nome == null || nome.isBlank() || prontuario == null || prontuario.isBlank()) {
            throw new IllegalArgumentException("Nome e prontuario sao obrigatorios");
        }
        this.nome = nome;
        this.prontuario = prontuario;
    }

    public String getNome() { return nome; }
    public String getProntuario() { return prontuario; }
```

```
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Pessoa outra)) return false;
        return prontuario.equals(outra.prontuario);
    }

    @Override
    public int hashCode() {
        return Objects.hash(prontuario);
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + " " + nome + " (" + prontuario + ")";
    }
}
```

src/sia/modelo/Professor.java

```
package sia.modelo;

public final class Professor extends Pessoa {
    private final String disciplina;

    public Professor(String nome, String prontuario, String disciplina) {
        super(nome, prontuario);
        this.disciplina = disciplina;
    }

    public String getDisciplina() { return disciplina; }

    public void lancarNota(Aluno aluno, double nota) {
        aluno.lancarNota(disciplina, nota);
    }
}
```

src/sia/modelo/Turma.java

```
package sia.modelo;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class Turma implements Avaliavel {
    public static final double MEDIA_APROVACAO = 6.0;

    private final String nome;
    private final Map<String, Aluno> alunos = new TreeMap<>();

    public Turma(String nome) {
        this.nome = nome;
    }

    @Override
    public String nome() { return nome; }

    public void matricular(Aluno aluno) {
        if (alunos.containsKey(aluno.getProntuario())) {
            throw new IllegalArgumentException("Aluno ja matriculado: " + aluno.getProntuario());
        }
        alunos.put(aluno.getProntuario(), aluno);
    }

    public Aluno buscar(String prontuario) {
        return alunos.get(prontuario);
    }

    public int totalAlunos() {
        return alunos.size();
    }

    @Override
    public double media() {
        return alunos.values().stream().mapToDouble(Aluno::media).average().orElse(0.0);
    }

    public List<Aluno> ranking() {
        List<Aluno> lista = new ArrayList<>(alunos.values());
        lista.sort(Comparator.comparingDouble(Aluno::media).reversed().thenComparing(Aluno::getNome));
        return lista;
    }

    public Map<String, List<Aluno>> porSituacao() {
        Map<String, List<Aluno>> resultado = new LinkedHashMap<>();
        resultado.put("APROVADO", new ArrayList<>());
        resultado.put("REPROVADO", new ArrayList<>());
        for (Aluno a : ranking()) {
            String chave = a.media() >= MEDIA_APROVACAO ? "APROVADO" : "REPROVADO";
            resultado.get(chave).add(a);
        }
        return resultado;
```

```
    }
}
```

src/sia/servico/Boletim.java

```
package sia.servico;

import sia.modelo.Aluno;
import sia.modelo.Turma;

import java.util.Iterator;
import java.util.StringJoiner;

public final class Boletim {

    private Boletim() { }

    public static String emitir(Aluno aluno) {
        StringBuilder sb = new StringBuilder();
        sb.append("========== BOLETIM: ").append(aluno.getNome())
          .append(" (").append(aluno.getProntuario()).append(") ==========\n");
        sb.append(String.format("%-12s %-22s %6s%n", "Disciplina", "Notas", "Media"));
        Iterator<String> it = aluno.disciplinas();
        while (it.hasNext()) {
            String disciplina = it.next();
            StringJoiner notas = new StringJoiner(" ");
            for (double n : aluno.notasDe(disciplina)) {
                notas.add(String.format("%.1f", n));
            }
            sb.append(String.format("%-12s %-22s %6.2f%n", disciplina, notas, aluno.mediaDisciplina(disciplina)));
        }
        sb.append("-----------------------------------------\n");
        String situacao = aluno.media() >= Turma.MEDIA_APROVACAO ? "APROVADO" : "REPROVADO";
        sb.append(String.format("Media geral: %.2f | %s%n", aluno.media(), situacao));
        return sb.toString();
    }
}
```

src/sia/servico/Importador.java

```
package sia.servico;

import sia.modelo.Aluno;
import sia.modelo.Turma;

import java.util.ArrayList;
import java.util.List;

public class Importador {
    private final List<String> rejeitadas = new ArrayList<>();

    public int importar(Turma turma, List<String> linhas) {
        int aceitas = 0;
        int numero = 0;
        for (String linha : linhas) {
            numero++;
            try {
                String[] campos = linha.split(";");
                if (campos.length != 4) {
                    throw new IllegalArgumentException("esperados 4 campos, encontrados " + campos.length);
                }
                double nota = Double.parseDouble(campos[3]);
                Aluno aluno = turma.buscar(campos[0]);
                boolean novo = aluno == null;
                if (novo) {
                    aluno = new Aluno(campos[1], campos[0]);
                }
                aluno.lancarNota(campos[2], nota);
                if (novo) {
                    turma.matricular(aluno);
                }
                aceitas++;
            } catch (NumberFormatException e) {
                rejeitadas.add("Linha " + numero + ": nota nao numerica");
            } catch (IllegalArgumentException e) {
                rejeitadas.add("Linha " + numero + ": " + e.getMessage());
            }
        }
        return aceitas;
    }

    public List<String> getRejeitadas() {
        return new ArrayList<>(rejeitadas);
    }
}
```

src/sia/main/Principal.java

```
package sia.main;

import sia.dados.Utilitario;
import sia.modelo.Aluno;
import sia.modelo.Professor;
import sia.modelo.Turma;
```

```java
import sia.servico.Boletim;
import sia.servico.Importador;

import java.util.List;
import java.util.Map;

public class Principal {
    public static void main(String[] args) {
        Turma turma = new Turma("3A");
        Importador importador = new Importador();
        int aceitas = importador.importar(turma, Utilitario.carregarNotas());

        System.out.println("Linhas aceitas: " + aceitas);
        System.out.println("Alunos matriculados: " + turma.totalAlunos());
        for (String motivo : importador.getRejeitadas()) {
            System.out.println("  rejeitada -> " + motivo);
        }

        Professor prof = new Professor("Lucas", "PF001", "POO");
        Aluno eva = turma.buscar("SC3005");
        prof.lancarNota(eva, 9.0);
        try {
            prof.lancarNota(eva, -1);
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }
        System.out.println("Media de Eva em POO: " + eva.mediaDisciplina("POO"));

        System.out.printf("Media da turma: %.2f%n", turma.media());
        System.out.println("--- Ranking");
        for (Aluno a : turma.ranking()) {
            System.out.printf("%-12s %.2f%n", a.getNome(), a.media());
        }
        System.out.println("--- Situacao");
        for (Map.Entry<String, List<Aluno>> e : turma.porSituacao().entrySet()) {
            StringBuilder nomes = new StringBuilder();
            for (Aluno a : e.getValue()) {
                nomes.append(a.getNome()).append("; ");
            }
            System.out.println(e.getKey() + ": " + nomes);
        }
        System.out.print(Boletim.emitir(turma.buscar("SC3001")));
        try {
            turma.matricular(new Aluno("Outra Ana", "SC3001"));
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Critérios de correção

#	Pts	Distribuição sugerida
1	1,5	`sealed ... permits` correto e subclasses `final` (0,5); validação, `equals`/`hashCode`, `toString` (0,5); `Professor.lancarNota` delegando para a disciplina (0,5).
2	0,5	Interface `Avaliavel`.
3	1,5	Mapa ordenado e `lancarNota` com validação (0,5); `mediaDisciplina`/`notasDe` (0,4); `disciplinas()` por cópia (0,2); `media()` como média das médias, tratando vazio (0,4).
4	1,0	Mapa por prontuário e `matricular` com exceção (0,4); `buscar`/`totalAlunos` (0,2); `media()` com stream (0,3); constante (0,1).
5	1,0	Comparador com média decrescente e desempate por nome, sobre cópia.
6	1,0	Chaves sempre presentes, na ordem, e classificação correta (0,7); listas na ordem do ranking (0,3).
7	2,0	Separação e contagem de campos (0,4); `NumberFormatException` tratada (0,4); `IllegalArgumentException` da nota (0,4); **aluno novo só matriculado após sucesso** (0,5); mensagens e numeração (0,2) e `getRejeitadas` por cópia (0,1).
8	1,0	`StringBuilder` e `StringJoiner` (0,4); formatação (0,4); média geral e situação (0,2).
9	0,5	Roteiro completo.

Comentários

- **Herança controlada.** `sealed` com `permits Aluno, Professor` impede que alguém crie outra subclasse de `Pessoa`. Cada permitida precisa ser `final`, `sealed` ou `non-sealed`. Escolhemos `final`.
- `equals` **por prontuário, na superclasse.** Garante que um prontuário identifica a pessoa independentemente da subclasse (`instanceof Pessoa`).
- **Importador: ordem das operações.** Convertemos a nota **antes** de procurar ou criar o aluno. O aluno novo é criado, mas só entra na turma **depois** de `lancarNota` ter dado certo. Por isso `Davi Reis` (linha com 3 campos) nunca é matriculado, e o resultado são 4 alunos, não 5. Já `Carla Dias` existe porque sua primeira linha era válida. As linhas com `abc` e `11.5` apenas não somam notas.
- **Ordem dos** `catch`**.** Novamente `NumberFormatException` antes de `IllegalArgumentException`, que é sua superclasse. A exceção "campos" e a de "nota fora do intervalo" são ambas `IllegalArgumentException` e caem no segundo `catch`, que usa `e.getMessage()`.

- **Numeração.** O contador `numero++` roda no início do laço, **para toda linha**, inclusive a vazia (linha 13). `"".split(";")` devolve um vetor de **1** elemento, por isso a mensagem "encontrados 1".
- **Conferência.** Carla: POO 10,0 e BD 7,5, média 8,75. Bruno: POO (4,0 + 6,5)/2 = 5,25 e BD 9,0, média (5,25 + 9,0)/2 = 7,125, impressa como `7.13`. Ana: POO 7,75 e BD 5,5, média 6,625, impressa como `6.63`. Eva, após o 9,0 do professor: POO 6,0 e BD 4,5, média 5,25. Média da turma: 27,75 / 4 = 6,9375, impressa como `6.94`.
- **Streams.** `alunos.values().stream().mapToDouble(Aluno::media).average().orElse(0.0)`: `orElse` trata a turma vazia.

Gabarito da Prova Simulada 5: Estação Meteorológica IoT

src/estacao/eventos/Barramento.java

```java
package estacao.eventos;

import estacao.modelo.Leitura;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Barramento {
    public static final String TODOS = "*";

    private final Map<String, List<Ouvinte>> ouvintes = new HashMap<>();
    private int falhasDeOuvintes = 0;

    public void registrar(String tipoSensor, Ouvinte ouvinte) {
        if (!ouvintes.containsKey(tipoSensor)) {
            ouvintes.put(tipoSensor, new ArrayList<>());
        }
        ouvintes.get(tipoSensor).add(ouvinte);
    }

    public void registrarParaTodos(Ouvinte ouvinte) {
        registrar(TODOS, ouvinte);
    }

    public int publicar(Leitura<Double> leitura) {
        List<Ouvinte> destinatarios = new ArrayList<>();
        if (ouvintes.containsKey(TODOS)) {
            destinatarios.addAll(ouvintes.get(TODOS));
        }
        if (ouvintes.containsKey(leitura.getTipo())) {
            destinatarios.addAll(ouvintes.get(leitura.getTipo()));
        }
        int notificados = 0;
        for (Ouvinte o : destinatarios) {
            try {
                o.aoReceber(leitura);
                notificados++;
            } catch (RuntimeException e) {
                falhasDeOuvintes++;
            }
        }
        return notificados;
    }

    public int getFalhasDeOuvintes() {
        return falhasDeOuvintes;
    }
}
```

src/estacao/eventos/Ouvinte.java

```java
package estacao.eventos;

import estacao.modelo.Leitura;

@FunctionalInterface
public interface Ouvinte {
    void aoReceber(Leitura<Double> leitura);
}
```

src/estacao/modelo/Estacao.java

```java
package estacao.modelo;

import estacao.eventos.Barramento;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class Estacao {
    private static final int TAMANHO_JANELA = 3;
```

```
    private final Map<String, Sensor> sensores = new TreeMap<>();
    private final Map<String, Janela<Double>> janelas = new HashMap<>();
    private final Map<String, List<Leitura<Double>>> historico = new HashMap<>();
    private final List<String> ocorrencias = new ArrayList<>();
    private final Barramento barramento;
    private int instante = 0;
    private int leiturasInvalidas = 0;

    public Estacao(Barramento barramento) {
        this.barramento = barramento;
    }

    public void registrar(Sensor sensor) {
        if (sensores.containsKey(sensor.id())) {
            throw new IllegalArgumentException("Sensor repetido: " + sensor.id());
        }
        sensores.put(sensor.id(), sensor);
        janelas.put(sensor.id(), new Janela<>(TAMANHO_JANELA));
        historico.put(sensor.id(), new ArrayList<>());
    }

    public int ciclo() {
        instante++;
        int validas = 0;
        for (Sensor sensor : sensores.values()) {
            try {
                Leitura<Double> leitura = sensor.coletar(instante);
                janelas.get(sensor.id()).adicionar(leitura.getValor());
                historico.get(sensor.id()).add(leitura);
                barramento.publicar(leitura);
                validas++;
            } catch (LeituraInvalidaException e) {
                leiturasInvalidas++;
                ocorrencias.add("t" + instante + ": " + e.getMessage());
            }
        }
        return validas;
    }

    public double mediaMovel(String idSensor) {
        Janela<Double> janela = janelas.get(idSensor);
        if (janela == null) {
            throw new IllegalArgumentException("Sensor desconhecido: " + idSensor);
        }
        return janela.media();
    }

    public Leitura<Double> maiorLeitura(String idSensor) {
        List<Leitura<Double>> lista = historico.get(idSensor);
        if (lista == null) {
            throw new IllegalArgumentException("Sensor desconhecido: " + idSensor);
        }
        return Estatisticas.maximo(lista);
    }

    public Map<String, Double> mediasGerais() {
        Map<String, Double> medias = new TreeMap<>();
        for (String id : sensores.keySet()) {
            double media = historico.get(id).stream()
                    .mapToDouble(Leitura::getValor)
                    .average()
                    .orElse(0.0);
            medias.put(id, media);
        }
        return medias;
    }

    public int getLeiturasInvalidas() { return leiturasInvalidas; }

    public List<String> getOcorrencias() { return new ArrayList<>(ocorrencias); }
}
```

src/estacao/modelo/Estatisticas.java

```
package estacao.modelo;

import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
import java.util.NoSuchElementException;

public final class Estatisticas {

    private Estatisticas() { }

    public static <T extends Comparable<T>> T maximo(Collection<T> itens) {
        if (itens.isEmpty()) {
            throw new NoSuchElementException("Colecao vazia");
        }
        T maior = null;
        for (T item : itens) {
            if (maior == null || item.compareTo(maior) > 0) {
                maior = item;
            }
        }
        return maior;
```

```
    }

    public static <T extends Comparable<T>> List<T> ordenar(Collection<T> itens) {
        List<T> copia = new ArrayList<>(itens);
        Collections.sort(copia);
        return copia;
    }
}
```

src/estacao/modelo/Janela.java

```
package estacao.modelo;

import java.util.ArrayList;
import java.util.List;

public class Janela<T extends Number> {
    private final int capacidade;
    private final List<T> valores = new ArrayList<>();

    public Janela(int capacidade) {
        if (capacidade <= 0) {
            throw new IllegalArgumentException("Capacidade deve ser positiva: " + capacidade);
        }
        this.capacidade = capacidade;
    }

    public void adicionar(T valor) {
        valores.add(valor);
        if (valores.size() > capacidade) {
            valores.remove(0);
        }
    }

    public int tamanho() {
        return valores.size();
    }

    public double media() {
        if (valores.isEmpty()) {
            return 0.0;
        }
        double soma = 0;
        for (T v : valores) {
            soma += v.doubleValue();
        }
        return soma / valores.size();
    }
}
```

src/estacao/modelo/Leitura.java

```
package estacao.modelo;

public class Leitura<T extends Comparable<T>> implements Comparable<Leitura<T>> {
    private final String sensorId;
    private final String tipo;
    private final T valor;
    private final int instante;

    public Leitura(String sensorId, String tipo, T valor, int instante) {
        this.sensorId = sensorId;
        this.tipo = tipo;
        this.valor = valor;
        this.instante = instante;
    }

    public String getSensorId() { return sensorId; }
    public String getTipo() { return tipo; }
    public T getValor() { return valor; }
    public int getInstante() { return instante; }

    @Override
    public int compareTo(Leitura<T> outra) {
        int cmp = valor.compareTo(outra.valor);
        return cmp != 0 ? cmp : Integer.compare(instante, outra.instante);
    }

    @Override
    public String toString() {
        return tipo + "[" + sensorId + "]=" + valor + " @t" + instante;
    }
}
```

src/estacao/modelo/LeituraInvalidaException.java

```
package estacao.modelo;

public class LeituraInvalidaException extends RuntimeException {
    public LeituraInvalidaException(String idSensor, double valor) {
        super("Leitura invalida de " + idSensor + ": " + valor);
    }
}
```

`src/estacao/modelo/Sensor.java`

```
package estacao.modelo;

public abstract class Sensor {
    private final String id;
    private final double[] amostras;
    private int proxima = 0;

    protected Sensor(String id, double... amostras) {
        if (amostras.length == 0) {
            throw new IllegalArgumentException("Sensor sem amostras: " + id);
        }
        this.id = id;
        this.amostras = amostras.clone();
    }

    protected abstract double minimo();

    protected abstract double maximo();

    public abstract String unidade();

    public String id() { return id; }

    public String tipo() { return getClass().getSimpleName(); }

    public final Leitura<Double> coletar(int instante) {
        double valor = amostras[proxima % amostras.length];
        proxima++;
        if (valor < minimo() || valor > maximo()) {
            throw new LeituraInvalidaException(id, valor);
        }
        return new Leitura<>(id, tipo(), valor, instante);
    }
}
```

`src/estacao/modelo/SensorFactory.java`

```
package estacao.modelo;

public final class SensorFactory {

    private SensorFactory() { }

    public static Sensor temperatura(String id, double... amostras) {
        return new SensorTemperatura(id, amostras);
    }

    public static Sensor umidade(String id, double... amostras) {
        return new SensorUmidade(id, amostras);
    }
}
```

`src/estacao/modelo/SensorTemperatura.java`

```
package estacao.modelo;

final class SensorTemperatura extends Sensor {

    SensorTemperatura(String id, double... amostras) {
        super(id, amostras);
    }

    @Override protected double minimo() { return -50.0; }
    @Override protected double maximo() { return 60.0; }
    @Override public String unidade() { return "C"; }
}
```

`src/estacao/modelo/SensorUmidade.java`

```
package estacao.modelo;

final class SensorUmidade extends Sensor {

    SensorUmidade(String id, double... amostras) {
        super(id, amostras);
    }

    @Override protected double minimo() { return 0.0; }
    @Override protected double maximo() { return 100.0; }
    @Override public String unidade() { return "%"; }
}
```

`src/estacao/main/Principal.java`

```
package estacao.main;

import estacao.eventos.Barramento;
import estacao.eventos.Ouvinte;
import estacao.modelo.Estacao;
```

```java
import estacao.modelo.Leitura;
import estacao.modelo.SensorFactory;

import java.util.Map;
import java.util.TreeMap;

public class Principal {
    private static final int CICLOS = 6;

    public static void main(String[] args) {
        Barramento barramento = new Barramento();
        Estacao estacao = new Estacao(barramento);
        estacao.registrar(SensorFactory.temperatura("T1", 22.0, 25.5, 31.0, 36.5, 99.0, 28.0));
        estacao.registrar(SensorFactory.umidade("U1", 40, 55, 70, 105, 60, 45));

        barramento.registrarParaTodos(l -> System.out.println("LOG    " + l));
        barramento.registrar("SensorTemperatura", l -> {
            if (l.getValor() > 35.0) {
                System.out.println("ALERTA calor: " + l.getValor());
            }
        });
        barramento.registrar("SensorTemperatura", l -> {
            if (l.getValor() == 31.0) {
                throw new IllegalStateException("ouvinte com defeito");
            }
        });
        final Map<String, Integer> contagem = new TreeMap<>();
        barramento.registrar("SensorUmidade", new Ouvinte() {
            @Override
            public void aoReceber(Leitura<Double> leitura) {
                contagem.put(leitura.getSensorId(), contagem.getOrDefault(leitura.getSensorId(), 0) + 1);
            }
        });

        for (int i = 0; i < CICLOS; i++) {
            int validas = estacao.ciclo();
            System.out.println("-- ciclo " + (i + 1) + ": " + validas + " leitura(s) valida(s)");
        }

        System.out.printf("Media movel T1 = %.2f%n", estacao.mediaMovel("T1"));
        System.out.printf("Media movel U1 = %.2f%n", estacao.mediaMovel("U1"));
        System.out.println("Maior T1 = " + estacao.maiorLeitura("T1"));
        System.out.println("Maior U1 = " + estacao.maiorLeitura("U1"));
        for (Map.Entry<String, Double> e : estacao.mediasGerais().entrySet()) {
            System.out.printf("Media geral %s = %.2f%n", e.getKey(), e.getValue());
        }
        System.out.println("Invalidas = " + estacao.getLeiturasInvalidas() + " " + estacao.getOcorrencias());
        System.out.println("Falhas de ouvintes = " + barramento.getFalhasDeOuvintes());
        System.out.println("Contagem umidade = " + contagem);
        try {
            estacao.mediaMovel("X9");
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Critérios de correção

#	Pts	Distribuição sugerida
1	1,0	Genérica com limite `Comparable<T>` (0,4); `compareTo` valor/instante (0,4); `toString` (0,2).
2	1,25	Exceção **não verificada** (0,15); construtor com varargs e cópia (0,3); abstratos e `tipo()` (0,2); `coletar` **final** com ponteiro circular que avança mesmo na amostra inválida (0,6).
3	1,0	Duas subclasses `final` com visibilidade e construtores de pacote (0,5); fábrica pública com varargs devolvendo `Sensor` (0,5).
4	1,25	Limite `T extends Number` (0,4); descarte do mais antigo (0,4); `media()` com `doubleValue()` e caso vazio (0,3); validação (0,15).
5	0,75	Dois métodos genéricos corretos, com exceção para coleção vazia e classe não instanciável.
6	1,5	Interface funcional (0,2); registro por tipo e por `"*"` (0,3); ordem `"*"` e depois tipo (0,3); retorno correto (0,3); **isolamento de falhas** com contador (0,4).
7	1,5	Injeção do barramento (0,2); mapa ordenado, janela e histórico (0,3); `ciclo` com tratamento da exceção (0,6); ocorrências e retorno (0,3); sem impressão (0,1).
8	1,0	`mediaMovel`/`maiorLeitura` com validação (0,4); `mediasGerais` com streams e `TreeMap` (0,4); getters por cópia (0,2).
9	0,75	Fábrica usada; lambdas e **classe anônima**; roteiro completo.

Comentários

- **Genéricos com limite.** `Leitura<T extends Comparable<T>>` permite `valor.compareTo(...)` sem *cast*. Já `Janela<T extends Number>` permite `doubleValue()`. São os "limites superiores" da Aula 06. Os métodos de `Estatisticas` têm limites parecidos, e `maximo` serve para qualquer coisa comparável: `Estacao` o usa com `Leitura<Double>`, que implementa `Comparable<Leitura<Double>>`.
- **Método-modelo (`final`).** `coletar` define o **algoritmo** (pegar amostra, validar, montar leitura) e delega as partes variáveis (`minimo`, `maximo`) às subclasses. Sendo `final`, nenhuma subclasse pode alterá-lo, e todas as validações são

idênticas.

- **`proxima++` antes de validar.** A amostra inválida é "consumida". Sem isso, um sensor com valor ruim ficaria preso nele para sempre. É isso que produz `t4` inválido para `U1` (105) e `t5` inválido para `T1` (99).
- **Construtores de pacote e fábrica.** `SensorTemperatura` e `SensorUmidade` são `final` e **sem `public`**: só `SensorFactory` (mesmo pacote) as constrói, e o `Principal` (outro pacote) só enxerga `Sensor`. Se quiser um novo tipo de sensor, a fábrica é o único ponto a alterar. Em `umidade("U1", 40, 55, ...)`, os inteiros são ampliados automaticamente para `double` nos varargs.
- **Isolamento de falhas no barramento.** O `catch (RuntimeException)` dentro do laço faz o ouvinte "com defeito" (que lança no valor 31,0) ser contado em `getFalhasDeOuvintes()` sem impedir o log, o alarme nem o contador de receberem a mesma leitura.
- **Janela móvel.** `T1` só tem 5 leituras válidas (22,0; 25,5; 31,0; 36,5; 28,0), mas a janela guarda as **3 últimas** (31,0; 36,5; 28,0), com média 31,83. Para `U1`, as válidas são 40, 55, 70, 60 e 45, e a janela guarda 70, 60 e 45, com média 58,33.
- **Conferência das médias gerais.** `T1`: (22 + 25,5 + 31 + 36,5 + 28) / 5 = 28,60. `U1`: (40 + 55 + 70 + 60 + 45) / 5 = 54,00. A contagem do ouvinte de umidade é 5, uma por leitura válida.
- **Estação independente do barramento concreto.** `Estacao` só chama `publicar`; ela não sabe quem escuta. É a mesma ideia de inversão de dependência da Aula 06, aplicada a eventos.

Parte V: Síntese dos conteúdos mais importantes para revisão

Esta síntese reúne o que se repetiu nas duas provas de referência e nos exercícios e simulados. Os itens estão em ordem aproximada de frequência.

1. Herança

- `extends` cria a relação "é um". A subclasse inicia o objeto pelo construtor da superclasse. Se a superclasse não tem construtor sem parâmetros, o `super(...)` explícito é **obrigatório**.
- `@Override` com mesma assinatura. O nível de acesso nunca pode ficar **mais restrito**. Métodos `static` e `private` não são sobrescritos. `super.metodo()` reaproveita a versão do pai.
- `final` em classe impede herança; em método impede sobrescrita. `sealed ... permits` limita **quem** herda (subclasses `final`, `sealed` ou `non-sealed`).
- **Classe abstrata** não se instancia e pode ter métodos abstratos (sem corpo). Uma classe com método abstrato tem de ser abstrata.
- **Acesso:** `private` só a classe; sem modificador, o pacote; `protected`, pacote e subclasses; `public`, todos. Classes de implementação interna costumam ser **de pacote**.
- **`equals`/`hashCode`.** *Entidade* é igual pela chave (id, sku, prontuário). *Objeto de valor* é igual pelo estado todo. Sempre sobrescreva **os dois**. `Objects.equals` e `Objects.hash` ajudam. Com hierarquia em que a identidade vale para todas as subclasses, use `instanceof SuperClasse`, **não** `getClass()`.

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Produto outro)) return false;   // vale para toda a hierarquia
    return Objects.equals(sku, outro.sku);
}
@Override public int hashCode() { return Objects.hash(sku); }
```

2. Interfaces e polimorfismo

- **Interface é contrato**: métodos abstratos (públicos por padrão), constantes, `default` e `static`. Uma classe herda **uma** classe, mas implementa **várias** interfaces. Interface não tem estado e não se instancia.
- **Polimorfismo dinâmico:** o método executado depende da **classe real** do objeto, não do tipo da variável. Use a **interface como tipo** de variáveis, parâmetros, retornos, vetores e coleções.
- `instanceof` com padrão (`if (x instanceof Mago m)`) faz o teste e a conversão. *Cast* direto sem teste pode lançar `ClassCastException`.
- **Inversão e injeção de dependência:** a regra de negócio depende de **abstrações**; as implementações concretas são injetadas no construtor (`ServicoPedido`, `Estacao`, `Pedido` com `RegraFrete`). **Fábricas** devolvem a interface.
- **Genéricos:** `Classe<T>`, limites `T extends Tipo`, métodos genéricos (`<T extends X> T metodo(...)`). Primitivos exigem *wrappers*. Evite **raw types**.
- **Interface funcional** (um só método abstrato) pode ser implementada por **lambda** ou classe anônima. A classe anônima pode ter estado; a lambda só captura variáveis *efetivamente finais*. Referências a métodos: `Aluno::media`, `String::toUpperCase`.
- **Método-modelo:** o algoritmo fica num método `final` na classe base, e as variações são métodos abstratos nas subclasses.

3. Coleções

Estrutura	Característica	Quando usar
`List` (`ArrayList`)	admite repetidos, mantém posição	sequências, ordenar, índice
`Set` (`HashSet`, `LinkedHashSet`, `TreeSet`)	**sem repetidos** (por `equals`/`hashCode`, ou `compareTo` no `Tree`)	unicidade; `LinkedHashSet` preserva a inserção; `TreeSet` ordena
`Map` (`HashMap`, `LinkedHashMap`, `TreeMap`)	chave **única** → valor	busca rápida por chave; `LinkedHashMap` preserva a ordem; `TreeMap` ordena as chaves

- **Não conte com ordem** de `HashSet`/`HashMap`.
- **Objetos como chave ou elemento de `Set`** exigem `equals`/`hashCode` corretos.
- **Ordenação:** `Comparable` (ordem natural, dentro da classe) ou `Comparator` (externo, vários critérios). `compareTo`: negativo (this < outro), 0, positivo. Para ordem decrescente, inverta os operandos de `Integer.compare`.
- **Comparadores compostos:** `Comparator.comparingInt(...).reversed().thenComparing(...)`. Às vezes o lambda precisa de tipo explícito: `(T t) -> t.pontos()`.
- **Remover durante a iteração:** use `Iterator.remove()`. Remover com `for-each` lança `ConcurrentModificationException`.
- **Encapsulamento:** nada de `getLista()` que devolva a coleção interna. Ofereça métodos de inserção/remoção, um

`Iterator` ou uma **cópia** (`new ArrayList<>(...)`). Cuidado com `subList` (é uma visão).

- **Mapas de listas:** `get` devolve `null` na primeira vez; crie a lista sob demanda. `getOrDefault` evita `null` em contadores.
- **Streams:** `filter`, `map`, `mapToInt`/`mapToDouble`, `sum`, `average().orElse(0)`, `toList()`. `List.of` é **imutável**.
- **Varargs:** `tipo... nome`, único e último parâmetro; dentro do método é um vetor.

4. Exceções (peso menor)

- `try` / `catch` / `finally` (o `finally` roda sempre). `throw` lança; `throws` declara.
- **Verificada** (`extends Exception`): o compilador obriga a tratar ou declarar. **Não verificada** (`extends RuntimeException`): erros de uso (`IllegalArgumentException`, `NullPointerException`, `NumberFormatException`...).
- Ordem dos `catch`: **subclasse antes da superclasse** (`NumberFormatException` antes de `IllegalArgumentException`).
- Valide argumentos no início e lance a exceção adequada com mensagem útil. Em operações compostas, valide **tudo antes** de alterar qualquer coisa (atomicidade).

5. Estratégia para as provas

1. **Leia a tabela inteira antes de codar** e siga a ordem dos itens: eles são a ordem natural de construção.
2. **Respeite nomes, assinaturas, formatos de saída e mensagens** dos enunciados. Os quadros mostram o formato exato.
3. **Não imprima nas classes de domínio**; retorne `String` e deixe `Principal` imprimir.
4. **Pergunte-se sempre:** isso precisa de `equals`/`hashCode`? Estou expondo uma coleção? O comportamento depende do tipo (então é método polimórfico)? A classe deveria ser `final`, abstrata ou ter construtor de pacote?
5. **Rode seu `Principal`** e compare com os quadros. Confira casos de borda: coleção vazia, id repetido, valor inválido, limite exato (≥ × >).
6. **Exemplos de penalidade** nas provas de referência: método ineficiente (usar lista onde se pedia mapa, por exemplo) e não seguir as instruções de entrega do projeto.

6. Mapa de quem treina o quê

Atividade	Foco principal
Ex. 1, 2	herança básica; interface e polimorfismo
Ex. 3, 4	`List`, `Comparable`, `Iterator`; exceções
Ex. 5, 6	`equals`/`hashCode` e `Map`; genéricos e `Set`
Ex. 7, 8	interface funcional e DIP; herança + interface + `Map`
Ex. 9, 10	eventos e lambdas; integração total
Sim. 1	herança + interface + exceção verificada + `Map`
Sim. 2	`default`, `Set`, varargs, genéricos, streams
Sim. 3	DIP, fábrica, repositório genérico, atomicidade
Sim. 4	`sealed`, importação com tratamento de exceções, `Map`/`Comparator`
Sim. 5	genéricos com limite, método-modelo, construtores de pacote, eventos